

UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS
INGENIERÍA DE SISTEMAS



Tesis de grado

Predicción de enlaces en redes sociales con enfoque de aprendizaje

Director: Dr. Marcelo G. Armentano

Co-director: Dr. Ariel J. Monteserin

Autores: Luis María Coussirat
Emilio Bongiorno

Resumen

El objetivo del presente trabajo es el estudio de las técnicas de aprendizaje, en este caso supervisado, para la predicción de enlaces inexistentes u ocultos, entre usuarios de una misma red social.

Las redes pueden representarse mediante grafos, en los cuáles los nodos corresponden a usuarios, y las aristas a las relaciones entre ellos. Desde el punto de vista del aprendizaje supervisado, es necesario crear datasets que contengan toda la información de los grafos. Las instancias de los mismos corresponden a cada par de nodos, y los atributos a los valores de similitud entre ellos a partir de distintas funciones. La clase de las instancias está dada por la existencia o no de enlace entre los nodos, por lo tanto, la predicción de enlaces pasa a ser un problema de clasificación binaria.

Se utilizó la herramienta Gephi como base sobre la cual implementar los requerimientos necesarios para realizar la experimentación. La misma cuenta con utilidades para manipulación de grafos y el cálculo de distintas funciones de similitud entre pares de nodos. Adicionalmente, fue necesario agregar funcionalidades para la creación y clasificación de datasets a partir de grafos, y se agregaron un conjunto de técnicas para mejorar y flexibilizar el proceso de clasificación: filtrado de instancias, selección de atributos y balanceo de datasets.

Para la experimentación se utilizaron cuatro redes con características muy diferentes, ya sea en cuanto a tamaño o estructura intrínseca. Tres de las redes poseen tamaños de pequeño a mediano, y la cuarta y más importante, corresponde a un escaneo de una porción de Twitter y, por ende, de tamaño considerable. A su vez es la red más desconectada de las cuatro.

Debido a la gran cantidad de combinaciones posibles dadas por las redes, técnicas y clasificadores a probar, la experimentación se realizó por etapas. Dado que las funciones de similitud individuales son útiles por sí mismas para la predicción de enlaces, aunque con un funcionamiento muy diferente, el objetivo de la última etapa fue la comparación del enfoque de aprendizaje supervisado contra las funciones, cotejando la calidad de ambos tipos de modelos predictores.

Se obtuvieron variadas conclusiones. El filtrado de instancias por vecinos comunes mejora en todos los casos a la clasificación. La selección de atributos fue beneficiosa en el caso de las redes pequeñas y medianas. El balance beneficia especialmente al dataset de Twitter donde el desbalance extremo impedía clasificaciones significativamente útiles. Finalmente, el enfoque de aprendizaje posee mayor calidad y flexibilidad que las funciones individuales, sin embargo, se necesita mayor cantidad de pruebas y de configuración. En redes grandes, el enfoque de funciones se vuelve inútil debido a su baja flexibilidad y rendimiento.

Índice

1. Introducción	4
1.1. Motivación	4
1.2. Propuesta	6
1.3. Organización	8
2. Marco conceptual y teórico	9
2.1. Marco Teórico	9
2.1.1. Redes sociales y grafos	9
2.1.2. Link Prediction	11
2.1.3. Aprendizaje automático	12
2.1.4. Funciones de similitud	14
2.1.5. Datasets	19
2.1.6. Clasificadores	22
2.1.7. Métricas	30
2.2. Trabajos relacionados	35
3. Desarrollo de la propuesta	41
3.1. Análisis de las herramientas	41
3.2. Creación de datasets	42
3.3. Integración de clasificadores	44
3.4. Balanceo de datasets	46
3.5. Selección de atributos	47
3.6. Comparación con las métricas de similitud	48
3.7. Proceso de experimentación	49
4. Diseño e implementación	53
4.1. Restricciones y requerimientos	53
4.2. Diseño	55
4.3. Implementación	59
4.3.1. Frontend	60
4.3.2. Backend	67
5. Experimentación	71
5.1. Casos de estudio	71
5.1.1. Les Misérables	71
5.1.2. Jazz Musicians	72
5.1.3. Contactos Empresa	73
5.1.4. Small Twitter	75
5.2. Resultados y observaciones	78
5.2.1. Pre-procesamiento	78

5.2.2. Selección de atributos	83
5.2.3. Balanceo	91
5.2.4. Aprendizaje supervisado vs. métricas de similitud individuales	99
6. Conclusiones	108
6.1. Aportes de este trabajo	108
6.2. Trabajos futuros	110
7. Referencias	112

1. Introducción

En esta primera sección se explica en forma general el contexto de la temática del trabajo, la motivación que provoca el interés en desarrollar el mismo, y la propuesta que se planteó para encarar de forma exitosa la presente investigación.

1.1. Motivación

Este trabajo surge a raíz del interés de estudiar el comportamiento y evolución de las redes sociales a través del tiempo.

La popularidad de estas redes ha ido aumentando cada vez más en los últimos años, como consecuencia del auge de los contenidos creados por los usuarios y la, cada vez más fácil, interacción a través de internet mediante múltiples tipos de dispositivos. Esto abrió la posibilidad a los usuarios de compartir información de cualquier tipo, desde cualquier dispositivo y lugar.

Todo este universo de posibilidades dio origen a distintos tipos de redes sociales, que brindan diferentes características para la interacción entre los usuarios.

Por ejemplo, existen redes sociales para compartir contenido multimedia, como es el caso de Youtube, donde los usuarios pueden suscribirse a los llamados “canales” de los demás usuarios, y poder así seguir los videos que publican. Similar es el funcionamiento de Twitter, aunque en este caso, la información que se comparte es mayormente de tipo textual y permitiendo enlazar contenidos de otros sitios. Para contrastar, en el caso de Facebook, los usuarios pueden subir diversos tipos de contenidos (imágenes, videos, texto) y relacionarse mutuamente entre ellos mediante la opción de “ser amigos”, aunque a su vez, es posible sólo seguir el contenido de otros usuarios sin necesidad de ser amigos de ellos. En un enfoque orientado a lo profesional, se puede hacer referencia a LinkedIn, cuyo fin es conectar personas que buscan relacionarse desde el punto de vista laboral, ya sea buscando u ofreciendo empleo. Y de esta forma se pueden mencionar infinitudes de variantes entre las distintas redes sociales.

Esta clase de información tiene una importancia subjetiva para los usuarios, debido a que, cada quién se relaciona con aquellos usuarios que posean o provean contenido relevante para ellos. Pero a su vez, estas relaciones que se crean debido a la interacción entre usuarios, pueden ser analizadas a un nivel de abstracción mayor para obtener nueva información derivada como, por ejemplo, patrones estructurales en las redes, tendencias en los nuevos contenidos, entre otros. Esta información puede ser utilizada por entidades que buscan algún beneficio a partir de los datos de una red social, sin que los usuarios participantes de la misma sean conscientes de ello. A modo de ejemplo, si los usuarios comienzan a compartir contenido de la final de un torneo de fútbol importante, los encargados de la red

detectarán la tendencia emergente, y obtendrán mayores beneficios vendiendo su espacio de publicidad a marcas de indumentaria deportiva.

La importancia de las redes sociales es tal, que han surgido investigaciones desde distintos ámbitos estudiando su evolución, dinámica, estructura global, entre otras características que puedan ser de utilidad. Algunas investigaciones son enfocadas en sacar beneficios económicos, por ejemplo, buscando el aumento de los usuarios de una red o de las relaciones entre ellos para intentar vender algún producto o servicio. Pero también existen otros enfoques más teóricos, que resultan útiles al aplicarse de forma práctica en distintos campos de investigación.

Este trabajo se enfoca en el problema de predecir futuras interacciones entre los usuarios de una misma red social utilizando un enfoque de aprendizaje supervisado. Este problema es conocido como predicción de enlaces (“Link Prediction”, en inglés) y consiste en predecir con un buen índice de acierto futuras relaciones que se generen entre los usuarios a partir de la información que se posee de la red en un momento dado.

El enfoque de aprendizaje supervisado implica que la información de los usuarios y sus relaciones son estructuradas en diversos modelos de datos a modo de entrenamiento. Luego, estos modelos se utilizan para clasificar otras relaciones nuevas que aparezcan y determinar si existe enlace o no entre los usuarios de las mismas. Por ser una técnica supervisada, la clase de los datos usados para entrenamiento es conocida de antemano. En este caso particular, la información entre dos usuarios de una red social se clasificará en dos tipos, dependiendo de si existe interacción entre ellos o no.

Los modelos de datos anteriormente mencionados son conocidos como clasificadores. A partir de un conjunto de ejemplos que poseen determinadas características y clase conocidas, los clasificadores entrenan sus modelos internos. Los mismos se van modificando y ajustando a los ejemplos de entrenamiento que se les proveen, y se estructuran de diferente manera dependiendo del clasificador que se utilice. Una vez entrenados, los clasificadores pueden determinar la clase de nuevos ejemplos a partir de sus características intrínsecas.

La técnica de predicción de enlaces es útil en varios campos de aplicación. Por ejemplo, dado el seguimiento de las actividades, intereses y relaciones de un usuario, es posible recomendar otros usuarios que posean intereses semejantes, ofrecer publicidad de ciertos productos, etc. En el área de la medicina, se puede predecir la forma en que interaccionan distintas proteínas con el fin de desarrollar medicamentos o tratamientos nuevos. También es útil para estudiar grupos de personas específicas como, por ejemplo, buscar relaciones no conocidas entre integrantes de una banda criminal, y de esa manera desbaratar sus operaciones, lo cual mejorará la seguridad para los ciudadanos en general. Estos son sólo algunos ejemplos de una gran variedad de aplicaciones que posee la técnica de predicción de enlaces, y que reflejan la importancia de su estudio.

Debido a la gran diversidad de clasificadores existentes, y la amplia variedad de redes con diferentes características, resulta interesante poder contar con una herramienta que permita modelar cualquier red, y analizar el funcionamiento de la

predicción de enlaces a partir de los clasificadores existentes. De esta forma, los investigadores interesados en el estudio de la predicción de enlaces a través del aprendizaje supervisado, tendrían la posibilidad de realizar análisis sobre todas las redes que deseen investigar, en una misma herramienta integradora.

A su vez, resulta importante poder comparar el funcionamiento de las predicciones utilizando, por un lado, cada una de las funciones de similitud individuales, y, por otro lado, el enfoque de aprendizaje supervisado que permite condensar la información de varias funciones a la vez. Al utilizar un enfoque de aprendizaje supervisado, se espera conseguir, en líneas generales, mejores resultados que otros enfoques con los cuáles se ha estudiado esta técnica, ya que se le provee de antemano el conocimiento de que entre dos usuarios puede o no haber relación entre ellos para poder clasificarlos.

Con este estudio se busca también interpretar qué características pueden representar una red social, qué técnicas de clasificación permiten obtener buenos resultados a la hora de predecir enlaces, y cuáles son los factores que determinan dichos resultados para los distintos tipos de redes.

1.2. Propuesta

El trabajo propuesto consiste en desarrollar una herramienta que modele diferentes tipos de redes, sobre la cual se pueda obtener la información necesaria para generar los conjuntos de datos útiles en la predicción de enlaces. Para realizar esto, se necesita que la red pueda ser modelada en forma de grafo, permitiendo así poder calcular ciertas características necesarias para el aprendizaje supervisado.

En una red social que se representa con un grafo, cada nodo corresponde con un usuario de la red, y las aristas representan las relaciones entre ellos. Con el fin de analizar redes representadas mediante grafos, se utilizará una herramienta de código libre llamada Gephi ¹, que brinda todas las funcionalidades básicas de modelado de grafos, sobre la cual es posible analizar diferentes representaciones de redes, con sus respectivas topologías e información.

A su vez, se cuenta con una funcionalidad adicional desarrollada en el trabajo de tesis titulado “Predicción de enlaces en redes sociales modeladas por grafos” [Ricotta y Santamarina, 2016] sobre la misma plataforma. Esta funcionalidad permite calcular diferentes grados de similitud entre cada par de nodos, según diversas características basadas en la topología del grafo o en la semejanza entre los atributos propios de cada nodo.

Se propone extender la funcionalidad de la herramienta para crear los datasets necesarios a partir de las funciones de similitud entre pares de nodos, sabiendo de antemano si existe o no relación entre ellos. Además, se pretende integrar clasificadores que puedan utilizar estos datasets para entrenar sus modelos de datos, y posteriormente realizar una evaluación de los datasets sobre dichos

¹ Sitio web de la herramienta Gephi. <https://gephi.org/>.

modelos, obteniendo las métricas que caracterizan el rendimiento del algoritmo. También se propone incluir funcionalidades adicionales para filtrar instancias, balancear y seleccionar atributos de los datasets, lo cual permite variar las distintas etapas del proceso de clasificación.

Para facilitar la implementación de todo lo relacionado a la clasificación, se propone integrar dentro de Gephi la plataforma de software llamada Weka ², también de código abierto, que brinda funcionalidades muy completas con todo lo relacionado al aprendizaje automático. De esta forma, es posible utilizar los datasets generados, entrenar los clasificadores a partir de ellos, y mostrar toda la información relevante brindada por Weka, desde la herramienta Gephi.

A partir del desarrollo de la herramienta, se llevarán a cabo diferentes experimentos, acompañados de un análisis de los resultados obtenidos, para poder corroborar y analizar el funcionamiento de los módulos implementados. Con el fin de evitar que el estudio quede sesgado por las características de una red específica, se contará con varias redes sociales con características diferentes en cuanto a tamaño, topología, dominio de aplicación, entre otras.

Se realizará un estudio individual sobre cada red, analizando la mayor cantidad posible de características que sean de interés, dado el enfoque de aprendizaje supervisado. Se estudiarán distintas formas de calcular las características de las relaciones entre usuarios, cuáles son las más representativas, por qué, y cómo disminuir el ruido en la creación de los datasets. Una vez finalizadas las pruebas anteriormente mencionadas, se introducirán técnicas que modifiquen la creación y clasificación de los datasets, utilizando las funcionalidades adicionales implementadas. Con esto se analizará la manera en que tales técnicas influyen en la clasificación sobre las redes, ya sea de manera positiva o negativa. Finalmente, se pretende realizar una comparación contra las predicciones de las funciones de similitud individuales, esperando obtener mejores resultados mediante la utilización de clasificadores.

Para la comparación entre distintos experimentos, cualesquiera sean estos, se utilizarán un conjunto de métricas que permiten realizar una comparación objetiva y sin ningún tipo de sesgo. Estas métricas son medidas del rendimiento general de una clasificación sobre un dataset, calculado del grafo que representa una red. Por esta causa se realizarán varios experimentos, buscando mejorar los valores de las métricas finales de cada uno, y así, obtener la mejor forma de clasificar las redes.

Una vez finalizada la etapa de experimentación, se procederá a realizar las conclusiones pertinentes considerando los resultados de los experimentos.

² Sitio web de la herramienta Weka. <http://www.cs.waikato.ac.nz/ml/weka/>.

1.3. Organización

A continuación, se presenta un resumen del contenido de las secciones del presente documento de tesis.

En forma introductoria, la sección 2 se encarga de analizar todo el marco teórico, correspondiente a la predicción de enlaces en redes sociales y al aprendizaje automático. Se realiza una extensa explicación de cada uno de los conceptos fundamentales, necesarios para entender cada uno de los componentes que dan forma al trabajo desarrollado. Además, se resumen varios trabajos relacionados que sirvieron de inspiración y guía.

La sección 3 describe en detalle la propuesta anteriormente planteada. Por lo tanto, se especifican las diferentes técnicas y requerimientos que orientaron el desarrollo de la aplicación. Finalmente, se expone el proceso de experimentación que permite abarcar los casos de estudio y técnicas propuestas de manera ordenada.

En la cuarta sección se plantean formalmente las restricciones y requerimientos que condicionaron y dieron forma a la aplicación. Como el desarrollo de la misma fue bastante extenso, la explicación se dividió en diseño e implementación, y esta última a su vez en backend y frontend. Se utilizan diagramas formales e informales, que facilitan la visualización de las decisiones de diseño que posibilitaron la extensión de las herramientas existentes.

En la sección 5 se procede a realizar el proceso de experimentación. Primero se describen brevemente los distintos casos de estudio utilizados (redes), y posteriormente, las etapas del proceso donde se encuentran los resultados y observaciones de los diferentes experimentos realizados sobre la aplicación.

Finalmente, en la sexta sección, se encuentran las conclusiones generales obtenidas a partir de este trabajo en su totalidad. Se describen varias cuestiones generales inferidas a partir del desarrollo de la herramienta y de los experimentos realizados sobre la misma. Adicionalmente, se mencionan distintas maneras de extender la aplicación, así como también otros tipos de experimentos que podrían realizarse en el futuro.

2. Marco conceptual y teórico

El objetivo de esta sección consiste en situar el problema de investigación dentro de un conjunto de conocimientos. Con el fin de presentar una introducción al dominio del problema, se detallan los conceptos teóricos más relevantes. A su vez, se lleva a cabo un análisis de diferentes trabajos relacionados, que permiten conocer el avance actual sobre la rama investigación que se trata en este trabajo.

2.1. Marco Teórico

2.1.1. Redes sociales y grafos

Una red social [Lozares, 1996] es una estructura compuesta por actores que están relacionados entre sí de acuerdo a algún criterio. Constituyen una forma útil de representar relaciones de todo tipo y obtener una visión global de la estructura social de algún conjunto de individuos u organizaciones.

Debido al avance exponencial de la tecnología, en los últimos años se han producido cambios radicales en la manera en que se comportan y relacionan las personas. Desde cualquier dispositivo, lugar y manera, dos personas se pueden encontrar intercambiando información de diversos tipos, con el objetivo de realizar alguna tarea útil, o simple entretenimiento. Esto se debe en gran parte a la existencia de las redes sociales virtuales, que permiten que sus usuarios puedan comunicarse de formas muy variadas (texto, video, audio, etc.) prácticamente al instante. Las redes sociales virtuales u online, también conocidas como servicios de red social tienen como característica fundamental que sus actores son personas, denominados usuarios, y por ser un medio de comunicación cuyo canal es internet. Dentro de este tipo de red social, los usuarios pueden crear perfiles que los identifican con cierta información personal, establecer contacto con otros usuarios de la misma red y compartir contenido multimedia. Debido a la velocidad que proporciona Internet como canal de comunicación, todo el tiempo se crean o desaparecen usuarios y relaciones entre los mismos. [Wang et al., 2015]

El contenido que se comparte es efímero, y cualquier evento de gran importancia en el mundo se distribuye rápidamente entre los usuarios. Por estas, y algunas otras características más, las redes sociales son un objeto de estudio muy importante, debido a que, es posible inferir información relevante de su estructura y dinámica.

El análisis de redes sociales [Freeman, 2006] estudia la estructura social aplicando la teoría de grafos. Un grafo es una estructura de datos utilizada por las ciencias de la computación, compuesta por un conjunto de vértices o nodos, que se unen entre sí mediante enlaces o aristas que representan relaciones binarias entre los nodos.

De esta forma, la red puede ser representada mediante un grafo en el cual los usuarios son los nodos, y las relaciones entre los usuarios son las aristas.

La teoría de grafos [Godsil y Royle, 2001] es la rama de las matemáticas que se encarga del estudio de las características de los grafos. Debido a las técnicas que abarca la teoría, éstos pueden ser utilizados para representar diversos tipos de problemas y situaciones de la vida real, como en este caso, las redes sociales.

La representación de redes sociales mediante grafos proporciona la ventaja de que las herramientas que se utilizan en la teoría de grafos pueden extrapolarse al estudio de las redes sociales realizando una adecuada interpretación de los datos. Estas herramientas permiten estudiar cómo la estructura general afecta a los individuos como un todo, cómo cada individuo afecta a los demás, los patrones del grafo que se forman a alto nivel debido las relaciones de los usuarios, las características de la red desde un punto de vista topológico, y un gran sinfín de aplicaciones prácticas y teóricas posibles.

Existen varias características que pueden estudiarse de una red social, y que, en consecuencia, identifican a cada una de ellas. Por ejemplo, cada red social puede utilizarse con un fin específico, debido a las posibles interacciones que pueden darse entre las personas. Existen redes con el objetivo de crear lazos de amistad entre sus usuarios como, por ejemplo, Facebook, de crear relaciones laborales como en LinkedIn, o simplemente de conexión para conocer los contenidos que los otros usuarios comparten, como en Youtube. Las organizaciones dueñas de las redes sociales están interesadas en obtener la mayor cantidad de ganancias posibles. Para maximizar esto, pueden utilizar la información de sus redes y aplicar técnicas del análisis de grafos, para conocer cuáles usuarios son los más influyentes, cuáles publicidades son las más vistas, y cómo son distribuidas mediante las relaciones. Por este tipo de utilidades, analizar las redes sociales es de suma importancia ya que la información derivada puede resultar muy valiosa.

Hay información sobre las redes que no puede ser estudiada directamente mediante grafos, pero sí influye notablemente en el resto de datos. Por ejemplo, el objetivo con el que los usuarios utilizan la red, no es de interés en sí mismo desde el punto de vista de la teoría de grafos, pero sí afecta a la estructura, dinámica y evolución de la red y su topología a través del tiempo. En redes donde las relaciones entre usuarios tienden a durar poco, la topología global de la red se encuentra en constante evolución y, por lo tanto, la teoría de grafos puede ser útil para realizar medidas sobre la estructura del grafo, y poder obtener diferentes métricas de interés que, interpretadas correctamente, pueden ser útiles, por ejemplo, para el conocimiento del estado futuro de la red. También es posible calcular la densidad de la red en general, la centralidad de un nodo cualquiera, que indicaría la importancia de un usuario en la red social, la cantidad de nodos con respecto a la cantidad de enlaces existentes, del cual se puede derivar el nivel de relaciones que existen entre los usuarios, y así, varias medidas más que pueden resultar útiles.

Un problema conocido que ha sido estudiado desde varios puntos de vista, incluyendo el de grafos, es el efecto de mundo pequeño [Barabási, 2003]. Este

fenómeno es la hipótesis de que dos nodos cualesquiera de la red están separados mutuamente por una corta distancia comparado con el tamaño real de la red. Desde la teoría de grafos, este efecto se puede medir utilizando la distancia media entre todos los pares de nodos de una red. Desde el punto de vista de las redes sociales significa que los contenidos de los usuarios se esparcen rápidamente por la red entera, debido a la corta distancia entre sus integrantes. Esto puede ser útil para las empresas publicitarias. Sin embargo, puede resultar contraproducente en los casos en los que dos usuarios que tal vez nunca se relacionan, se encuentran separados por una distancia muy corta.

Otro problema conocido que puede valerse de las herramientas del análisis de grafos es el de predicción de enlaces entre usuarios. Este problema es el objeto de estudio de la tesis, y es explicado en detalle en la siguiente sección.

2.1.2. Link Prediction

[Weng et al., 2015] La predicción de enlaces consiste en el problema de predecir enlaces inexistentes en un momento dado del grafo, pero que si podrían existir en otro momento determinado. Tener la posibilidad de estudiar la forma en que surgen nuevos enlaces en las redes sociales, implica poder entender cómo los usuarios tienden a relacionarse entre sí, y desde un punto de vista más general, la manera en que la redes evolucionan con el tiempo.

El estudio de los enlaces es bastante complejo debido a la dinámica de las redes sociales, en las cuáles las relaciones y usuarios aparecen y desaparecen todo el tiempo. Link Prediction solo se centra en el estudio de los enlaces, asumiendo que los nodos del grafo permanecen constantes. Sin embargo, la información de los nodos que participan de un enlace es vital a la hora de determinar las características del enlace formado, ya que identifican al enlace con ciertas propiedades. Si los nodos hubiesen tenido otras propiedades podrían haber provocado que el enlace entre ellos no existiera.

Existen varias formas de realizar Link Prediction, pero hay dos principales que dependen de que los enlaces del grafo a estudiar posean marcas de tiempo o no, según se argumenta en [Wang et al., 2015].

En el primer caso se toman dos instantes de tiempo cualquiera del grafo siendo uno de los instantes menor al otro ($t_0, t_1 / t_0 < t_1$). A continuación, se intentan predecir los enlaces no presentes en el t_0 , pero que si están presentes en el t_1 . Si hay nodos que se encuentran en t_1 y no en t_0 , sus enlaces no podrán ser predichos dado que Link Prediction solo se centra en el estudio de los enlaces, y no tiene en cuenta la aparición o desaparición de nodos. En este tipo de casos se dice que los grafos son dinámicos, ya que presentan cambios a través del tiempo.

La otra forma de estudiar la predicción de enlaces es cuando no se poseen marcas de tiempo en los enlaces y, por lo tanto, el grafo a estudiar es considerado estático, debiéndose utilizar diferentes estrategias para el estudio. Una estrategia usualmente utilizada es ocultar un conjunto de enlaces, y utilizar Link Prediction para verificar si

se predicen correctamente ese conjunto de enlaces ocultos. Otra estrategia es utilizar la red tal cual está e intentar predecir los enlaces ya existentes correctamente, a partir de entrenamiento previo con esta u otras redes. Ambas técnicas caen dentro de lo que se conoce como aprendizaje automático, y la última descrita, es la que se estudia en el marco de este trabajo.

El enfoque de predicción de enlace de [Ricotta y Santamarina, 2016], consiste en calcular un grado de similitud para toda potencial arista. Esta función se calcula de acuerdo a un criterio específico, y por ello existen varias funciones, cada una de acuerdo a un criterio y enfoque diferente. Como se utiliza la representación mediante grafos, los enfoques toman ventaja de distintas características de los mismos. Por ejemplo, algunos se basan en la topología del grafo para extraer información y decidir la similitud entre dos nodos. Otros enfoques utilizan la información propia contenida en los nodos a comparar. Una vez calculados los valores de similitud para todas las potenciales aristas, las mismas se ordenan de forma decreciente (o creciente) de acuerdo al valor de similitud que poseen. Cuanto más grande (o chico) sea el valor de similitud, más probable es que exista la potencial arista entre dos nodos. A partir del ranking armado con las potenciales aristas, se eligen las K mejores, las cuáles pasan a ser las consideradas como aristas reales, y el resto, son considerados como aristas no existentes, por lo tanto, los nodos participantes son predichos como si no tuviesen conexión alguna. Finalmente, se comparan las predicciones con el grafo real correspondiente, dependiendo si es estático o dinámico, y se calculan distintas métricas del rendimiento final de las predicciones.

2.1.3. Aprendizaje automático

El aprendizaje automático [Mitchell, 1997] [Flach, 2012], también conocido como aprendizaje de máquina (“Machine Learning” en inglés), se basa en conceptos y resultados de muchos campos, incluyendo estadística, inteligencia artificial, filosofía, teoría de la información, biología, ciencia cognitiva y complejidad computacional. Dentro del campo de la inteligencia artificial, el objetivo es desarrollar técnicas que permitan a las computadoras “aprender” a partir de la experiencia. De forma más concreta, se trata de crear programas capaces de generalizar comportamientos a partir de una información no estructurada suministrada en forma de ejemplos. Es, por lo tanto, un proceso de inducción del conocimiento.

En la Figura 1 se observan las distintas técnicas que el proceso inductivo abarca para extraer reglas o patrones a partir de un conjunto de ejemplos. Dentro de estas técnicas, se destacan las conocidas como aprendizaje supervisado y aprendizaje no supervisado.

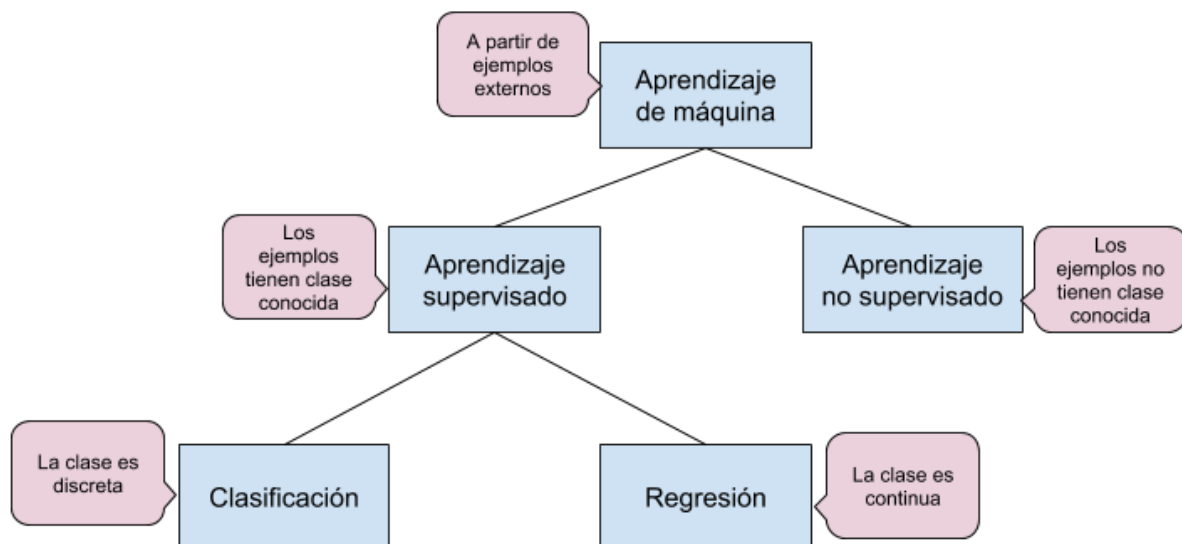


Figura 1 - Técnicas del proceso inductivo

En el aprendizaje no supervisado, no existe un conocimiento a priori de los datos a evaluar. Todo el proceso de modelado se lleva a cabo sobre un conjunto de ejemplos formado tan sólo por entradas al sistema. No se tiene información sobre las categorías de esos ejemplos, por lo que, el sistema tiene que ser capaz de reconocer patrones para poder etiquetar las nuevas entradas. Para decidir los tipos de las entradas, comúnmente se utiliza el algoritmo de agrupamiento, también conocido como “clustering”, mediante el cual se crean distintos “clusters” o conjuntos, dada la similitud de las características calculadas de los ejemplos. Cada grupo que queda al final del algoritmo, es una categoría diferente, y las entradas quedan categorizadas de acuerdo al grupo al que pertenecen.

Por su parte, el aprendizaje supervisado consiste en deducir una función o modelo a partir de datos de entrenamiento. Esta función establece una correspondencia entre las entradas y las salidas deseadas del sistema. Los datos consisten en pares de objetos compuestos por los datos de entrada en un lado, y los resultados deseados en el otro. La salida de la función puede consistir en un valor numérico (la clase es continua), o una etiqueta (la clase es discreta). La primera es conocida como un problema de regresión, mientras que la segunda consiste en un problema de clasificación. El objetivo del aprendizaje supervisado es el de crear una función capaz de predecir el valor correspondiente a cualquier objeto de entrada válida después de haber visto una serie de ejemplos llamados datos de entrenamiento. Cada ejemplo posee diferentes características propias o calculadas, y la clase a la que pertenecen. Estos datos se utilizan para calcular las características propias de cada clase, y le permite al clasificador, clasificar correctamente los futuros ejemplos que se presenten. El conjunto de ejemplos que engloba a los utilizados en el entrenamiento o en el testeo se lo conoce como dataset de la clasificación.

La predicción de enlaces utiliza el aprendizaje supervisado de una manera bastante diferente, pero conservando ciertos elementos comunes [Wang et al., 2015]. Los

distintos ejemplos a analizar siguen siendo pares de nodos, y como en cada par de nodos cualquiera puede existir o no enlace entre ellos, se utiliza esa característica para clasificar. Por lo tanto, la clase de los ejemplos es discreta y puede tomar dos valores diferentes: existe enlace y no existe enlace. Como anteriormente se explicó, se suelen utilizar varios ejemplos de ambos tipos para entrenar al clasificador, para que éste clasifique posteriormente otros ejemplos de manera correcta, es decir, realice buenas predicciones acerca de los enlaces.

Además de la clase, también son importantes otras características que identifiquen a cada par de nodos, ya que el clasificador necesita ser entrenado para saber cuáles valores de las demás características corresponden a que exista enlace o no entre un par de nodos cualquiera. Las funciones de similitud explicadas previamente son buenos candidatos como elementos que caracterizan a un par de nodos, ya que dan como resultado un valor numérico de la similitud, dependiendo del criterio elegido. Estos valores de similitud se utilizan para construir el dataset que es utilizado posteriormente en la clasificación.

Los datasets de ejemplos permiten almacenar los valores de varias características o atributos a la vez, junto a la clase de cada ejemplo. Gracias a esto, se condensa más información con respecto a las características de un grafo en un dataset utilizado por el método de aprendizaje supervisado. Sin embargo, esto no significa un mejor rendimiento a la hora de predecir enlaces. Para eso, se debe realizar previamente todo un análisis al respecto, el cual es uno de los objetivos que se abordarán en la etapa de experimentación.

2.1.4. Funciones de similitud

Como ya se explicó con anterioridad, las funciones o métricas de similitud poseen un rol muy importante en el aprendizaje automático. De forma individual se utilizan para determinar el ranking de ejemplos, para posteriormente seleccionar los K mejores. En este caso pueden compararse diferentes funciones de similitud entre sí, pero no pueden utilizarse varias a la vez, salvo contadas excepciones en las cuáles si es posible, debido a una normalización similar de los valores de las funciones. [Santamarina y Ricotta, 2016]

En el método supervisado [Wang et al., 2015], se pueden considerar como características para la construcción de los datasets a utilizar en la clasificación. Pueden incluirse varias a la vez, y como se suponen todas independientes, no existen límites al respecto. En el caso de no ser independientes, el problema que ocurrirá es la introducción de ruido en el dataset y, por lo tanto, un peor rendimiento a la hora de la clasificación. Para eliminar o disminuir el ruido, es posible realizar una etapa de selección de las características que mejor representan al grafo, para luego crear el dataset que se utilizará para clasificar.

Según Wang, existen distintas categorías que engloban a las funciones de similitud, que utilizan criterios semejantes a la hora de calcular la semejanza entre dos nodos. Una forma de diferenciar las funciones de similitud, es separando entre aquellas que

aprovechan las características topológicas de la red y aquellas que aprovechan la información contenida en los nodos de la red.

Las técnicas que se basan en las características topológicas de la red, fundamentan su criterio en la teoría de grafos y pueden ser muy variadas. Se clasifican a su vez en dos subcategorías posibles dependiendo el tipo de información que utilizan: locales y globales.

Los métodos locales son aquellos que utilizan la información cercana a los nodos tales como los vecinos de los mismos. Tienden a ser los métodos más simples y rápidos dada la poca información y el poco procesamiento que implica calcular las cuestiones asociadas a los vecinos. Por ejemplo, varios métodos consideran que dos nodos tienen más probabilidad de poseer enlace cuanto más vecinos en común poseen. Desde el punto de vista de las redes sociales, cuantos más amigos en común poseen dos personas, mayor es la probabilidad de que en algún momento esas dos personas se vuelvan amigos.

En cambio, los métodos globales utilizan la información disponible de la topología total de la red. Debido a esto tienden a ser técnicas computacionalmente costosas ya que recorren los caminos entre dos nodos buscando, por ejemplo, el camino más corto. Se espera que el aporte de información que brindan estas técnicas justifiquen el mayor costo computacional, y así, mejoren el rendimiento de las técnicas de aprendizaje automático. No siempre ocurre esto y, por ende, se necesita un análisis que se encargue de estudiar todas las cuestiones asociadas.

Las funciones de similitud que utilizan la información contenida en los nodos de la red se basan en similitudes entre la información de cada nodo del par. Por ejemplo, las redes sociales virtuales permiten asociar a cada usuario. Este tipo de redes permiten crear un perfil público, enviar mensajes, crear posts con distintos tipos de contenidos, y un sin fin más de información que se puede asociar a cada uno de los participantes de la red. Tener en cuenta esta información conlleva un costo computacional adicional, pero en ciertos casos puede mejorar considerablemente el rendimiento de las técnicas de aprendizaje automático. La gran desventaja de este tipo de funciones de similitud recae en que son dependientes del dominio de aplicación de la red que está siendo analizada. En consecuencia, la implementación de estas técnicas es única en cada caso, y sus resultados no son extrapolables a otras redes.

A continuación, se realiza un breve análisis sobre varias de las funciones de similitud entre dos nodos descritas por Wang. Se describe el tipo de función, el criterio utilizado, el funcionamiento y la fórmula para calcular la similitud, en caso de existir.

Notación: dado un nodo x , se denota $\Gamma(x)$ al conjunto de nodos vecinos de x , o sea, aquellos nodos con enlace a x pertenecen a $\Gamma(x)$. Así también se denota $|\Gamma(x)|$ como la cantidad total de vecinos de x , también llamado grafo del nodo x .

Common neighbors (CN)

Topológica. Local. Su criterio se basa simplemente en la cantidad de vecinos en común que poseen dos nodos.

$$\text{Similitud}(x,y) = |\Gamma(x) \cap \Gamma(y)|$$

Jaccard coefficient (JC)

Topológica. Local. Mejora de la anterior función cuyo valor se divide con la cantidad de vecinos diferentes en total que poseen ambos nodos. Se busca que los nodos están fuertemente relacionados, más allá de tener muchos vecinos.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{|\Gamma(x) \cup \Gamma(y)|}$$

Sørensen Index (SI)

Topológica. Local. Divide la cantidad de vecinos en común con la suma de las cantidades de vecinos de cada nodo. En consecuencia, según este criterio, los nodos con vecinos comunes poseen mayor similitud cuando tienen menor cantidad de vecinos en total.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{|\Gamma(x)| + |\Gamma(y)|}$$

Salton Cosine Similarity (SC)

Topológica. Local. Considera que la similitud entre dos nodos está dada por la medida seno común.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{\sqrt{|\Gamma(x)| \cdot |\Gamma(y)|}}$$

Hub Promoted (HP)

Topológica. Local. HP define la superposición topológica entre los nodos x e y. Se divide la cantidad de vecinos en común por el grado más bajo entre x e y.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{\min(|\Gamma(x)|, |\Gamma(y)|)}$$

Hub Depressed (HD)

Topológica. Local. Muy similar a HP con la diferencia de que el cociente es el grado más alto entre x e y.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{\max(|\Gamma(x)|, |\Gamma(y)|)}$$

Leicht-Holme-Nerman (LHN)

Topológica. Local. Muy similar a la función SC pero que utiliza el producto de la cantidad de vecinos como cociente sin usar la raíz de esa multiplicación. Asigna más similitud a aquellos pares de nodos con muchos vecinos comunes en comparación con el total esperado de tales vecinos.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{|\Gamma(x)| \cdot |\Gamma(y)|}$$

Parameter-Dependent (PD)

Topológica. Local. Se dispone de un parámetro libre λ que permite modificar el valor final de la similitud dada por esta función. Es de utilidad cuando se prefiere variar los valores de similitud en un rango de posibilidades. Si $\lambda = 0$, funciona como CN; si $\lambda = 0.5$ deriva en el método de SC; y si $\lambda = 1$ da los mismos resultados que LHN.

$$\text{Similitud}(x,y) = \frac{|\Gamma(x) \cap \Gamma(y)|}{(|\Gamma(x)| \cdot |\Gamma(y)|)^\lambda}$$

Adamic-Adar coefficient (AA)

Topológica. Local. Es otra función que busca mejorar el concepto de vecinos comunes. La mejora radica, en que se le asigna mayor peso a aquellos vecinos comunes que poseen grado bajo, o sea, pocos enlaces.

$$\text{Similitud}(x,y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{\log|\Gamma(z)|}$$

Preferential attachment (PA)

Topológica. Local. Se basa en la idea de que un nodo aumenta la probabilidad de generar enlaces cuantos más vecinos posee. Por esta causa, la similitud es el producto de la cantidad de vecinos entre ambos nodos.

$$\text{Similitud}(x,y) = |\Gamma(x)| \cdot |\Gamma(y)|$$

Resource allocation (RA)

Topológica. Local. Es muy similar a AA ya que utiliza los datos de los vecinos de los nodos para calcular la similitud. Sin embargo, penaliza aún más a aquellos vecinos en común con mayor grado. En redes con un promedio bajo del grado de los nodos, AA y RA dan resultados muy similares, pero en redes con promedio alto, RA suele funcionar mejor.

$$\text{Similitud}(x,y) = \sum_{z \in \Gamma(x) \cap \Gamma(y)} \frac{1}{|\Gamma(z)|}$$

Shortest Path (SP)

Topológica. Global. Un camino entre dos nodos está formado por las aristas y nodos intermedios que unen a ambos, y debido a esto, existen múltiples caminos posibles. El objetivo de esta función es buscar el camino de menor peso entre dos nodos, utilizando todos los caminos posibles entre ellos. Las aristas poseen un peso asociado que refleja alguna característica de la relación entre los nodos. Sin embargo, en las redes sociales que se estudian en este trabajo, todos los enlaces poseen el mismo peso, por ende, el camino más corto se refleja como el camino que posee menos nodos intermedios. Algo importante a tener en cuenta es que, cuanto más similares son dos nodos, más bajo es el valor de SP entre ellos, a diferencia de la mayoría de funciones de similitud. Esto significa que cuanto más cerca se encuentren los nodos, más parecidos son.

Hitting Time (HT)

Topológica. Global. Para calcular la función se realiza un recorrido aleatorio (random walk, en inglés) desde el nodo x al nodo y, contando la cantidad de pasos dados. El recorrido comienza en uno de los nodos, avanza hacia alguno de los vecinos de forma aleatoria y así sucesivamente hasta alcanzar al otro nodo. HT refleja la cantidad de pasos dados para llegar de un nodo a otro, y como en SP, cuanto más bajo es el valor (menos saltos), más similares son los nodos. Dada la naturaleza azarosa del algoritmo que calcula esta función, distintas corridas de HT pueden dar valores diferentes para el mismo par de nodos.

Node Information (NI)

Información de los nodos. La función utiliza los datos internos de los nodos para calcular la similitud entre ellos. Cabe aclarar que no cualquier atributo puede ser utilizado para el cálculo de esta función. Deben ser atributos que no sean identificadores, sino todos los nodos serían diferentes, y que posean valor en la mayoría de los nodos, sino todos los nodos serían iguales. Por ejemplo, en una red social, atributos válidos de una persona para NI pueden ser la ciudad de nacimiento, la profesión actual, ciudad de residencia, color favorito, entre otros.

Para el cálculo de NI, se toman cada uno de los atributos elegidos por separado de los nodos, se pondera su valor, se suman todos los valores ponderados, y así, se obtiene el valor de similitud entre los nodos. Las ponderaciones deben ser valores entre 0 y 1, y la suma de las ponderaciones debe dar 1. Por ende, el valor final de NI se encontrará entre 0 y 1, y cuanto más alto es, más similares son los nodos. Dependiendo de cuánta importancia se le quiera dar a cada atributo, se modifica la ponderación asociada a cada uno. Siguiendo el ejemplo anterior, si se quisiera reflejar que la ciudad de residencia es más importante que la ciudad de nacimiento, entonces la ponderación de la ciudad de residencia será mayor a la de nacimiento (ej: $NI = \text{residencia} \cdot 0.6 + \text{nacimiento} \cdot 0.1 + \text{color} \cdot 0.05 + \text{profesión} \cdot 0.25$).

Como se explicó anteriormente, la efectividad de NI depende de los atributos y su ponderación que, a su vez, dependen del dominio de aplicación de la red social que se estudia. Como consecuencia, debe realizarse un análisis variando los atributos y las ponderaciones elegidas para encontrar la que mejor refleje a la red en cuestión, y debido a esto, los valores de NI que se obtengan no son extrapolables a otras redes que posean otro dominio de aplicación.

Vector Node Information (VNI)

Información de los nodos. De forma similar a NI, utiliza la información de los nodos, pero de otra manera. La diferencia radica en que los valores del atributo a evaluar deben ser conjuntos o listas de palabras. El criterio para calcular la similitud entre los nodos se basa en cuantas palabras iguales poseen ambos, dado el mismo atributo elegido. Como también ocurría en la anterior función, no cualquier atributo es útil para ser elegido, debe ser un atributo que posee gran cantidad de palabras para que exista cierta cantidad de coincidencias entre los distintos nodos.

El funcionamiento de esta función es simple, se toman las listas de palabras de ambos nodos, y se comparan una a una, si coinciden o no. En caso de coincidencia se suma uno. El valor de similitud final entre dos nodos está dado por la cantidad de coincidencias. Es posible realizar un preprocesamiento de las palabras previo al cálculo de esta función, con el objetivo de eliminar palabras innecesarias como conectores, artículos, etc., y así, mejorar la fidelidad del valor final de la función. Para realizar esta tarea se pueden utilizar la librerías de procesamiento de palabras como, por ejemplo, Lucene ³.

2.1.5. Datasets

Los conocidos comúnmente como datasets [Zytkow y Rauch, 1999], son colecciones de datos que generalmente corresponden a contenidos de una única tabla de base de datos, donde cada columna representa un atributo o variable en particular, y cada fila representa una instancia determinada del conjunto de datos en cuestión.

Un dataset contiene valores para cada una de sus variables, que corresponde a cada instancia del conjunto de datos. Cada uno de estos valores se los conoce con el nombre de dato. Las variables del dataset pueden ser representadas por valores numéricos o por etiquetas. La importancia de esta representación de la información, es que posibilita la creación de un modelo que permita clasificar (predecir) nuevos datos a partir de la distribución de los datos actuales.

Previamente a construir estos modelos, es necesario haber especificado el problema. Para ello, se debe definir una determinada cantidad de atributos que representen a cada instancia del conjunto de datos. Como el problema consiste en predecir alguno de estos atributos, debe existir un conjunto de atributos o variables independientes, y una variable dependiente del resto de los atributos. Por ejemplo, los datos crudos sobre el clima en una región, como la temperatura, la velocidad y dirección del viento, el porcentaje de humedad y la nubosidad, pueden determinar si va a llover o no. Por lo tanto, es posible construir un dataset con la información histórica de cada una de estas variables, incluyendo también si existieron precipitaciones o no. De esta forma, se puede llegar a predecir la ocurrencia de precipitaciones basándose en los datos recolectados, mediante los métodos de aprendizaje ya mencionados.

El problema que se plantea para el presente trabajo consiste en la clasificación de la existencia o no de un vínculo entre dos nodos de una red. Por lo tanto, el dataset puede ser representado de forma que cada instancia contenga información relativa a cada par de nodos de la red, y la variable dependiente dentro del mismo sea un atributo con dos valores posibles: link y nolink. Los atributos que pueden representar a un par de nodos pueden ser de diferente índole, y dependiendo de cada red, pueden ser más o menos representativos. A su vez, existen atributos que permiten

³ Sitio web de la librería Lucene <https://lucene.apache.org/core/>.

obtener mejores clasificaciones que otros dependiendo del algoritmo de clasificación que se implemente.

En el marco de este trabajo, se considerarán como posibles atributos las similitudes entre cada par de nodos, basadas en las funciones de similitud ya explicadas. Puede ocurrir el caso en el cual, para ciertos atributos, no se conozca el valor asociado como, por ejemplo, que una función de similitud es null para algún par de nodos. Estos valores se anotan como “?” y son conocidos como valores faltantes. El tratamiento que se realiza con estos valores depende del clasificador que se utilice en la clasificación. Cabe destacar que, si un atributo posee valores faltantes para la gran mayoría de instancias, entonces es importante analizar si ese atributo es realmente útil a la hora de clasificar.

Cuando se construyen datasets pensados para realizar clasificación supervisada con algoritmos de aprendizaje automático, es importante considerar la distribución de la cantidad de instancias de cada clase [Arrieta y Mera, 2015]. Se dice que un dataset para clasificación es desbalanceado cuando el número de observaciones no es el mismo para todas las clases. En estos casos, los algoritmos de clasificación tienden a favorecer a la clase predominante, lo cual puede derivar en obtener métricas sesgadas. Un dataset desbalanceado puede ser particularmente problemático cuando se requiere clasificar de forma correcta una clase “rara” o minoritaria. Lo que sucede en estos casos, es que los valores de exactitud de la clasificación son altos, pero son resultado de la clasificación correcta de la clase predominante, y no de la clase minoritaria.

Al crear un dataset basado en las observaciones de una red, donde cada instancia contiene información sobre cada par de nodos con el fin de clasificar la existencia del vínculo entre ellos, se cuenta únicamente con dos clases. La distribución de valores de esta clase va a ser dependiente de que tan conexas sea la red: cuanto mayor sea el grado medio de conexión de la red, más valores de “link” contendrá el dataset.

De todas formas, en la mayoría de las redes es común contar con una considerable mayoría de pares de nodos que no son “link”, y esta diferencia tiende a aumentar cuando las redes son más grandes, ya que, si la cantidad de nodos aumenta linealmente, la cantidad de enlaces posibles aumenta exponencialmente. Por lo tanto, al incluir las observaciones sobre todos los pares de nodos de la red, se puede obtener un dataset sumamente desbalanceado, con una gran cantidad de clases “nolink”, y sólo unas pocas clases con valor “link”.

Existen varios métodos que permiten tratar datos desbalanceados, los cuales pueden agruparse en cuatro categorías: submuestreo, sobremuestreo, generación de datos sintéticos y aprendizaje sensible al costo [Arrieta y Mera, 2015] [Pozzolo et al., 2015]. Todos estos métodos modifican la proporción de las clases y el tamaño del dataset original.

Los métodos de submuestreo consisten en eliminar las observaciones de la clase mayoritaria con el fin de igualar la distribución de las clases. Para equiparar los tamaños de las clases en los datasets generados para la clasificación de enlaces,

se utilizarán algunas variantes de este método, eliminando instancias del dataset cuya clase sea “nolink”. En cambio, el sobremuestreo consiste en duplicar, de acuerdo a ciertos criterios, las muestras de la clase minoritaria para igualarlas a las de la clase mayoritaria. Esta técnica provoca varios problemas, entre los que se encuentran el aumento del tamaño de los datasets que afecta negativamente al rendimiento, y el sobreajuste de los clasificadores que empeora la clasificación de nuevas muestras. La generación de datos sintéticos se puede considerar una forma especial de sobremuestreo, mediante la cual se crean nuevas muestras a partir de varias combinadas pertenecientes a la clase minoritaria, y finalmente, ambas clases quedan balanceadas. Como desventaja, las relaciones no lineales entre las características de la muestra original pueden perderse. Por último, el aprendizaje sensible al costo abarca diferentes técnicas que permiten penalizar las clasificaciones erróneas de la clase minoritaria, y que es posible utilizarlas con casi cualquier clasificador de manera conjunta, aunque afecta negativamente al rendimiento debido al procesamiento extra necesario.

Una forma de balancear estos datasets mediante submuestreo, es simplemente seleccionando aleatoriamente algunas instancias del conjunto de “nolink”, hasta tener una distribución igualada de clases. Se debe considerar que los resultados de la clasificación luego de aplicar esta técnica pueden estar sesgadas por la selección aleatoria de instancias. Para paliar este problema, se puede dividir el conjunto de instancias con clase “nolink” en diferentes subconjuntos excluyentes entre sí, de igual cantidad de instancias que el número de observaciones con clase “link”. Posteriormente, se pueden formar diferentes datasets, donde cada uno contiene el total de observaciones con clase “link” y uno de los subconjuntos de observaciones con clase “nolink”. De esta forma, puede aplicarse el algoritmo de clasificación en cada uno de ellos, y promediar los resultados obtenidos para evitar que estos estén sesgados por los resultados arrojados por uno sólo de los subconjuntos.

Otra alternativa para disminuir la cantidad de instancias de “nolink”, es descartando los pares de nodos que no aportan información para la clasificación. Por ejemplo, si los atributos del dataset contienen información sobre las funciones de similitud basadas en la topología local, sólo tendrán información relevante aquellos pares de nodos que tengan al menos un vecino en común (a excepción de Preferential Attachment). Entonces, podrían descartarse del dataset todos los pares de nodos que no tengan ningún vecino en común. Es importante aclarar que, si bien esta técnica reducirá la cantidad de observaciones de la clase “nolink”, también puede eliminar algunos pares de nodos que son “link”. Estos consisten en aquellos pares que, si bien están vinculados entre sí, no tienen ningún vecino en común, y dependiendo de la red, en líneas generales representan a la minoría de los pares de nodos (en muchos casos nulo). Otra consideración de esta práctica es que, si bien tiende a equiparar la distribución de las clases, no obtiene un dataset completamente balanceado.

Otro tema importante a tener en cuenta es que no todos los atributos presentes en el dataset aportan información útil para la clasificación. Puede ocurrir el caso en el que los valores de los atributos están repartidos de forma aproximadamente azarosa en instancias con distintas etiquetas de clase. Determinar cuáles son las características que no aportan información útil a la clasificación es un problema complejo, que se puede realizar de forma empírica por prueba y error, o utilizar ciertos algoritmos automáticos que miden la relación entre los atributos y seleccionan los mejores. Este problema se conoce como “feature selection” [Guyon y Elisseeff, 2003] [James et al., 2013] (selección de características). Se busca encontrar un subconjunto de atributos más relevantes y menos redundantes posibles. Además, de mejorar la calidad de la clasificación, la selección de características provee otras ventajas: simplifica los modelos con los que se trabaja facilitando su interpretación, disminuye el tiempo de entrenamiento y mejora la generalización, evitando el sobreajuste. Generalmente la selección de características realiza medidas sobre los valores de los atributos, y así, obtener medias y varianzas para determinar el grado de correlación entre los distintos atributos, y así determinar aquellos irrelevantes o redundantes.

La selección de características posee dos etapas fundamentales. La evaluación es la etapa en la cual los atributos son evaluados de acuerdo a sus características y valores, siguiendo algún tipo de criterio cómo la correlación entre ellos, la relación con los valores de clase, o la maximización (o minimización) de cierta métrica, utilizando un esquema de aprendizaje. La otra etapa es la de búsqueda, y su objetivo es ir comparando los subconjuntos de atributos obtenidos en la etapa anterior para encontrar el mejor. Existen varios algoritmos que implementan de distinta manera las dos etapas por separado. Cada uno sigue criterios diferentes y posee ventajas y desventajas asociadas.

2.1.6. Clasificadores

La clasificación es una de las dos técnicas que caben dentro de la categoría de aprendizaje automático supervisado [Nasa y Suman, 2012] [Kotsiantis, 2007]. Difiere de la regresión, dado que la salida de la función consiste en un conjunto de valores finito, conocidos como etiquetas de clase. Como ya se explicó anteriormente, la clasificación utiliza los datos de entrenamiento para generalizar una función que clasifique lo mejor posible los datos de testeo.

Previo a la clasificación deben tomarse ciertas consideraciones importantes. Primero se debe determinar el tipo de los datos de entrenamiento (si van a ser numéricos, palabras, etc.), la manera de obtenerlos de la fuente de datos a analizar y las etiquetas de clase que se utilizarán. Además, se debe obtener un gran conjunto de datos para no sesgar ninguno de los experimentos a realizar durante la clasificación, aunque esto debe ir variando para evitar el sub y el sobreentrenamiento. Como siguiente paso, debe determinarse la forma en que los

datos se ingresan como entrada a la clasificación (esto abarca a todos los datos: entrenamiento y testeo).

Comúnmente se utiliza un vector de características junto a la etiqueta de clase correspondiente a cada ejemplo. De esta forma se crea un dataset descrito en la sección anterior: las características (variables predictoras) y la etiqueta de clase (variable clase) son los atributos, y cada ejemplo, es una instancia diferente del dataset. A continuación, se procede a elegir la estructura de la función que generaliza y clasifica los datos, o sea, el clasificador. Existen clasificadores de distintos tipos, cada cual, con ventajas y desventajas, por lo tanto, es importante saber analizar y elegir cual es el que mejor se adapta al conjunto de datos. Finalmente, se pueden configurar parámetros del clasificador y de la técnica de validación, que determina el desempeño de la clasificación, medido por varias métricas que caracterizan la eficiencia y eficacia de la técnica.

Hay una gran variedad de clasificadores con distintas características, que condicionan distintas etapas de la clasificación como: la forma de obtener la función clasificadora, el rendimiento de la clasificación, el tiempo de entrenamiento, el tratamiento de los valores faltantes, entre otras. Debido a esto, existen varias categorías que engloban a clasificadores similares entre sí [Kotsiantis, 2007].

A continuación, se explican los clasificadores más conocidos, junto con una breve introducción de la categoría a la que pertenecen.

Naive Bayes:

Consiste en un clasificador bayesiano, lo cual implica que el modelo de clasificación se basa en la teoría de Bayes [Nasa y Suman, 2012] [Rennie et al., 2003] [Rish, 2001] [Kotsiantis, 2007]. Para estructurar los datos, se crea una red bayesiana, el cual es un grafo acíclico donde cada nodo representa una variable y cada arco una dependencia probabilística. La construcción de este grafo posee dos etapas: aprendizaje estructural, donde se crea la topología del grafo que representa las relaciones de dependencia e interdependencia, y aprendizaje paramétrico, donde se obtienen las probabilidades a priori y condicionales.

Este clasificador se llama “naive” (ingenuo en inglés), debido a varias hipótesis simplificadoras para disminuir la cantidad ingente de parámetros en un modelo bayesiano clásico. Se supone que las variables predictoras son independientes entre sí, dada la variable clase. Por ejemplo, para considerar que un objeto es una manzana se espera que, sea roja, redonda y de alrededor 7 cm. Dado un clasificador de este tipo, que la manzana sea roja, es independiente de que sea redonda, y así, para la ausencia o presencia de todas las demás características. Este tipo de clasificador posee la ventaja de requerir poco entrenamiento para estimar los pocos parámetros que se necesitan (medias y varianzas de las variables) para clasificar.

Finalmente, el modelo de probabilidades en forma de grafo se combina con una regla de decisión que recoge la hipótesis del más probable (conocido como el máximo a posteriori o MAP), que decide la clase de las instancias de testeo. Esta regla es importante ya que decide la clase más probable dada una instancia de

testeo, y tiene la ventaja de que lo puede hacer de manera correcta más allá de que las estimaciones de probabilidad de las clases no sean del todo exactas.

J48:

Es una implementación de un clasificador basado en modelos de árbol [Salzberg, 1994] [James et al., 2013] [Kotsiantis, 2007]. Se crea un grafo con forma de árbol en el cual las hojas representan a las etiquetas de clase, y las ramas representan las conjunciones de las características que conducen a las etiquetas de clase. El objetivo es crear un árbol de decisión mínimo que, mediante la estructura de árbol, representa una hipótesis acerca de cómo clasificar futuros ejemplos. La creación del árbol mínimo es un problema de optimización NP-Hard, por lo tanto, se utiliza un método top-down que divide y conquista para realizar una búsqueda codiciosa (greedy) y obtener un árbol de decisión mínimo optimal. Este algoritmo selecciona una característica que genera subconjuntos de ejemplos similares en una clase, y así recursivamente, hasta que los ejemplos de los subconjuntos son todos de la misma clase o que la división en nuevos subconjuntos no aporta ganancia de información. En tal caso el subconjunto se vuelve un nodo hoja del árbol.

El clasificador J48 es una implementación del algoritmo de árbol de decisión C4.5, el cual, a su vez, es una extensión del algoritmo ID3. En el caso del algoritmo C4.5, en cada nodo se utiliza la medida de ganancia de información que compara la entropía actual contra la entropía de realizar distintas divisiones dados diferentes atributos. La división que menos entropía da y, por ende, aporta mayor ganancia de información es la que se utiliza para dividir el nodo en nuevos subconjuntos. La entropía es una medida de la incertidumbre de una colección de ejemplos. Dada la colección de ejemplos S , la entropía para una clasificación binaria como en Link Prediction es: $\text{Entropía}(S) = -p_1 \log_2 p_1 - p_0 \log_2 p_0$ siendo p_0 y p_1 las fracciones de ejemplos de cada clase.

Las ventajas de este tipo de algoritmo es que el árbol es fácil de interpretar y se pueden derivar reglas del mismo. Las desventajas son: el elevado tiempo de entrenamiento, un documento solo puede pertenecer a una rama a la vez, un error en una rama afecta a todo el subárbol inferior, y que puede sufrir de sobreajuste, Cómo el modelo no puede generalizar correctamente las clases, los datos de testeo se clasifican peor.

Random Forest

Se trata de un clasificador basado en bagging y modelos de árbol [Ho, 1995]. Bagging, también conocido como empaquetado en español, es un meta algoritmo de aprendizaje automático diseñado para mejorar la estabilidad y precisión de otros algoritmos de aprendizaje automático usados en clasificación o regresión. Con el empaquetado se obtiene una mayor cantidad de conjuntos de entrenamiento, mediante muestreo uniforme y reemplazo a partir del conjunto original, lo cual crea múltiples modelos de clasificación similares que existen a la vez. El objetivo es obtener y promediar múltiples modelos ruidosos, pero aproximadamente imparciales y, por lo tanto, reducir la varianza y evitar el sobreajuste. A la hora de clasificar un

ejemplo de testeo, el empaquetado obtiene los resultados de las distintas clasificaciones de sus modelos internos, y mediante votación, decide la clase final del ejemplo. Debido a todo lo anteriormente explicado, esta técnica puede ser utilizada con cualquier tipo de algoritmo clasificador como modelo interno y el bagging solo se encargará de la creación de los múltiples modelos de entrenamiento y de la votación para clasificar.

En el caso de Random Forest, es una combinación de árboles predictores tal que cada árbol depende de los valores de un vector aleatorio probado independientemente y con la misma distribución para cada uno. Es una modificación de bagging que construye un gran conjunto de clasificadores basados en árboles no correlacionados y luego los promedia.

Posee varias ventajas entre las cuáles se encuentran: ser uno de los clasificadores más certeros existentes si se utiliza un set de datos suficientemente grande, poseer buena eficiencia, estimar las variables importantes en la clasificación, estimar datos perdidos, detectar relaciones entre variables y la clasificación, entre otras. Como desventajas de este algoritmo, puede ocurrir que haya casos en los cuales, exista sobreajuste con datos muy ruidosos, se de ventaja a los grupos más pequeños de atributos, y que el modelo de clasificación es difícil de interpretar por el hombre.

IBk

Es un clasificador de tipo “lazy learner” (aprendiz perezoso en inglés) [Altman, 1992] [Kotsiantis, 2007]. Este tipo de clasificadores no construyen una representación explícita del conocimiento adquirido, sino que confían en las etiquetas de los ejemplos de entrenamiento similares a los ejemplos de testeo. La función solo se aproxima localmente y la mayor parte del procesamiento se realiza en la etapa de clasificación. La similaridad entre ejemplos se mide en relación a la distancia de los pesos.

IBk es una implementación del algoritmo clasificador K-NN (K-vecinos más cercanos). K-NN es un método no paramétrico que estima el valor de la función de densidad de probabilidad o la probabilidad a posteriori de que un ejemplo pertenezca a alguna de las clases. El modelo de clasificación consiste de un espacio p -dimensional donde p es la cantidad de variables predictoras. Los ejemplos de entrenamiento son considerados vectores característicos en este espacio multidimensional, y los valores de cada característica, determinan la ubicación del ejemplo en el espacio. En la etapa de aprendizaje sólo se almacenan los vectores característicos y las etiquetas de clase de los ejemplos de entrenamiento. En cambio, en la etapa de clasificación, a cada ejemplo de testeo (que también se representan mediante vectores característicos), se les calcula la distancia euclídea contra los vectores almacenados, y se toman en cuenta los k vecinos más cercanos. Cada ejemplo es clasificado de acuerdo a la etiqueta de clase más repetida entre sus k vecinos más cercanos. Debido a esto, el k debe ser determinado experimentalmente, ya que para cada dataset único, el k que mejor clasifica es diferente, y en un principio, desconocido. Un k más grande tiende a reducir el efecto del ruido en la clasificación, pero crea límites entre clases

parecidas.

Este clasificador posee la ventaja de un entrenamiento simple y de no necesitar mucha cantidad de ejemplos para ser efectivo. Como desventajas se pueden enumerar que: la etapa de clasificación requiere mucho procesamiento, características que no son relevantes pueden atenuar el efecto de las características que si son relevantes para la clasificación y que el K debe seleccionarse experimentalmente.

SMO

Es un clasificador lineal [Platt, 1998] [Nasa y Suman, 2012] [Kotsiantis, 2007]. Se utiliza la combinación lineal de las características de un objeto para realizar la clasificación. Los ejemplos de entrenamiento son interpretados como vectores p-dimensionales donde cada variable predictora ubica al ejemplo cómo un punto en el espacio, y la variable de clase, indica la clase del punto. Por ser lineal, el clasificador busca un hiperplano que separe de forma óptima a los puntos de una clase de la otra. Para una óptima clasificación, el hiperplano debe tener la distancia máxima (margen) a los puntos de cada clase más cercanos a él mismo. Así, los puntos de cada clase quedan en lados distintos del hiperplano. A los vectores formado por los puntos más cercanos al hiperplano se los conoce como vectores de soporte.

Las máquinas de vectores de soporte (SVMs) son clasificadores lineales que buscan calcular los vectores de soporte durante el entrenamiento, para obtener el hiperplano y realizar una efectiva clasificación. Como tal, el problema del cálculo de vectores de soporte, es un problema de optimización ya que se busca que los mismos posean margen máximo. Para que las SVMs se usen en casos reales, donde la curva de separación entre ejemplos tiende a no ser lineal, se utiliza una función kernel. Ésta se encarga de proyectar los ejemplos que no son separables linealmente, a un espacio de mayor dimensión, donde si son separables linealmente y el criterio de las SVMs es aplicable.

SMO es un algoritmo para resolver el problema de programación cuadrática que surge al entrenar una SVM. Se divide el problema de optimización en subproblemas más pequeños de optimización, y así recursivamente.

ZeroR

Es el método de clasificación más simple que solo se basa en la variable de clase e ignora las variables predictoras [Nasa y Suman, 2012]. Simplemente predice la clase que es mayoría, y debido a esto, no posee poder predictivo. Su utilidad es marcar un punto de partida para el desempeño de los demás clasificadores. Es el método más impreciso de clasificación, y si una clasificación mediante otro método posee un rendimiento aún peor, entonces ese método es totalmente inútil. El funcionamiento es simple, se cuentan la cantidad de ocurrencias de cada clase en el entrenamiento. En la clasificación todos los ejemplos pertenecen a la clase que poseía más ocurrencias en el entrenamiento.

Existen diversas técnicas de tratamiento de los datos para su uso y validación. Cada una es utilizada con fines distintos y poseen distintas características que deben ser tenidas en cuenta.

Para poder construir un clasificador a partir de un determinado algoritmo, es necesario disponer de una determinada cantidad de instancias. De esta forma, el clasificador puede “aprender” a clasificar nuevas instancias a partir de los datos ya cargados. Pero para poder evaluar su funcionamiento, también es necesario disponer de datos que serán utilizados como prueba, con el fin de validar el clasificador y poner en números sus clasificaciones.

En algunos casos se disponen de datos específicamente pensados para la evaluación de un clasificador, pero cuando no se tiene esta información se deben aplicar algunas técnicas que permiten entrenar y evaluar un clasificador a partir de un único conjunto de datos. Según [Novak y Novotny, 2010] y [Refaeilzadeh et al., 2008], es posible llevar adelante este proceso a través de dos técnicas conocidas: Holdout (Percentage Split) y K-Fold Cross Validation.

Percentage Split, o en español “división porcentual”, es la técnica más sencilla y consiste en dividir aleatoria y porcentualmente el conjunto de datos en dos subconjuntos complementarios, de forma que pueda realizarse el análisis a partir de uno de ellos (datos de entrenamiento), y validar dicho análisis a partir del otro subconjunto (datos de prueba). Por lo general, se suele utilizar el subconjunto más grande para el entrenamiento, dejando un pequeño porcentaje de los datos para la evaluación.

Este método es muy rápido computacionalmente, pero no es demasiado preciso. Debido a que, la función de aproximación se ajusta al conjunto de datos destinados para entrenamiento, existe una variación de resultados obtenidos para diferentes datos de entrenamiento. La evaluación por su parte, puede depender en gran medida de cómo es la división entre los datos de entrenamiento y de prueba, por lo que, los resultados de la misma pueden ser significativamente diferentes dependiendo de cómo se realice esta división.

Para evitar la dependencia de los resultados sobre la división de los datos, se puede utilizar la técnica de validación cruzada (cross-validation en inglés). La misma garantiza que la evaluación de los resultados es independiente de la partición entre datos de entrenamiento y prueba. La validación cruzada consiste en entrenar y evaluar los clasificadores realizando diferentes particiones sobre el conjunto de datos. Resulta útil para entornos donde el objetivo principal es la predicción, y se quiere estimar qué tan preciso es un modelo que se llevará a cabo a la práctica.

Existen diferentes tipos de validaciones cruzadas. Los más populares son la validación cruzada de K iteraciones, la validación cruzada aleatoria y la validación cruzada dejando uno fuera.

En la validación cruzada de K iteraciones, también conocida como K-fold cross-validation en inglés, los datos se dividen en K subconjuntos. Uno de los subconjuntos es utilizado como datos de prueba, mientras que el K-1 restante de los subconjuntos se utiliza para entrenamiento. Este proceso se repite durante K iteraciones, alternando con cada uno de los subconjuntos de prueba posible.

Finalmente, para obtener un único resultado, se realiza la media aritmética de los resultados de cada iteración. Si bien este método es sumamente preciso, puede resultar lento desde el punto de vista computacional, dependiendo de la cantidad de datos que se disponen y de la cantidad de iteraciones a realizar. El número de iteraciones más conveniente depende del tamaño del conjunto de datos, pero en la práctica suele utilizarse la validación cruzada de 10 iteraciones, conocido como 10-fold cross-validation. Como ejemplo, la Figura 2 muestra una validación cruzada con $K = 4$, donde se observan 4 iteraciones, en las cuáles van variando los datos de prueba que se seleccionan.

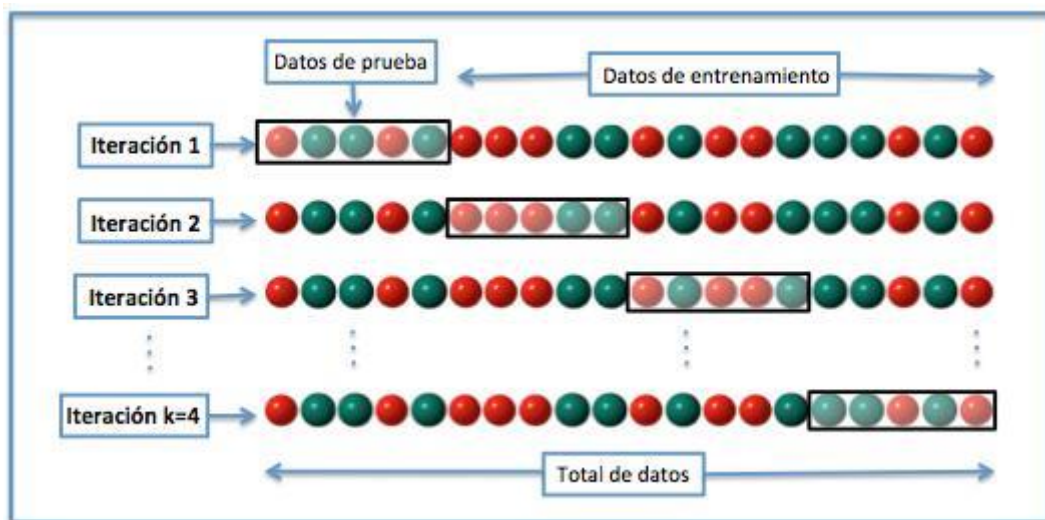


Figura 2 - Validación cruzada con 4-folds

La validación cruzada aleatoria consiste en dividir aleatoriamente el conjunto de datos de entrenamiento y de prueba. En cada división, se ajusta la función de aproximación a partir del conjunto de datos de entrenamiento, y se calcula los valores de salida para el conjunto de datos de prueba. El resultado final se calcula a partir de la media aritmética de cada uno de los valores obtenidos en las diferentes divisiones. A diferencia de la validación cruzada de K iteraciones, en este caso la división de datos de entrenamiento y prueba no depende del número de iteraciones, pero estos subconjuntos se pueden solapar. Esto se debe a que, hay algunas muestras que quedan sin evaluar y otras que se evalúan más de una vez. En la Figura 3 se puede observar lo anteriormente dicho de forma gráfica.

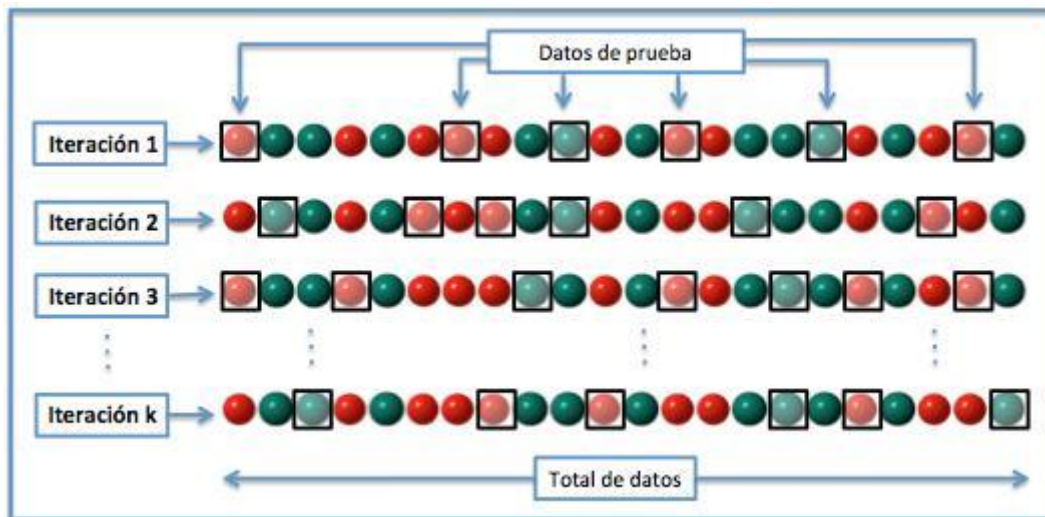


Figura 3 - Validación cruzada aleatoria

La validación cruzada dejando uno fuera por su parte, implica separar los datos para cada iteración, considerando una sola muestra para los datos de prueba, y utilizando todo el resto de los datos como entrenamiento, como se visualiza en la Figura 4. La ventaja de este tipo de validación cruzada es que el error es muy bajo. Sin embargo, el costo computacional que implica es considerablemente elevado, ya que es necesario realizar una gran cantidad de iteraciones (tantas como N muestras se tengan), y analizar para cada una los datos de entrenamiento y de prueba.

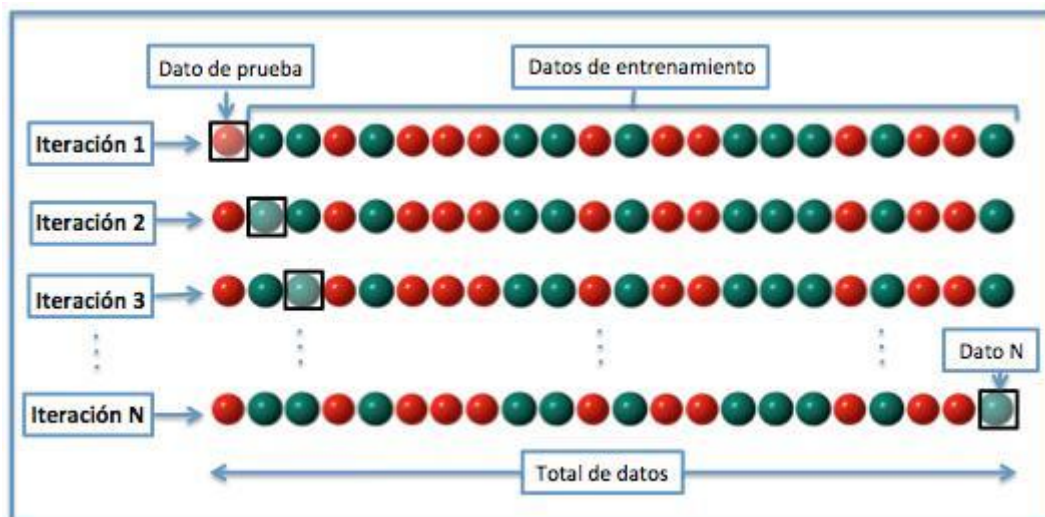


Figura 4 - Validación cruzada dejando uno fuera

En resumen, las técnicas de validación cruzada son más adecuadas a la hora de estimar la precisión de los modelos cuando el objetivo principal es la predicción, pero implican un costo computacional mucho mayor que la técnica de división porcentual.

2.1.7. Métricas

Es importante conocer el desempeño de un clasificador y poder plasmarlo en unidades numéricas de distintas métricas, que miden diferentes características de una clasificación. Estas métricas permiten medir de cierta manera, características generales y comunes a cualquier clasificación, permitiendo realizar una comparación objetiva entre distintos experimentos, sin importar el clasificador, el dataset ni la forma de validación de los datos que se haya usado en cada uno. De esta forma, se pueden realizar ciertos ajustes a la hora de la experimentación y esperar que al configurar de distinta manera ciertos parámetros, el cambio se refleje en las distintas métricas, ya sea de forma positiva o negativa. [Nasa y Suman, 2012] Como se explicó anteriormente, el resultado de un clasificador puede ser un número real (valor continuo), o puede ser un resultado discreto como, por ejemplo, una etiqueta que indica una de las clases. El problema particular de la predicción de enlaces, es un caso de predicción de clases binario, donde los resultados se etiquetan como positivos o como negativos, según la existencia o no de un enlace entre cada par de nodos.

Algunas métricas básicas son descritas en [Nasa y Suman, 2012] y [Fawcett, 2006]. Estas permiten llevar a cabo diferentes análisis de los resultados, por lo que, es importante entender qué representan sus valores. A continuación, se detallan las más importantes que serán tenidas en cuenta en este trabajo.

Matriz de Confusión

En estos tipos de clasificadores binarios, existen cuatro posibles resultados. Cuando la predicción del clasificador es positiva, y el valor real también lo es, se dice que el resultado es un Verdadero Positivo. En cambio, si el valor real es negativo, se conoce como Falso Positivo. Del mismo modo, cuando tanto la predicción y el valor real son negativos, se tiene un Verdadero Negativo, mientras que, si el valor real es positivo, entonces el resultado se conoce como Falso Negativo.

Estos resultados pueden ser representados mediante una herramienta llamada matriz de confusión, la cual facilita la visualización del desempeño del algoritmo de aprendizaje supervisado. En la misma, cada columna representa a las predicciones de cada clase, mientras que cada fila representa los valores reales de la clase. Esta distribución no es estricta, por lo que, se pueden encontrar matrices donde las columnas y las filas son dispuestas de forma inversa a lo mencionado. Un ejemplo de matriz de confusión que refleje los resultados de una clasificación binaria se visualiza en la Tabla 1.

		Valor predicho	
		Positivo	Negativo
Valor en la	Positivo	Verdaderos Positivos	Falsos Negativos

realidad	Negativo	Falsos Positivos	Verdaderos Negativos
----------	----------	------------------	----------------------

Tabla 1 - Matriz de confusión para clasificación binaria

A partir de estos resultados, es posible calcular diferentes métricas que permitan evaluar el desempeño del clasificador en distintos aspectos.

Precisión y Recall

La precisión de un clasificador consiste en el porcentaje de aciertos respecto del total de instancias que se clasificaron como positivas. En la predicción de enlaces, puede verse como la fracción de enlaces predichos correctamente sobre el total de predicciones que arrojaron la existencia del enlace como resultado, incluyendo también aquellas cuyo valor real es la inexistencia del enlace. Esta métrica puede calcularse a partir de los valores de la matriz mediante la siguiente fórmula:

$$\text{Precisión} = \text{Verdaderos Positivos} / (\text{Verdaderos Positivos} + \text{Falsos Positivos})$$

Nótese que en esta métrica no se consideran aquellos enlaces reales que no fueron predichos como tal (Falsos Negativos), por lo que, realizar predicciones incorrectas sobre enlaces reales no afecta al resultado de la misma.

El recall, también conocido como sensibilidad, es el porcentaje de aciertos respecto del total de instancias positivas reales que se clasificaron. En este caso, la fracción consiste en los enlaces predichos de forma correcta, sobre el total de enlaces reales existentes dentro del conjunto de instancias que se busca clasificar. Esta métrica se calcula de la siguiente manera:

$$\text{Recall} = \text{Verdaderos Positivos} / (\text{Verdaderos Positivos} + \text{Falsos Negativos})$$

Mientras que la precisión responde a la pregunta “¿Qué porcentaje de las predicciones de enlaces fueron correctas?”, el recall responde al interrogante “¿Qué porcentaje de los enlaces reales se predijeron correctamente?”. La precisión puede ser vista como una medida de exactitud o calidad, mientras que el recall es una medida de completitud o cantidad.

Estas dos métricas ayudan a evaluar el desempeño de un clasificador con respecto a la predicción de enlaces. Sin embargo, con lo mencionado hasta el momento, ninguna de ellas considera el desempeño con respecto a la predicción para aquellos pares de nodos que no son enlace. Para ello, es posible calcular de forma análoga las mismas métricas, considerando los enlaces negativos en lugar de los positivos. De esta forma, ambas métricas pueden evaluar el desempeño de la predicción de los “no enlaces”, a través de los siguientes cálculos:

$$\text{Precisión} = \text{Verdaderos Negativos} / (\text{Verdaderos Negativos} + \text{Falsos Negativos})$$

$$\text{Recall} = \text{Verdaderos Negativos} / (\text{Verdaderos Negativos} + \text{Falsos Positivos})$$

F-Measure

Una forma de combinar ambas métricas, es a través de la media armónica de ambos valores, conocido como F-Measure. Esta es una medida de exactitud de una prueba, que se puede calcular para cada clase de un clasificador binario. Como en la predicción de enlaces las clases corresponden a si existe enlace o no, el F-measure puede considerarse como positivo y negativo respectivamente. La fórmula no varía, sólo cambia que se utiliza la precisión y recall de cada clase.

La métrica F-Measure tradicional o balanceada se conoce como “F₁ Score”, se basa en los valores de precisión y recall de una determinada clase, y se calcula de la siguiente manera:

$$F_1 = 2 \cdot \frac{1}{\frac{1}{\text{recall}} + \frac{1}{\text{precision}}} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Promedio ponderado

Si bien los valores de precisión y recall se calculan para uno de los valores (positivos o negativos), también se puede calcular un promedio ponderado de ambas métricas, conocido como “Weighted Average”.

El promedio ponderado en general, es una medida más exacta para obtener resultados de diversos datos con diferentes grados de importancia. En el caso de los clasificadores binarios, este valor se calcula a partir de ponderar las métricas obtenidas para cada clase, respecto del total de instancias de esa clase en el conjunto de instancias de prueba. El cálculo consiste en multiplicar las métricas obtenidas para ambas clases por la cantidad de instancias de la clase correspondiente en el conjunto de prueba, sumar ambos valores y dividir el total de instancias de prueba por dicho valor.

Este promedio es útil para analizar los resultados de métricas como la precisión y el recall, de tal forma que se consideren los resultados para cada clase ponderados por la cantidad de instancias incluidas en la evaluación, de forma que se obtenga una métrica de desempeño del clasificador más general. De todas formas, se debe considerar que con este promedio no se está midiendo la precisión ni el recall para la predicción de enlaces propiamente dicho, sino que se considera el desempeño de la predicción tanto para existencia como para la ausencia de enlaces.

Curva ROC

Debido a que, todas estas métricas consisten en diferentes cálculos a partir de los valores de la matriz de confusión, es esta última herramienta la que contiene mayor información para analizar el desempeño general del clasificador. De todas formas, existe una forma más genérica para analizar el desempeño y es a través de lo que se llama Curva ROC.

Las siglas ROC provienen del acrónimo “Receiver Operating Characteristic” (Característica Operativa del Receptor), y consiste en una representación gráfica de la Razón de Verdaderos Positivos (VPR) frente a la Razón de Falsos Positivos

(FPR). La VPR mide hasta qué punto un clasificador es capaz de detectar los casos positivos correctamente, de entre todos los casos positivos de la prueba, es equivalente a la sensibilidad y se representa en el eje Y, mientras que la FPR define cuántos resultados positivos son incorrectos, de entre todos los casos negativos disponibles durante la prueba, es igual a 1-especificidad, y se representa sobre el eje X. Cada resultado de predicción o instancia de la matriz de confusión, representa un punto en el espacio ROC. Gráficamente posee la forma que se observa en la Figura 5.

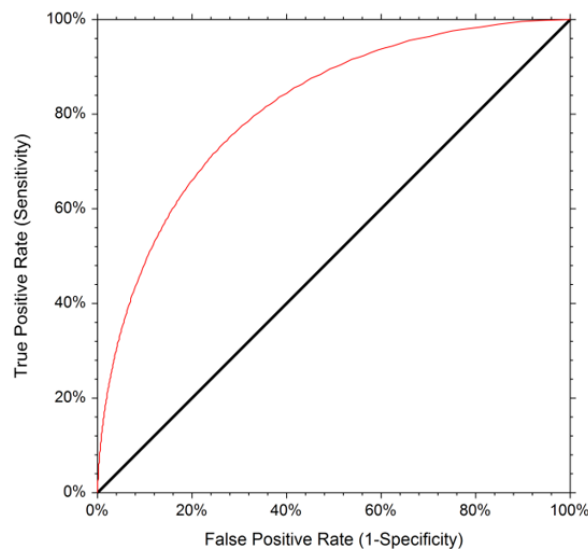


Figura 5 - Curva ROC

El punto situado en la esquina superior izquierda del espacio ROC, es llamado clasificación perfecta, ya que representa al mejor método posible de predicción, con un 100% de sensibilidad y especificidad (ningún falso negativo ni falso positivo). Una línea diagonal desde el extremo inferior izquierdo hasta el superior derecho por su parte, representa a una clasificación totalmente aleatoria, y es conocida como la línea de no-discriminación. Esta diagonal divide el espacio de forma tal que los puntos por encima de la diagonal representan buenos resultados de clasificación (mejor que el azar), y los puntos por debajo los resultados pobres. De todas formas, si la salida de un predictor es sumamente pobre, podría ser invertida para conseguir un buen predictor, por lo que, en definitiva, los peores resultados son los que arrojan curvas más cercanas a la diagonal.

El análisis de la curva ROC ayuda a seleccionar los modelos posiblemente óptimos y descartar los subóptimos, independientemente de la distribución de las clases en la población. Esta última característica hace que la curva ROC sea de suma utilidad en el análisis de desempeño de un clasificador para la predicción de enlaces, ya que, en la mayoría de las redes, las clases suelen estar distribuidas de forma desigual, con un alto porcentaje de pares de nodos negativos, y un pequeño porcentaje de instancias positivas.

Área bajo la curva ROC

A partir de la curva ROC, se puede generar un indicador muy utilizado en la comunidad de aprendizaje para la comparación de modelos llamado área bajo la curva ROC, o también conocido como “AUC” debido a las siglas en inglés de “Area Under Curve”. Este índice se puede interpretar como la probabilidad de que un clasificador ordenará o puntuará una instancia positiva elegida aleatoriamente más alta que una negativa. Debido a que, esta métrica engloba el desempeño del clasificador tanto para los casos positivos como los casos negativos, y a su vez es independiente de la distribución de clases en la población, es posible utilizar su valor para realizar comparaciones generales entre diferentes clasificadores y determinar cuál modelo obtuvo efectivamente mejores resultados.

Otras métricas

Existe una gran variedad de métricas además de las ya mencionadas, que permiten evaluar diferentes aspectos de una clasificación. Algunas de las más destacadas son el Coeficiente Kappa, la Media de error absoluto, la Raíz del error cuadrático medio, el Error absoluto relativo y la Raíz del error cuadrático relativo.

Kappa es una medida que refleja el acuerdo entre las clasificaciones y los valores reales de las clases. En una evaluación de un clasificador, se pueden ver las predicciones y los valores reales como dos observaciones. De esta forma, la métrica se calcula tomando el acuerdo observado menos el acuerdo esperado por el azar, y dividiendo dicho resultado por el máximo acuerdo posible. La ecuación es la siguiente:

$$\kappa = \frac{\text{Pr}(a) - \text{Pr}(e)}{1 - \text{Pr}(e)}$$

Donde $\text{Pr}(a)$ es el acuerdo observado relativo entre los observadores y $\text{Pr}(e)$ es la probabilidad hipotética de acuerdo por azar. Si los evaluadores están completamente de acuerdo (las clasificaciones y los valores reales coinciden), entonces κ será igual a 1, mientras que, si no hay acuerdo entre los clasificadores distinto al que cabría esperar por azar (definido por $\text{Pr}(e)$), entonces κ será igual a 0.

Las demás métricas mencionadas están relacionadas con un concepto llamado “correlación”. Este indica que tan relacionados están los valores reales (denotados con el símbolo θ), y los valores estimados al usar algún algoritmo (representado como $\hat{\theta}$ con circunflejo). Los valores posibles de correlación van desde -1 hasta 1, donde 0 significa que no hay relación, 1 implica una relación muy fuerte, y -1 es una relación linealmente inversa.

Las ecuaciones de las métricas de correlación son las siguientes:

- Media de error absoluto:

$$MSE = \frac{1}{N} \sum_{i=1}^N |\hat{\theta}_i - \theta_i|$$

- Raíz del error cuadrático medio:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{\theta}_i - \theta_i)^2}$$

- Error absoluto relativo:

$$RAE = \frac{\sum_{i=1}^N |\hat{\theta}_i - \theta_i|}{\sum_{i=1}^N |\bar{\theta} - \theta_i|}$$

- Raíz del error cuadrático relativo:

$$RRSE = \sqrt{\frac{\sum_{i=1}^N (\hat{\theta}_i - \theta_i)^2}{\sum_{i=1}^N (\bar{\theta} - \theta_i)^2}}$$

Todas estas estadísticas comparan a los valores reales con sus estimaciones, pero de una forma ligeramente diferente. Indican cuán alejados están los valores estimados del valor real de Theta. En algunos casos se utiliza la raíz cuadrada y en otros el valor absoluto debido a que, cuando se hace uso de la raíz cuadrada, los valores extremos tienen más influencia en el resultado.

En MSE y RMSE, se mira la diferencia promedio entre ambos valores, y deben ser interpretados en comparación con la escala de la variable. Por ejemplo, un MSE de 1 punto indica una diferencia de 1 entre Theta y Theta circunfleja.

En RAE y RRSE, se dividen dichas diferencias por la variación de Theta para que tengan una escala de 0 a 1, y al multiplicar este valor por 100 se obtiene la similitud en una escala de 0 a 100 (porcentual). Los valores de los numeradores en ambas ecuaciones indican cuánto difiere Theta de su valor medio. Estas dos estadísticas arrojan resultados relacionados con la escala de Theta, y es por este motivo que, son llamadas “relativas”.

2.2. Trabajos relacionados

La predicción de enlaces en redes sociales ha sido un problema estudiado por muchos investigadores en los últimos años, dado el auge de las redes y de la gran utilidad asociada al procesamiento de los datos que fluyen en las mismas, como se explicó anteriormente. Existen muchas investigaciones en las que se han utilizado múltiples enfoques para atacar el problema de la predicción de enlaces. Este trabajo utilizó como fuente principal de información cuatro publicaciones distintas, que encaran el problema de la predicción de enlaces desde distintos puntos de vista, pero sin alejarse tanto del enfoque propuesto para esta investigación.

“Link Prediction in Social Networks: the State-of-the-Art” [Wang et al., 2015] es una investigación que se centra en analizar el estado del arte en lo que a predicción de enlaces se refiere. Se analizan varias problemáticas relacionadas, distintas técnicas para resolverlas y varias aplicaciones posibles en el mundo real. El trabajo enuncia los desafíos inherentes a la hora de analizar las redes sociales, dinamismo e incompletitud, y la importancia de la predicción de enlaces para conocer la evolución futura de una red social. Por ejemplo, esta técnica es útil para sistemas de recomendación, como puede ser recomendar nuevos amigos en redes sociales, o productos a comprar, en una página de venta online. Sirve para encontrar enlaces ocultos en una red conocida de forma parcial, o hasta es aplicable en bioinformática y biotecnología, en redes de expresiones de genes o de relaciones entre proteínas. A su vez, se realiza un análisis de las investigaciones realizadas en los últimos años.

Se visualiza y explica el framework general para predecir enlaces, donde se compara, mediante módulos, los distintos enfoques para resolver este tipo de problemas. De manera similar, se relacionan las distintas técnicas de predicción, agrupadas en categorías, junto a las distintas técnicas de aprendizaje, a su vez relacionadas con los distintos tipos de problemas que engloba esta temática.

Se realiza un repaso breve sobre las posibles técnicas que pueden ser utilizadas a la hora de predecir enlaces, agrupándolas en cuatro categorías diferentes:

- Métricas basadas en nodos, las cuales utilizan la información individual de cada nodo, y supone que, cuanto más similar sea el par, más probable es que exista enlace.
- Métricas topológicas, que aprovechan la estructura de grafo mediante la cual se representan las redes, a su vez divididas en métricas basadas en vecinos, en caminos, y en caminos aleatorios.
- Métricas basadas en redes sociales, que utilizan conceptos como homofilia, balance estructural, lazos fuertes o débiles, entre otros.
- Métodos basados en aprendizaje: basados en las métricas básicas anteriores, pueden tratarse de clasificadores basados en características, modelos de grafo probabilísticos o matrices de factorización.

A su vez existen varias categorías de problemas dentro de esta temática. La predicción de enlaces temporal busca predecir enlaces que existan en algún momento determinado. La predicción en redes heterogéneas tiene en cuenta que pueden existir varios tipos de nodos y enlaces, por lo tanto, se toman diferentes consideraciones que dependen del tipo de nodo o enlace que se analice. La predicción con enlaces activos e inactivos, considera que el momento de la creación de un enlace es importante, y cuanto más jóvenes son, más influyen en el futuro de la red. En la predicción en redes de dos vías existen dos tipos de nodos que se relacionan de manera indiferente, por ejemplo, personas y productos en sitios de compra-venta online. La predicción para enlaces que desaparecen, considera que, no solo en la evolución de la red pueden aparecer nuevos enlaces, sino que pueden

desaparecer los existentes. Y finalmente, tener en cuenta la escalabilidad de las técnicas es un problema en sí mismo, debido a que, las redes pueden llegar a poseer miles de millones de nodos, lo cual limita y/o modifica las técnicas a utilizar.

El trabajo realiza un análisis de las aplicaciones posibles de la predicción de enlaces, entre las que se encuentran recomendación en redes sociales, predicción de relaciones recíprocas, completitud de una red, entre otras. Además, se describen varios grupos activos de investigación y los futuros desafíos que deben ser abordados por la predicción de enlaces.

Este paper ha sido de gran ayuda para determinar el contexto general de la temática y situar este trabajo en el universo de la predicción de enlaces.

“Exploiting Place Features in Link Prediction on Location-based Social Networks”

[Scellato et al., 2011]. En este estudio realizado en la Universidad de Cambridge, se utilizaron técnicas de aprendizaje supervisado para resolver el problema de predicción de enlaces. La particularidad del mismo consiste en que, además de incluir características sociales de la red para el entrenamiento, se tuvieron en cuenta también características basadas en las ubicaciones registradas por los usuarios.

La información utilizada para sus análisis fue extraída de una red social llamada “Gowalla”, lanzada en 2007 y cerrada en 2012. Los usuarios de esta red tenían la posibilidad de agregar amigos y compartir con estos los lugares en los cuales habían estado.

Se llevó a cabo la recolección de una gran cantidad de información de esta red, durante diferentes instantes en su evolución, con datos sobre lugares, usuarios, sus amigos y las ubicaciones registradas. A partir de analizar cómo las nuevas amistades son creadas a través del tiempo, se encontraron que alrededor de un 30% de los nuevos enlaces fueron entre personas que registraron una misma ubicación, reflejando la importancia de esta característica en la predicción de enlaces.

Se detectó que, al considerar los pares de usuarios en su totalidad, la distribución de clases resulta extremadamente sesgada. Esto a su vez implica que se intentarán predecir enlaces que probablemente nunca creen un vínculo entre sí. Por lo tanto, se decidió considerar únicamente aquellos pares que son amigos-de-amigos y amigos-de-lugares como dos conjuntos distintos, reduciendo 15 veces el total de pares de nodos, y pudiendo predecir aun así dos tercios de los nuevos enlaces. El primer conjunto comprende simplemente a aquellos con quienes comparte al menos una amistad sin estar conectados directamente. El segundo conjunto en cambio, corresponde a los usuarios que han registrado al menos un lugar en común pero no están conectados entre sí.

Considerando las características sociales, basadas en las ubicaciones, y un conjunto de características globales a ambas, se realizaron comparaciones entre las predicciones realizadas con cada una de ellas en forma individual, y las realizadas a través de algoritmos de aprendizaje supervisado. Se utilizó WEKA para realizar las evaluaciones, y se consideró la curva ROC como la principal herramienta para

evaluar el desempeño de las predicciones, siendo la precisión y el recall otras métricas importantes a considerar.

Para cada característica individual, se computaron puntajes de predicción correspondientes a cada par de usuarios desconectados, y se ordenaron numéricamente creando un ranking de candidatos basados en dicho puntaje.

En lo que respecta al aprendizaje supervisado, se utilizaron los siguientes clasificadores debido a su popularidad, siempre aplicando 10-fold cross validation: J48, Naive Bayes, Model Tree con regresión lineal en sus hojas, y Random Forest. Con esta técnica se obtuvieron mejores resultados, siendo Model Tree y Random Forest los más efectivos de los clasificadores para estos datasets. Esto se debió a que los mismos son capaces de aprovechar la información contenida en las características de las predicciones.

Una particularidad adicional que se analiza en el experimento, es que, con los métodos de predicción de enlaces estándar basados en características sociales, es imposible realizar predicciones para usuarios que todavía no concretaron ninguna amistad. Sin embargo, con el framework de aprendizaje supervisado descrito en la investigación, es posible predecir nuevos vínculos sociales incluso para usuarios que todavía no tienen ninguna amistad, siempre que hayan visitado y registrado lugares.

“Influence based Link Prediction using Supervised Learning” [Beladia y Vats, 2011]. Esta investigación está enfocada en cómo un nodo de una red influencia a otro, y cómo puede ser utilizada la influencia para la predicción de enlaces.

Los estudios fueron realizados sobre la red de un importante sitio de música llamado “Last.fm”. Si bien la misma cuenta con más de 40 millones de usuarios, se limitó el conjunto de datos a aproximadamente 10.000 usuarios de España y Reino Unido.

En lo que respecta a la información disponible sobre los usuarios, los investigadores contaron con los nombres, edades, domicilios, y datos relacionados con la actividad como el historial de reproducción y la lista de amigos.

Se seleccionó un conjunto de características apropiadas para llevar a cabo las predicciones a través de diferentes algoritmos de aprendizaje, considerando que cada observación representa el vínculo entre un par de nodos. Estas fueron divididas en tres grupos:

- Características basadas en la influencia: un nodo es influenciado por otro cuando el primero realiza la misma acción que realizó el segundo en un lapso corto de tiempo. En esta red en particular, se estableció que la acción de un usuario consiste en escuchar una pista de música.
- Características basadas en la similitud: se seleccionaron características que representan la similitud entre usuarios y que han demostrado ser eficientes en otras investigaciones. Las seleccionadas se basan en la proximidad del género de la música que escuchan dos usuarios, y la similitud de los intereses musicales en sus listas de reproducción, de su lista de amigos, de sus álbumes y de sus artistas.

- Características topológicas: estas fueron consideradas como unas de las más significantes en la predicción de enlaces según otras investigaciones. Para este estudio se consideró Shortest Path, que indica el camino más corto entre un par de nodos.

En cuanto a los algoritmos de aprendizaje automático para predecir si un par de usuarios están conectados o no, se tuvieron en cuenta Kernel SVM, Random Forest, Gradient Boosted Machines, Logistic Regression y AdaBoost. En lugar de utilizar las implementaciones de Weka, en este trabajo se utilizaron los paquetes de R para la construcción de los modelos y sus evaluaciones.

Para evaluar el desempeño de los clasificadores, se consideraron diferentes métricas como el área bajo la curva ROC (AUC), la precisión y el recall. También se tuvo en cuenta F-Measure, que considera las dos últimas métricas en conjunto.

Los resultados generales de los modelos para esta investigación fueron buenos, superando en todos los casos una exactitud por encima del 80%. Una característica particular es que, debido a la naturaleza de Last.fm donde algunos usuarios pueden estar conectados por razones que no son capturadas por las características (por ejemplo, por conocidos o compañeros de clase), los modelos arrojan más falsos negativos que falsos positivos. Esto se ve reflejado en valores más altos para la precisión que los valores de recall.

En lo que respecta a la selección de las características a considerar en los datasets, se utilizaron los algoritmos de Forward Stepwise Rank, Backward Stepwise y All Subset. Estos algoritmos permiten elegir un subconjunto de características que minimice el error de la función para el modelo utilizado. Para el dataset en cuestión, se encontró que Shortest Path es la característica más significativa de las topológicas, y la influencia de los vecinos en común entre dos nodos es la más significativa de las características basadas en la influencia.

“Finding Experts by Link Prediction in Co-authorship Networks.” [Pavlov y Ryutaro, 2007]. Esta investigación analiza las relaciones entre investigadores cuando realizan colaboraciones de forma conjunta. Mediante el uso de aprendizaje supervisado sobre redes en las cuales los nodos son investigadores y los enlaces las colaboraciones, la investigación busca predecir nuevas colaboraciones entre investigadores.

La motivación está dada por la difícil tarea que recae en las manos de los investigadores a la hora de decidir quiénes pueden ser los más indicados para realizar una colaboración, más cuando pertenecen a distintas disciplinas. Con esto se busca que los equipos colaborativos sean más organizados y sinérgicos, y que, en consecuencia, produzcan trabajos de investigación de mayor calidad.

Dada una red de investigadores representada mediante un grafo con pesos, se calculan distintas características de grafos para cada par de nodos, y con esa información, crear un vector de características. En el trabajo en cuestión se eligieron las siguientes características por ser de utilidad en una gran variedad de redes: Shortest Path, Common Neighbours, Jaccard's Coefficient, Adamic Adar,

Preferential Attachment, Katz, Weighted Katz, PageRank (min), PageRank (max), SimRank y Link value (que corresponde al peso). Una vez creado el vector de características para todos los pares de nodos, el mismo es utilizado para entrenar y testear distintos algoritmos clasificadores (llamados “predictores” en la investigación): SMO, J48, DecisionStump. Se utiliza además AdaBoost en combinación con los anteriores clasificadores para comparar.

En la experimentación se utilizó una red de colaboraciones de investigadores japoneses pertenecientes al IEICE, que contiene 110210 artículos publicados de 86696 autores desde 1993 hasta 2006. En el grafo, cada nodo, es un autor, y cada enlace significa, que dos autores colaboraron al menos una vez. En consecuencia, el peso de los enlaces representa la cantidad de colaboraciones entre dos autores.

Los 14 años de historia se dividieron en dos partes, de 7 años cada una, en las que se realizaron los mismos experimentos. A cada parte, se le filtraron los enlaces con peso 1, y los nodos que no hayan estado presentes durante los 7 años enteros. Con los nodos y enlaces restantes se calcularon las características de similitud (de los datos de los primeros 4 años), y la etiqueta de clase (de los datos de los restantes 3 años), para crear el vector de características. De las instancias resultantes se eliminaron aquellas consideradas repetidas (ninguno de sus atributos difiere por más de 0.001). Para la primera parte quedaron 29437 instancias únicas de las cuáles 1.6% poseían enlace. De la segunda parte quedaron 1.6 millones de instancias únicas de las cuáles 0.019% poseían enlace.

En la etapa de clasificación se utilizaron las herramientas de Weka que poseen las implementaciones de los clasificadores. Se utilizó el 90% de las instancias para entrenamiento y el 10% restante para testeo, repartidas de forma aleatoria. Para la comparación entre los rendimientos de las distintas clasificaciones se utilizó la medida de recall.

Para la primera parte SMO predice el 100% de los links bien debido principalmente a las métricas Katz y Preferential Attachment a las que le asigna mucho peso, dándole poca importancia a las demás. Ada/SMO y J48 le siguen en eficiencia, y como peores opciones quedan Ada/J48 y Ada/DecisionStump. Para la segunda parte, la tendencia se mantiene a excepción de que Ada/DecisionStump mejora a Ada/J48. Sin embargo, Ada/SMO no pudo ser calculado por la cantidad de memoria excesiva que se necesitaba.

Esta investigación es muy similar al presente trabajo en cuanto a problemática, técnicas y procesos. Demostró ser de mucha utilidad, ya que es un muy buen ejemplo de cómo realizar un trabajo de este tipo, ya sea en el pre-procesamiento, elección de funciones de similitud, elección de clasificadores, experimentación y demás cuestiones.

3. Desarrollo de la propuesta

En este capítulo se plantea la propuesta que permitirá llevar adelante la investigación. Previo al análisis de la predicción de enlaces, es necesario realizar un estudio sobre la herramienta que se utilizará para modelar diferentes redes. Una vez realizado esto, se debe comprender el diseño de la misma, con el fin de realizar las adaptaciones y extensiones necesarias para realizar los distintos experimentos. Además, se plantea utilizar técnicas adicionales, previas a la clasificación, que permitirían mejorar positivamente el rendimiento de los resultados finales. Para la etapa de experimentación, se plantea el por qué es necesario la creación de un proceso como método de trabajo, y se detalla el uso de las nuevas utilidades de la herramienta.

3.1. Análisis de las herramientas

Debido a que, el estudio no está centrado en el modelado de redes, sino en los métodos de predicción de enlaces dentro de las mismas, se considera necesario contar con algún software que facilite la funcionalidad de representación de diferentes tipos de redes. Es por ello que el punto de partida de este trabajo consiste en seleccionar la herramienta que mejor se adapte a las necesidades planteadas, es decir, que cuente con una funcionalidad básica de manipulación de grafos, y a su vez, sea de fácil adaptación y extensión para poder incorporar la funcionalidad necesaria para la investigación.

Tomando como punto de partida [Ricotta y Santamarina, 2016], se determinó que la herramienta Gephi cumple con los requerimientos planteados. La misma se trata de un software de visualización y análisis de redes, escrito en Java sobre la plataforma Netbeans ⁴. Este brinda una amplia funcionalidad de administración de grafos, y cuenta con una interfaz gráfica amigable y de uso sencillo. A su vez, su código es abierto y su implementación es considerablemente flexible, lo que facilita la extensión de la misma para poder incorporar nuevas funcionalidades.

Adicional al código base de la herramienta, se cuenta con funciones que calculan el grado de similitud entre pares de nodos de la red, según alguno de los criterios pertenecientes a una serie de predictores incorporados. Además, es posible utilizar estos últimos para predecir enlaces, a través del ocultamiento de enlaces existentes y la realización de un determinado número de predicciones. Estos predictores pueden ser utilizados como funciones de similitud, desde el punto de vista del aprendizaje supervisado. A partir de las funciones realizadas, la herramienta brinda diferentes métricas que permiten evaluar su funcionamiento sobre la red en la que se esté trabajando.

⁴ Sitio web del framework de desarrollo Netbeans <https://netbeans.org/>.

Como ventaja extra, la herramienta cuenta con una gran comunidad de usuarios y desarrolladores que aportan continuamente al crecimiento de la misma. Por ejemplo, existen varios plugins que amplían la funcionalidad base de la herramienta para mejorar su capacidad y satisfacer otras necesidades. También existe una wiki completa dedicada a la documentación de todos los aspectos de Gephi, la cual se mantiene actualizada constantemente y, además, una API que da soporte a la funcionalidad base de Gephi y es conocida como GraphStore. La misma se encuentra escrita en Java y es utilizada para crear grafos de forma rápida, robusta y eficiente. Fue diseñada para ser exportada a otras aplicaciones y utilizar toda la funcionalidad respectiva de los grafos, sin necesidad de incluir la parte gráfica de Gephi.

Por otra parte, es de suma importancia incorporar los diferentes algoritmos de aprendizaje dentro de la herramienta de modelado de grafos. Existe una gran variedad de plataformas de código abierto para el análisis de datos, que proveen software que facilita la aplicación de los diferentes algoritmos de aprendizaje supervisado. Entre las más conocidas, se pueden mencionar RapidMiner, R, Orange y Weka. Si bien todas estas plataformas cumplen con los requisitos para realizar los estudios necesarios, se consideró importante poder integrar los algoritmos de aprendizaje dentro de la herramienta de modelado de redes, con el fin de llevar a cabo todas las experimentaciones dentro de un mismo entorno. Por este motivo, se descartaron R y Orange, ya que ambas pueden ser utilizadas desde las herramientas gráficas que ofrecen, pero no es posible su integración dentro de Gephi ya que utilizan un lenguaje diferente.

Tanto RapidMiner como Weka son compatibles con la herramienta, e implementan todos los algoritmos que se buscan incorporar. Si bien RapidMiner es considerada una plataforma más completa que Weka, las funcionalidades adicionales que esta brinda no son necesarias para llevar a cabo el estudio. Por su parte, Weka es lo suficientemente completa y cuenta con una gran variedad de algoritmos de aprendizaje supervisado que pueden ser configurados manualmente. A su vez, se cuenta con experiencia previa en el uso de este software y se comprobó que brinda una facilidad de uso considerable. Por lo tanto, en lo que respecta a la incorporación de algoritmos de aprendizaje supervisado, se optó por integrar Weka dentro de Gephi para poder realizar las experimentaciones y el análisis dentro de una misma plataforma.

3.2. Creación de datasets

Para llevar a cabo las experimentaciones, es necesario poder crear los datasets que utilizarán los clasificadores. Estos datasets pueden estar formados por diferentes atributos calculados a partir de las funciones de similitud ya mencionadas. Con el fin de analizar el funcionamiento de distintos conjuntos de funciones de similitud, se

debe disponer de un mecanismo sencillo para la creación de diferentes datasets con distintos atributos. Para ello, resulta necesario contar con la posibilidad de seleccionar subconjuntos de las funciones durante la creación de los datasets. La propuesta para satisfacer esta necesidad consiste en incorporar a la interfaz gráfica, una lista con las funciones de similitud, que puedan ser seleccionadas a criterio del usuario, y posteriormente procesadas para la creación del archivo con los atributos basados en la selección.

El archivo generado poseerá formato ARFF, el cual es interpretado por Weka para llevar a cabo las experimentaciones. Se propone brindar la posibilidad de la elección de un nombre para cada archivo generado, con el fin de generar diferentes datasets de manera sencilla, facilitando el proceso de experimentación con diferentes datasets creados a partir de un mismo grafo.

Las funciones de similitud que serán añadidas para su incorporación dentro del dataset son las siguientes:

- Basadas en información topológica local
 - Common Neighbors
 - Preferential Attachment
 - Adamic Adar
 - Resource Allocation
 - Jaccard Coefficient
 - Sorensen Index
 - Salton Cosine
 - Hub Promoted
 - Hub Depressed
 - Leicht Holme Nerman
 - Parameter Dependent
- Basada en información topológica global
 - Hitting Time
 - Shortest Path
- Basadas en información interna al nodo
 - Node Information
 - Vector Node Information

También se debe incorporar la posibilidad de configurar los parámetros de cada función de similitud, para aquellas que sea necesario. Para la función de Parameter Dependent, se debe poder seleccionar el valor del exponente de la función, y para Hitting Time, la cantidad de iteraciones. En los casos de las funciones basadas en información de los nodos, se debe añadir la posibilidad de seleccionar el peso de cada atributo dentro de Node Information, y la posibilidad de seleccionar las características a considerar de cada nodo para Vector Node Information.

El hecho de incorporar instancias a partir de los pares de nodos que no tienen vecinos en común, puede introducir ruido e incertidumbre dentro del dataset, por lo

que, también se propone incluir una función que permita filtrar pares de nodos según la distancia que exista entre ellos. Esto implica calcular, para cada par de nodos, el camino más corto (Shortest Path). De esta forma, aquellos nodos cuya distancia sea mayor a la establecida como parámetro, no serán incorporados dentro del dataset.

El origen del ruido y la incertidumbre proviene de calcular una función de similitud para la cual no se poseen los suficientes datos. Por ejemplo, no se puede calcular Adamic Adar para pares de nodos que no poseen vecinos en común (y lo mismo ocurre con otras funciones similares). En consecuencia, el valor de AA para ese par de nodos es nulo, y en el dataset se refleja como un dato faltante "?". Los mismos son tratados de manera diferente por cada clasificador, y Weka, provee funcionalidades para reemplazarlos por valores arbitrarios calculados a partir de los valores de las demás funciones de similitud. Reemplazar los valores faltantes es un método artificial para reducir la incertidumbre y aumentar el desempeño de la clasificación. Los nuevos valores pueden no reflejar la realidad del grafo, ya que reemplazarlos es solo una necesidad inherente a la falta de datos para calcular ciertas funciones de similitud. Por eso, como objetivo se busca directamente minimizar la cantidad de datos faltantes, siendo esta la razón principal para proponer la creación de los filtros descritos anteriormente.

Debido a que, la función Shortest Path tiene un costo computacional considerable, es posible utilizar el mismo cálculo para incorporar el atributo en el dataset para los casos donde se incluya dicha función de similitud, de forma que sólo sea necesario realizar este cálculo una única vez para cada par de nodos. A su vez, si se filtran los nodos que tengan una distancia mayor a 2, es equivalente a incluir sólo aquellos pares que tengan al menos un vecino en común. Para este caso, es posible utilizar la función Common Neighbors en lugar de la función de Shortest Path, ahorrando un tiempo considerable de procesamiento, principalmente para las redes de gran tamaño.

Una ventaja de esta propuesta es que la creación de los datasets es totalmente independiente de la clasificación de los mismos. Como consecuencia, es factible utilizar los datasets desde otra herramienta que acepte archivos en formato ARFF. Además, como son procesos muy costosos desde el punto de vista computacional, separar las funcionalidades provee una mayor usabilidad para el usuario, ya que no debe realizar todo el procesamiento de una vez.

3.3. Integración de clasificadores

Los datasets que se generarán desde Gephi serán compatibles con Weka. Esto quiere decir que se pueden utilizar los mismos archivos dentro del programa de Weka, aprovechando las funciones de este en su totalidad. Este proceso implica buscar cada dataset generado desde la herramienta, y posteriormente abrirlo desde Weka, donde se debe configurar cada clasificador que se desea utilizar antes de

correr el algoritmo. En este caso, sólo se pueden aplicar las técnicas de balanceo disponibles dentro de Weka, ya que cualquier balanceo personalizado que se desee realizar debe ser implementado aparte. Finalmente, las métricas resultantes son visibles desde Weka, por lo que, no podrían ser manipuladas para realizar comparaciones entre diferentes clasificadores o incluso con las métricas arrojadas por las funciones de similitud individuales.

Ante esta evidencia, la propuesta consiste en integrar toda funcionalidad necesaria para la clasificación dentro de la misma herramienta de Gephi. De esta forma, es posible utilizar cualquier clasificador que se haya implementado, inmediatamente luego de haber creado el dataset, y analizar los resultados dentro de la misma plataforma sin necesidad de alternar entre dos programas distintos.

Si bien Weka posee una larga lista de clasificadores, para la integración se considerarán sólo los clasificadores más populares, que son los que se tendrán en cuenta para las experimentaciones. Los clasificadores a integrar son:

- Naive Bayes
- J48
- Random Forest
- IBk
- SMO
- ZeroR

Como cada algoritmo tiene sus características y parámetros particulares, para cada uno de ellos se presentará un panel personalizado según los posibles parámetros configurables.

Dentro del panel, también se debe dar la posibilidad de configurar todo lo correspondiente al entrenamiento y la evaluación de los clasificadores. Debido a que, la validación cruzada garantiza la independencia entre la partición entre datos de entrenamiento y prueba, se propone integrar esta técnica, dejando como parámetro configurable el valor de K, correspondiente a la cantidad de particiones a realizar.

Para visualizar los resultados de cada clasificador, se mostrará una ventana al finalizar la ejecución del algoritmo, que presentará toda la información relevante a la clasificación. Entre las métricas más importantes se incluyen:

- Precisión
- Recall
- Área bajo la curva
- Curva ROC
- Matriz de Confusión
- Tasa de verdaderos positivos

A su vez, al integrar la funcionalidad de Weka dentro de la plataforma, es posible manipular todos estos resultados para realizar comparaciones entre diferentes

algoritmos. Cualquier tipo de balanceo que se implemente aparte debe manipular los datasets y realizarse previamente a la clasificación.

3.4. Balanceo de datasets

Si bien Weka implementa la posibilidad de balancear los datasets, resultó interesante aplicar otras técnicas de balanceo orientadas a la distribución de los datos de las redes.

En los datasets representados por cada par de nodos, la distribución de clases para el atributo de la existencia del vínculo suele estar extremadamente desbalanceado: aproximadamente un 95-5, dependiendo del tamaño y el grado de conexión de la red. Esta desproporción influye considerablemente en los resultados, que estarán sesgados por la clase predominante.

Basándose en esta característica de los datasets de las redes, se propone integrar tres alternativas de balanceo basadas en la técnica de submuestreo, que serán consideradas para las experimentaciones. Si bien la posibilidad de que en una red predomine la clase “link” por sobre la clase “nolink” es sumamente remota, para todas estas técnicas se considerará el balanceo siempre sobre la clase predominante, sin importar cuál de las dos sea. De todas formas, en este trabajo se dará por supuesto que la clase predominante siempre es “nolink”. Las técnicas de balanceo a integrar tienen como objetivo mejorar los valores de las métricas resultantes de la clasificación afectadas por el desbalance de los datasets. Por ejemplo, para un dataset que posea 95-5 como distribución, la clasificación por si sola será muy efectiva, ya que casi todas las instancias serán clasificadas como la clase predominante debido a la inmensa cantidad de las mismas, y por ejemplo, el AUC no es afectado por esta problemática. Sin embargo, ciertas métricas como el error relativo, precisión o recall, si se ven afectadas en gran medida por el desbalance, ya que determinan el grado de precisión asociado a la clasificación individual de las instancias.

Balanceo simple

Este balanceo es implementado en Weka, y consiste en considerar únicamente un conjunto aleatorio de instancias con la clase predominante, de igual tamaño que la cantidad de instancias con clase no predominante. De esta forma, el dataset final tendrá igual cantidad de instancias para cada clase, quedando completamente balanceado.

La ventaja principal de este balanceo es la sencillez y velocidad, pero se debe considerar que los resultados son dependientes del subconjunto de instancias seleccionadas al azar, por lo que, los resultados pueden quedar sesgados.

Balanceo cross parametrizado

Consiste en una implementación que no se incluye dentro de Weka, y que permite solucionar el problema del balanceo simple sobre la dependencia del subconjunto seleccionado.

La idea consiste en distribuir aleatoriamente el total de instancias con la clase predominante en N subconjuntos y crear N datasets distintos, donde cada uno estaría compuesto por todas las instancias de la clase no predominante y uno de los subconjuntos de la clase predominante. Una vez creado los diferentes datasets, cada uno es entrenado con el algoritmo seleccionado y evaluado de forma independiente. Finalmente, se realiza un promedio de todos los resultados para obtener las métricas finales de la clasificación.

De esta forma, los resultados no estarán sesgados por la selección aleatoria de instancias, ya que todas son consideradas para el entrenamiento, y se promedian las métricas de todas las evaluaciones.

Una de las cuestiones a considerar sobre esta técnica de balanceo, es que los datasets generados no estarán necesariamente balanceados en su totalidad, ya que depende de la cantidad de divisiones que se lleven a cabo, establecidas por el parámetro N mencionado anteriormente.

Balanceo cross automático

Es una variante del anterior, donde se calcula automáticamente el número de divisiones a realizar de forma que los nuevos datasets se encuentren completamente balanceados.

En este caso, la cantidad de datasets generados va a depender de la distribución original de las clases, y cada uno estará compuesto por el total de instancias de la clase no predominante, y la misma cantidad de instancias de la clase predominante. Para los casos donde no se pueda distribuir de forma equitativa las instancias en todos los datasets, es posible distribuir las instancias sobrantes entre los conjuntos generados, quedando así datasets que no estarán completamente balanceados, porque tendrán una pequeña cantidad de instancias de la clase predominante mayor. Otra alternativa para estos casos, es descartar las instancias sobrantes, ya que en la mayoría de los casos no debería afectar a los resultados, siempre y cuando se tenga un conjunto considerable de instancias.

3.5. Selección de atributos

Como funcionalidad adicional, se propone integrar una serie de algoritmos de selección de atributos. Este tipo de algoritmos también se utilizan de forma nativa en Weka, pero se considera que es de suma utilidad que la herramienta cuente con soporte para los mismos. De manera previa a la clasificación, e incluso al balanceo, los datasets son analizados mediante este tipo de algoritmos para conocer cuáles atributos predictores son los que mejor representan al dataset. El objetivo es calcular a partir de los valores presentes en el dataset, la relación intrínseca entre

los atributos predictores, y filtrar aquellos que resultan ser irrelevantes y/o redundantes.

Como algoritmos de evaluación se propone utilizar los siguientes:

- Correlation: evalúa la importancia de los atributos usando el coeficiente de correlación de Pearson entre cada uno y la clase.
- InformationGain: evalúa la importancia de un atributo midiendo la ganancia de información con respecto a la clase.
- WrapperSubset: utiliza un esquema de aprendizaje para evaluar distintos sets de atributos. Se modelará para que realice clasificaciones con el mismo algoritmo con el que se clasifica posteriormente, maximizando el AUC.

Y como algoritmos de búsqueda, los siguientes:

- BestFirst: examina el espacio de atributos mediante una búsqueda greedy mejorada con backtracking. Se usará configurado para que la búsqueda sea de tipo forward (se comienza con un set resultado vacío, y se agregan uno a uno los atributos). No puede ser utilizado si se elige InformationGain.
- GreedyStepwise: realiza una búsqueda greedy a través del espacio de subconjuntos de atributos, y finaliza, cuando el agregado o borrado de los atributos restantes no mejora la evaluación. De manera análoga, la búsqueda es de tipo forward y no puede ser utilizado junto a InformationGain.
- Ranker: crea un ranking de las evaluaciones individuales de los atributos. Se utiliza junto a evaluadores de atributos, y por esta causa, es el único que funciona junto a InformationGain.

Estos tipos de algoritmos representan una forma de filtrar los atributos irrelevantes de los datasets. En consecuencia, los datasets se vuelven más pequeños y representativos, y así, se espera mejorar el rendimiento en general de la etapa de clasificación. De manera contradictoria, el costo computacional de realizar la selección de atributos puede ser muy elevado dadas ciertas condiciones, y por ello, es importante realizar un balance entre la selección y la clasificación, para encontrar la mejor configuración aplicable a cada caso. Se debe tener en cuenta, además, que los clasificadores construyen los modelos de distinta manera, dándole importancia a distintas características del dataset. Por ende, cada método de selección de atributos funciona de peor o mejor manera dependiendo del clasificador que se utilice.

3.6. Comparación con las métricas de similitud

Entre las ventajas que ya se mencionaron acerca de integrar funcionalidades de Weka dentro de la misma plataforma, se incluye la posibilidad de visualizar los

resultados de las clasificaciones y compararlos con las predicciones de los demás algoritmos implementados en la herramienta.

Como ya fue explicado anteriormente, dentro de Gephi se cuenta con la implementación de las funciones de similitud, útiles para predecir enlaces. Los resultados de estas predicciones también se miden con varias métricas entre las que se encuentran precisión, recall y AUC.

Uno de los objetivos de este trabajo es poder comparar el rendimiento final de las predicciones del aprendizaje supervisado, contra las correspondientes a las métricas de similitud individuales. Para realizar tal tarea se propone implementar un apartado específico en la herramienta, que ejecute múltiples funciones y clasificadores, de forma que los resultados arrojados por estos para una misma red puedan ser comparados entre sí. A su vez, dentro de la implementación se debe incluir la posibilidad de configurar los parámetros que se puedan necesitar, tanto para los predictores como para los clasificadores. Es importante destacar que, este apartado es considerado una manera sencilla y rápida de realizar la comparación entre ambos métodos de predicción de enlaces. Sin embargo, es preferible realizarla de forma manual y más detallada, ejecutando los métodos individuales por separado, y comparando las métricas de los reportes finales.

3.7. Proceso de experimentación

Debido a la cantidad incontable de combinaciones posibles que pueden darse en el momento de realizar las pruebas, se propone que los experimentos sigan una estructura justificada y ordenada. Reflejar mediante experimentos todas las combinaciones de cada una de las propuestas anteriores resulta imposible desde un punto de vista práctico.

Durante la etapa de preprocesamiento, dado un grafo cualquiera se buscará crear el dataset que lo represente. Existen innumerables conjuntos de funciones de similitud que pueden elegirse para crear el dataset, y a su vez, varias de ellas poseen parámetros configurables, con lo cual ejecutar una sola versión de las funciones no resulta suficiente a menos que se tomen ciertos recaudos. A su vez, la propuesta de filtrar pares de nodos separados por cierta distancia, implica que la cantidad final de instancias de un dataset depende fundamentalmente del parámetro de lejanía que se use. Por lo tanto, es posible que existan varios datasets, conformados por distintos atributos o distinta cantidad de instancias, que representen de manera ligeramente diferente a un mismo grafo.

Posteriormente, se buscará realizar la clasificación sobre algún dataset elegido en la etapa inicial dado algún criterio. Para esta segunda etapa pueden elegirse varios componentes configurables que impactan en el resultado final de la clasificación.

En primera instancia, la cantidad de folds que se utilizarán en el cross-validation, determinará la precisión del modelo y buscará disminuir el sobreajuste.

En segunda instancia, la propuesta del balanceo determina la calidad de las métricas finales en datasets típicamente desbalanceados, y existen cuatro variantes: sin balanceo, simple, cross automático, o cross manual, en el cual se configura la cantidad de subdatasets que se crearán.

En tercera instancia, es posible elegir un método de selección de atributos, y teniendo en cuenta las restricciones explicadas en la anterior sección, existen seis variantes: sin selección, Correlation con Best First o Greedy Stepwise, Wrapper Subset con Best First o Greedy Stepwise, e Information Gain con Ranker.

Finalmente, y como última instancia, es posible realizar todas las pruebas anteriores para cada uno de los clasificadores que se implementen dentro de la herramienta, dejando de lado las variantes que se pueden producir al modificar los parámetros configurables de cada clasificador.

Hay que tener en cuenta que la gran combinación de componentes descrita con anterioridad es aplicable a un solo grafo, y que, en caso de poseer varios grafos a analizar, debería aplicarse el mismo criterio a cada uno. En la Figura 6 se ilustra el problema descrito para un solo grafo:

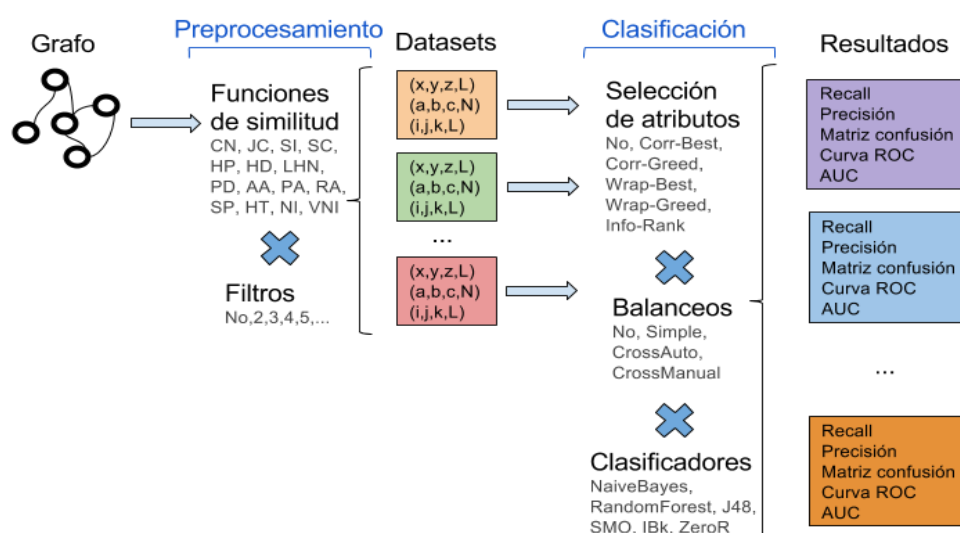


Figura 6 - Combinaciones posibles de las técnicas y componentes a probar

Se observa que la cantidad de combinaciones posibles de componentes es demasiado elevada como para realizar todas, más si se disponen de varias redes sociales con las que se debe experimentar. Sin embargo, este proceso de clasificación puede ser acotado si se toman en cuenta ciertas consideraciones. Debido a esto, se propone realizar un proceso de experimentación, donde serán probados todos los componentes y un gran conjunto de sus variantes, y así, reducir considerablemente la cantidad final de clasificaciones. Se propone que el proceso posea cuatro etapas en cadena.

Pre-procesamiento

Es la primera etapa en la cual debe crearse el grafo correspondiente a cada red social dentro de la herramienta Gephi, y mediante ésta, obtener el dataset

correspondiente en formato ARFF. Dependiendo de cada red y del formato en el que se encuentre representada, crear el grafo puede ser un proceso directo cómo simplemente abrir el archivo, o se puede necesitar de cierto procesamiento previo.

Una vez creado el grafo correspondiente, se procederá a calcular un solo dataset que represente a esa red con la mayor cantidad de información posible. Para esto, se utilizarán todas las funciones de similitud que se encuentren disponibles a elegir. Por ejemplo, en grafos donde los nodos no posean información adicional útil, no podrán ser usados NI ni VNI, pero si el resto de funciones. Además, PD no será utilizado, ya que variar su exponente deriva en otras funciones de similitud. Así, se evita introducir ruido al dataset disminuyendo la redundancia de poseer atributos similares.

Como paso siguiente, se utilizarán tres filtros diferentes: sin filtro, mínimo y medio. De esta manera, será factible demostrar si es mejor que los datasets posean más instancias y una mayor cantidad de datos faltantes, o menos instancias y menor cantidad de datos faltantes (debido al uso de funciones de vecinos comunes para nodos que no los poseen). Como consecuencia existirán tres datasets diferentes que representen a un mismo grafo, de los cuáles se elegirá el mejor midiendo la cantidad de datos faltantes e instancias finales.

Selección de atributos

En esta etapa se realizarán clasificaciones con y sin selección de atributos, tomando como base el dataset obtenido anteriormente. El selector elegido es WrapperSubset - BestFirst, el cual se presupone de mayor calidad ya que utiliza un esquema de aprendizaje. El costo computacional de este método es muy elevado, y si el dataset posee un tamaño prohibitivo, se elegirá otro método de selección de atributos alternativo que posea mejor rendimiento. De esta manera se busca mejorar el AUC seleccionando los mejores atributos que representan al dataset.

Balanceo

Utilizando el mismo dataset junto a la selección de atributos, en caso de mejorar los resultados, se procederá a analizar el funcionamiento de las clasificaciones si se les aplica o no el balanceo. Como anteriormente ya se experimentó la alternativa sin balancear, queda probar alguna técnica de balanceo. El balanceo cross manual no será tenido en cuenta, ya que el automático es el caso óptimo del anterior. Y el balanceo simple no posee suficiente calidad como para ser una técnica viable de balanceo que mejore el proceso de clasificación. Por ende, se utilizará balanceo cross automático, con el objetivo de mejorar aquellas métricas afectadas por el desbalance, al costo de mayor procesamiento.

Aprendizaje supervisado vs. métricas de similitud individuales

En la última etapa se procederá a comparar los mejores resultados obtenidos en la anterior etapa, contra los mejores resultados obtenidos en [Ricotta y Santamarina, 2016]. De esta manera se podrá determinar si son mejores las funciones individuales o el enfoque de aprendizaje, siempre y cuando la comparación sea

posible. Se utilizará AUC como medida de rendimiento principal, aunque también serán tenidas en cuenta otras métricas.

4. Diseño e implementación

El diseño e implementación necesarios para esta investigación se realizaron utilizando como punto de partida la herramienta Gephi. Cabe destacar que, sobre la misma, se encontraba implementado el cálculo de distintas funciones de similitud que sirven como predictores de enlaces, el cual fue realizado en [Ricotta y Santamarina, 2016]. También se utilizó la API de Weka para integrar la funcionalidad correspondiente al aprendizaje automático supervisado. De esta manera, cualquier prueba o experimento podrá ser realizado desde la misma herramienta sin necesidad de recurrir a otra utilidad, a excepción de pruebas que superen los límites impuestos por Gephi mismo (por ejemplo, en cuanto a rendimiento o capacidad de memoria).

A continuación, se detallan ciertos aspectos existentes de la herramienta, junto a la explicación de su funcionamiento, acompañados de diagramas en caso de ser necesario. Se explica la manera de extender la funcionalidad original para cumplir las necesidades de esta investigación, teniendo en cuenta ciertas restricciones y facilitando el uso de la herramienta para el usuario final. Se incluye pseudocódigo de aquellos algoritmos considerados de gran importancia junto a una explicación paso a paso. De esta manera, es posible explicar más a fondo el funcionamiento de la implementación sin ahondar en detalles no relevantes.

4.1. Restricciones y requerimientos

Para satisfacer los objetivos de la presente investigación, es necesario cumplir una serie de requerimientos obligatorios, en consonancia con las diferentes propuestas enunciadas en la sección 3. Como Gephi ya posee soporte para calcular distintas cuestiones relacionadas a los grafos y sus características, sólo debe extenderse su funcionalidad respetando el diseño previo y sus restricciones, aprovechando a su vez las facilidades que la API de Weka provee. A continuación, se listan todas las cuestiones anteriormente mencionadas, separadas por categoría.

Restricciones:

- El desarrollo debe realizarse utilizando Netbeans como entorno de desarrollo integrado (IDE) y utilizando Java como lenguaje de programación. Gephi fue desarrollado desde un principio en ese entorno, y al extender su funcionalidad, el entorno debe seguir siendo el mismo.
- El diseño e implementación de la extensión necesaria para este trabajo, debe respetar y acoplarse al diseño modular de Gephi. No respetar el modelo existente, puede producir problemas inesperados y dificultar cualquier intento futuro de extender este mismo trabajo, ya sea con otras funcionalidades o con nuevas características de Gephi.

Requerimientos funcionales cubiertos por Gephi y su extensión en [Ricotta y Santamarina, 2016]:

- Importación y exportación de grafos a partir de varios tipos de archivos. Cualquier dataset que represente una red social en forma de grafo debe poder ser utilizado por la herramienta.
- Cálculo de funciones de distinto tipo sobre grafos. Cualquier tipo de característica general sobre los grafos debe poder ser calculada.
- Visualización y almacenamiento de los resultados de ejecuciones de las funciones. Para permitir un análisis posterior sobre los mismos.
- Funciones de similitud para la predicción de enlaces. Para calcular características de similitud entre nodos que permitan construir un vector de características que alimente a los clasificadores.

Requerimientos funcionales:

- Construcción de un vector de características a partir de valores de similitud entre nodos, que constituyen un conjunto de instancias.
- Filtrado de instancias correspondientes a nodos separados por cierta distancia.
- Almacenamiento del vector de características en forma de dataset (ARFF).
- Selección de atributos de los datasets
- Balanceo de los datasets.
- Clasificación de datasets.
- Reportes de las clasificaciones

Requerimientos no funcionales:

- Equilibrada usabilidad para el usuario final. El mismo debe poder configurar todos los componentes posibles, sin que el proceso sea demasiado difícil ni contraintuitivo.
- Alto rendimiento en puntos críticos de procesamiento. El sistema debe poseer buena performance en ciertos algoritmos puntuales que pueden volverse bloqueantes para grandes datasets.
- Flexibilidad de diseño suficiente para que se tengan en cuenta futuras funciones de similitud para la creación de datasets. Además, se debe facilitar la inclusión de nuevos clasificadores.

Una vez definidas todas las restricciones y los requerimientos faltantes a cumplir, se debe proceder a realizar un estudio del diseño existente de Gephi. De esta manera, es posible decidir la forma más adecuada de extender la funcionalidad y utilizar la API de Weka de forma transparente al resto del sistema.

4.2. Diseño

Como ya se mencionó anteriormente Gephi posee un diseño modular. El mismo está compuesto por una arquitectura formada por múltiples módulos con roles definidos que trabajan de manera conjunta. La arquitectura se basa en el patrón de arquitectura de software MVC. Este patrón separa el sistema en tres componentes diferenciados: el modelo que representa los datos y la lógica de negocio, la vista que representa la interfaz de interacción para con el usuario final, y el controlador que representa la gestión de eventos y comunicación.

Cada módulo posee una responsabilidad bien definida dentro del sistema, lo cual debe ser respetado para agregar cualquier tipo de funcionalidad nueva. Además, cada módulo presenta APIs públicas al resto, para evitar dependencias circulares, y mantener el encapsulamiento y nivel de abstracción apropiados. A continuación, en la Figura 7, se muestra un esquema informal y aproximado de la arquitectura de Gephi:

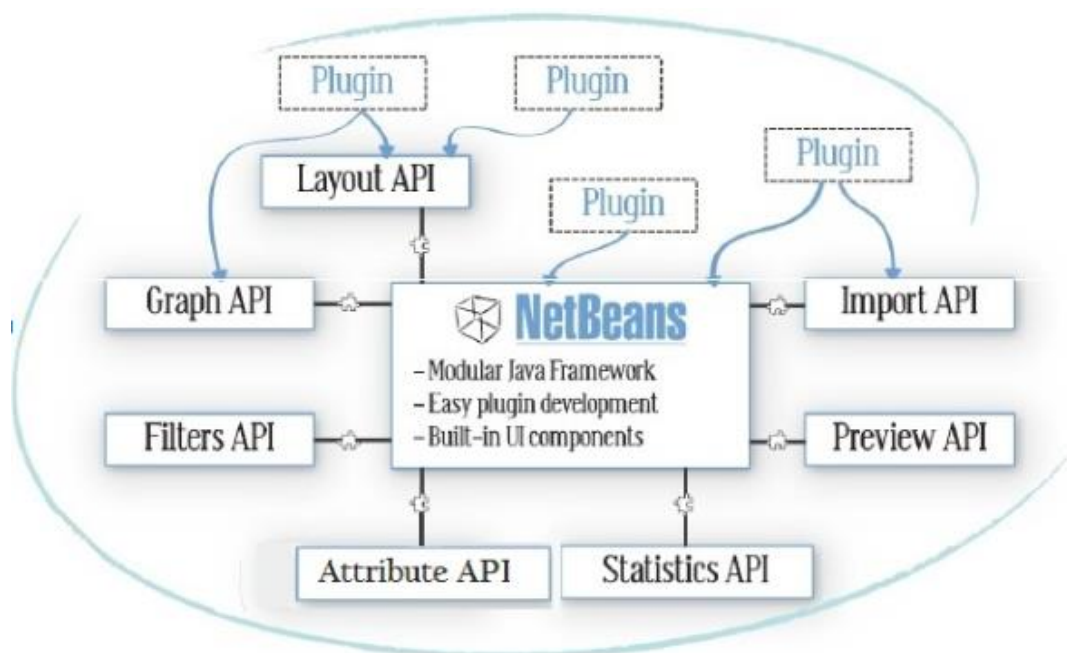


Figura 7 - Arquitectura modular de Gephi, con Netbeans como marco de trabajo

Como se puede observar, todos los módulos, junto a plugins y librerías, se orquestan a través del framework de desarrollo que propone Netbeans para dar forma a Gephi. Cada API posee una responsabilidad única y provee paquetes públicos para uso de los demás módulos. A continuación, se detallan las responsabilidades de cada API.

Layout API

Responsable de los algoritmos de diseño en tiempo real, que reubican los elementos en el eje X e Y según la distribución que se elija.

Graph API

Responsable de toda funcionalidad correspondiente a la manipulación de grafos. Permite agregar, modificar o eliminar nodos y aristas que conforman un grafo. Además, provee registro de eventos para deshacer y rehacer modificaciones, y provee soporte para crear subgrafos (como vistas del grafo original) siguiendo algún tipo de criterio.

Filters API

Contiene algoritmos que hacen posible el filtrado del grafo mediante diferentes criterios para crear una vista personalizada del mismo. Es posible exportar la vista para crear un nuevo espacio de trabajo.

Attribute API

Responsable de la manipulación de la tabla de datos, permitiendo agregar, modificar o eliminar columnas y filas de la misma. Esta tabla se encarga de la persistencia de cualquier tipo información adicional que posean nodos o aristas en forma de distintos tipos de datos.

Statistics API

Engloba a los algoritmos encargados de calcular funciones de todo tipo sobre los grafos, ya sea para cada elemento individual o de forma global a un grafo. Este tipo de tareas se pueden ejecutar de forma sincrónica y asincrónica, conociéndose su progreso y permitiendo ser canceladas. Los resultados se visualizan mediante un documento HTML que puede ser almacenado.

Preview API

Mantiene una representación flexible de la red junto a personalizaciones de alto nivel. Dado un grafo cualquiera, se utiliza para crear un mapa de red que puede ser exportado como imagen.

Import API

Encargada de las diferentes formas de importar datos dentro de la aplicación. Se realiza en dos pasos, primero se carga la información en un contenedor para mostrar un resumen al usuario, y después se traspasan los datos del contenedor al espacio de trabajo.

Este tipo de arquitecturas es muy flexible en caso de querer agregar un plugin como un nuevo módulo que se acople a los existentes. También es posible modificar un plugin existente sin afectar a los demás, ya que son todos independientes. Sin embargo, en este último caso deben respetarse ciertos contratos que el módulo cumple para que la extensión de la funcionalidad sea posible. Dado que el módulo StatisticsAPI se encarga del cálculo de cualquier tipo de función sobre los grafos, y

siguiendo la filosofía de implementación en [Ricotta y Santamarina, 2016], se decidió que la extensión necesaria para este trabajo sea realizada en dicho módulo. Para la versión 0.9.2 de Gephi, la cual es utilizada como base de implementación, el módulo contiene cuatro secciones encargadas de calcular diferentes características de los grafos. Adicionalmente al trabajo de Ricotta y Santamarina, se agregó una quinta sección con las funciones de similitud para predecir enlaces que se utilizarán en la construcción de los datasets.

En la Figura 8 se visualiza la ubicación de las funciones correspondientes al módulo, dentro de la interfaz de Gephi. La flecha indica que es necesario hacer scroll hacia abajo para visualizar el resto de opciones, debido a la gran cantidad de funciones presentes dentro de este módulo.

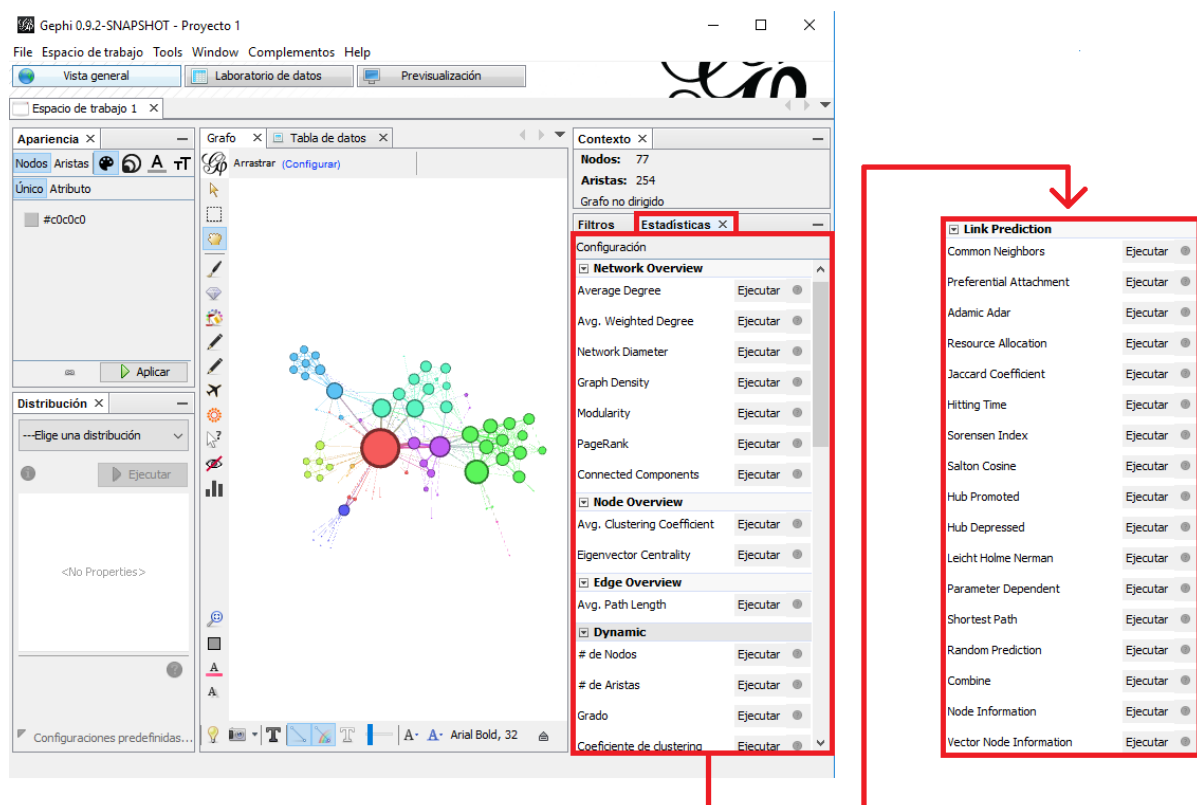


Figura 8 - Funciones del módulo StatisticsAPI, vistas desde la interfaz de Gephi

Como consecuencia de la ubicación de StatisticsAPI en la interfaz de Gephi, se decidió crear una sexta categoría dedicada a las funciones de aprendizaje supervisado, donde cada función corresponde a un clasificador distinto. De esta manera, todas las funciones de aprendizaje, sin importar su tipo, quedan englobadas en la misma categoría. Por lo tanto, deben tenerse en cuenta ciertas consideraciones respetando el diseño existente, y así hacer posible la decisión anterior.

Desde el punto de vista de diseño, este módulo se encuentra formado por dos submódulos: StatisticsPlugin, encargado de la lógica de las funciones y de la construcción de los resultados (backend), y StatisticsPluginUI, encargado de la interfaz gráfica con la que interactúa y visualiza los resultados el usuario final

(frontend). En consecuencia, la extensión necesaria impacta de igual manera a los tres módulos, y debe definir un framework que permita fácilmente agregar nuevos clasificadores una vez finalizado este trabajo. Además, se debe tener en cuenta que es posible que se agreguen nuevas funciones de similitud en la categoría anterior, que pueden ser de utilidad en la creación de datasets. Por lo tanto, el diseño e implementación del framework debe ser lo suficientemente potente para tener en cuenta ambos casos de extensibilidad, y cumplir los contratos de los módulos a la vez.

En la Figura 9, se visualiza un diagrama de componentes donde se muestra parte del diseño de módulos de Gephi, y como se acopla la API de Weka al mismo.

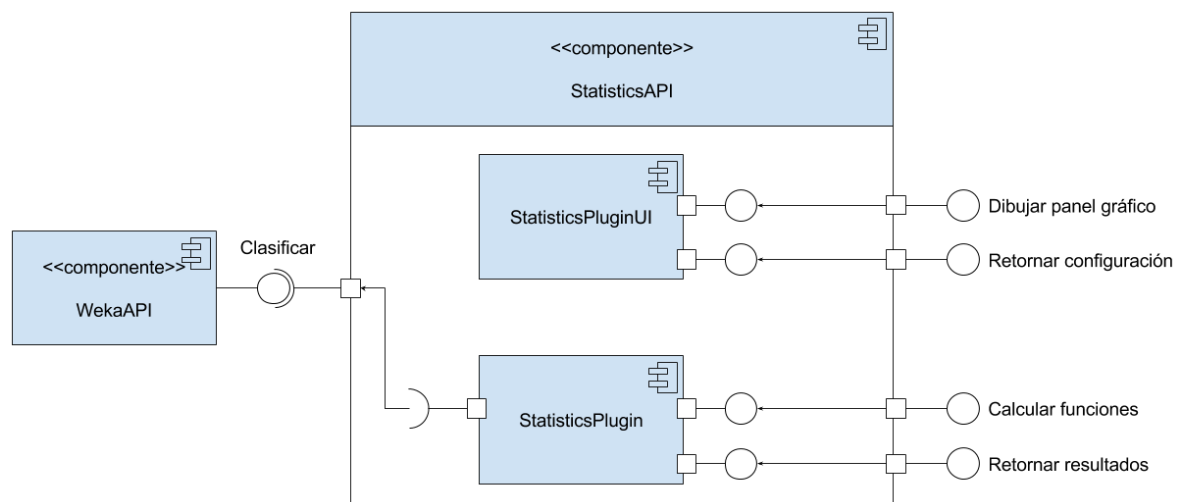


Figura 9 - Interfaces que provee el módulo StatisticsAPI

Dado el diagrama anterior, la API de Weka provee una interfaz con todas las funciones de clasificación al módulo StatisticsPlugin. Como las cuestiones relacionadas al aprendizaje supervisado tratan de procesamiento de distintos tipos de datos, caben dentro de la categoría de componentes de backend. En consecuencia, la interacción del usuario final con las nuevas herramientas de aprendizaje, mantiene la misma línea que el resto de Gephi. La inclusión de la API de Weka es transparente al resto del sistema y solo impacta en las nuevas clases de implementación necesarias para llevar a cabo la extensión. Además, la interfaz gráfica debe permitir configurar la mayor parte de componentes posibles que participen en el preprocesamiento o clasificación, siempre y cuando se respeten las restricciones de los paneles visuales de las funciones existentes.

Una vez decidido todo lo relacionado al diseño final de la extensión, se procedió a realizar un planteo inicial de la implementación que refleje todas las decisiones tomadas hasta este punto, y que cumpla con todos los requerimientos planteados anteriormente. Hasta este punto sólo se cumplen de forma parcial, los requerimientos no funcionales de usabilidad para el usuario final y de flexibilidad de diseño.

4.3. Implementación

La implementación realizada como extensión de la herramienta, será explicada en diferentes etapas para abarcar todos los detalles que sean necesarios, sin mezclar aspectos que no dependan entre sí. Además, se incluyen diagramas o pseudocódigo, siempre y cuando se considere necesario.

Como primer paso, era obligatorio crear la nueva categoría en la interfaz principal del programa principal. Para ello hubo que realizar dos cambios pequeños en dos módulos distintos. Primero, se definió la nueva categoría en la interfaz StatisticsUI del módulo StatisticsAPI. Dado que el texto “Supervised Link Prediction”, título de la categoría, es constante, se hace uso de la clase NbBundle. La misma es una clase de utilidad perteneciente a Netbeans para el almacenamiento de todo tipo de recursos. En el caso de Gephi, se utiliza especialmente para almacenar todo tipo de texto fijo perteneciente a la interfaz gráfica, con soporte de traducción a varios idiomas.

Una vez definida correctamente la categoría, se procedió a utilizarla en el algoritmo encargado de inicializar las mismas. Este se encuentra en la clase StatisticsPanel del módulo DesktopStatistics, en la cual debe hacerse explícita la inserción de la categoría junto a su posición en el panel. Para respetar la estructura original de Gephi, se insertó en el último lugar. Como consecuencia, la categoría queda debajo de todas las demás. A partir de este punto se pudo desarrollar toda la funcionalidad necesaria, sin ninguna otra restricción perteneciente a Gephi.

Debido a que, la implementación final es compleja, en la Figura 10 se creó un diagrama informal que representa gráficamente las relaciones entre las principales clases. Además, se refleja el framework desarrollado, que permite que la inserción de nuevos clasificadores sea lo menos costosa posible.

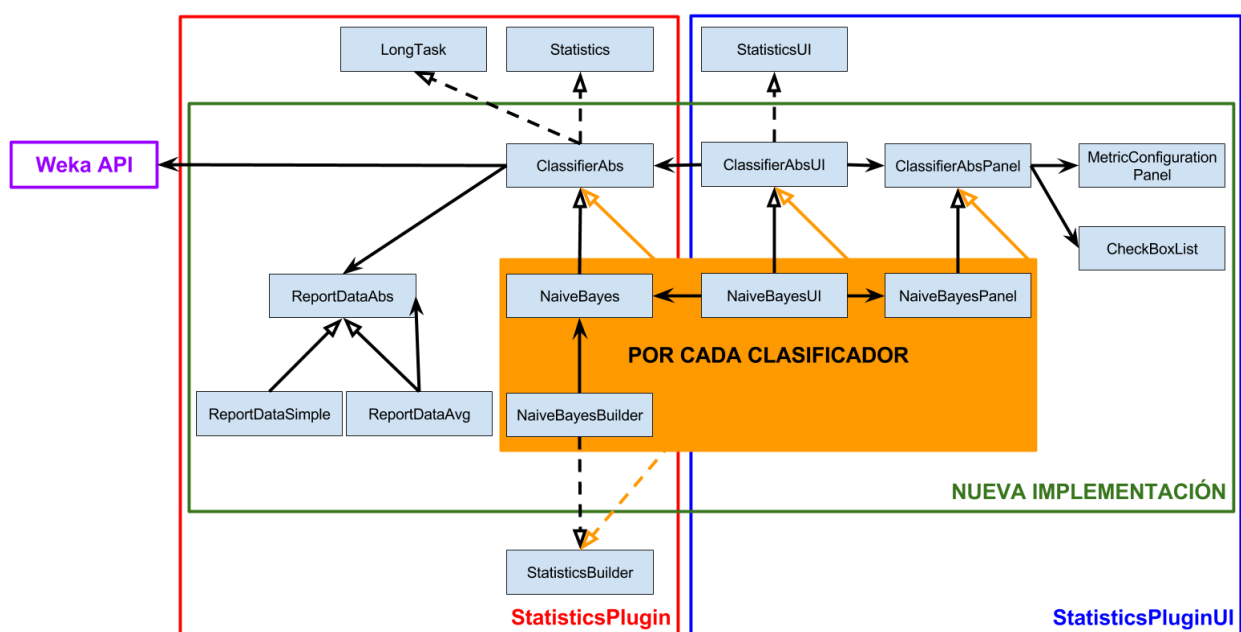


Figura 10 - Ubicación y relaciones de las principales nuevas clases

Se observa claramente que el requerimiento no funcional de flexibilidad de diseño se cumple. De esta forma, se facilitó la inclusión de todos los clasificadores, y se beneficia cualquier extensión futura de esta implementación.

Para insertar un nuevo clasificador a Gephi, solo deben implementarse las cuatro clases dentro del sector naranja. Todas son clases pequeñas e implementan muy pocos métodos en total. La más costosa es la correspondiente al panel gráfico, donde se diseña y define cuáles opciones de configuración se proveen al usuario final. Para esto, se realiza una traducción de componentes de configuración visuales a un array de string, el cual puede ser interpretado por cualquier clasificador de Weka en backend. Queda a responsabilidad de quien implementa realizar la traducción, y cumplir el contrato que espera el clasificador.

Existen dos clases abstractas principales que cumplen roles muy importantes dentro de la implementación. La primera se denomina `ClassifierAbs`, perteneciente a `StatisticsPlugin`, encargada de las cuestiones generales del cálculo de la clasificación. De la misma hedera cada clasificador, para proveer su propio modelo al proceso. De manera análoga, la segunda se denomina `ClassifierAbsPanel`, perteneciente a `StatisticsPluginUI`, y se encarga de las cuestiones generales de interfaz gráfica. Cada clase que herede de ella, define su interfaz propia, debido a que, cada clasificador posee diferentes capacidades de configuración. Las mismas fueron el punto de partida de la implementación, buscando maximizar la flexibilidad de diseño. Debido que a partir de este punto inicial se tomaron muchas decisiones que involucran a ambas clases, la explicación se divide en frontend y backend, para mayor claridad y orden.

4.3.1. Frontend

Esta sección corresponde a la implementación realizada sobre `StatisticsPluginUI`. Como ya se mencionó anteriormente, es el módulo encargado de la interfaz gráfica, correspondiente al cálculo de funciones sobre el grafo. Como tal, requiere que se cumplan ciertos requisitos para la creación de la interfaz gráfica correspondiente a una nueva función. Se debe crear una clase que implemente la interfaz `StatisticsUI` para incluir varios métodos que abarcan, entre varias cosas, obtener la categoría de la función, obtener y almacenar la configuración, y la más importante, obtener la clase que representa el panel gráfico, que debe heredar de `JPanel`. Esta última, es la clase en la cual se diseña la interfaz visual correspondiente a la función, mientras que la que implementa `StatisticsUI`, actúa como pasamanos de la configuración y los resultados, entre frontend y backend. Dado que se detectó cierto comportamiento común entre todos los clasificadores, se decidió que una clase abstracta llamada `ClassifierAbsUI` implemente `StatisticsUI` y que otra clase abstracta llamada `ClassifierAbsPanel`, implemente `JPanel`.

`ClassifierAbsUI` es la clase abstracta encargada de toda la funcionalidad general en cuanto al traspaso de datos entre frontend y backend. Por lo tanto, se generalizaron

aquellos métodos no afectados por las características intrínsecas de cada clasificador. Los métodos son aquellos que se encargan de aplicar la configuración predeterminada en la interfaz gráfica, y obtener la configuración seteada por el usuario y enviarla a backend. La cantidad de folds, el método de selección de atributos, el balanceo, las funciones de similitud, o las opciones de un clasificador, son consideradas distintas partes de la configuración.

Finalmente, se definen dos métodos abstractos que tienen que ver con la definición del panel visual, responsabilidad que se delega a cada clase que herede de ésta. Cada clase que hereda de `ClassifiersAbsUI` se encarga de definir la creación de la clase que define el panel visual a utilizar (ya que existe uno por clasificador), y de los demás métodos de la interfaz `StatisticsUI`, como la categoría, la posición, etc.

Un ejemplo de todo lo anteriormente comentado es el siguiente: `NaiveBayesUI` define el uso de `NaiveBayesPanel` y, por lo tanto, le traspasa la configuración por defecto. A continuación, el usuario en `NaiveBayesPanel` define sus preferencias de configuración y `NaiveBayesUI` es la encargada de traspasar esas opciones a `NaiveBayes`, en backend.

En la Figura 11 se visualiza un diagrama de clases con lo anteriormente comentado. Cabe aclarar que ciertos atributos y métodos no relevantes de las clases se omitieron por cuestiones de legibilidad.

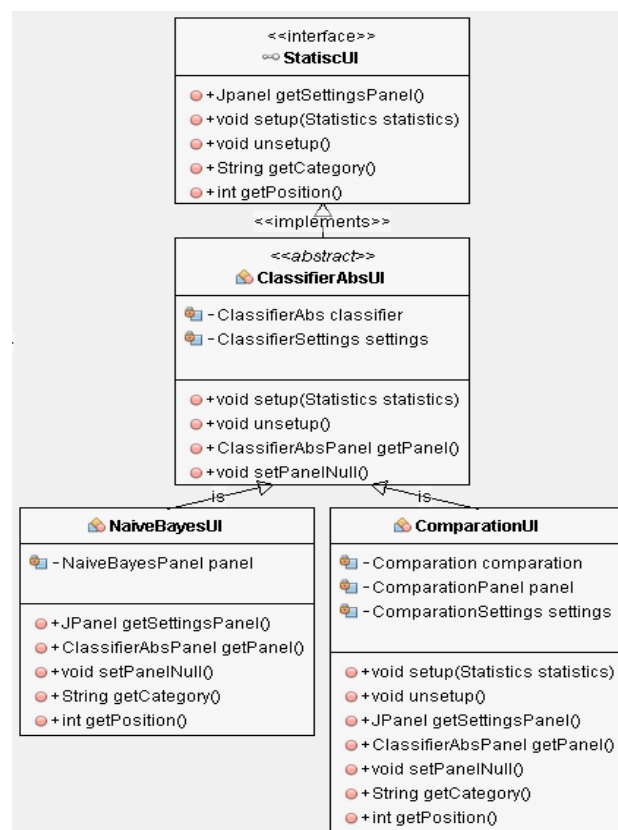


Figura 11 - Diagrama de clases que heredan de StatisticsUI

De manera análoga, `ClassifierAbsPanel` se encarga de todas las cuestiones generales que utilizan las interfaces gráficas de cada clasificador, sin definir ningún

elemento visual. Por ejemplo, se encarga de inicializar todas las variables que pueda necesitar la interfaz gráfica, de cargar un objeto perteneciente a la clase `CheckBoxList` con todas las funciones de similitud, de inicializar el panel de configuración de las funciones de similitud llamado `MetricConfigurationPanel`, y de la creación del vector de características como un dataset en formato ARFF. Además, define como abstractos todos los métodos encargados de extraer cada elemento individual de la configuración, para que cada clase hijo los retorne como corresponda.

Una de las limitaciones de este módulo es que sólo puede calcularse una función que involucre la invocación de backend por panel, y debido a que, se decidió reservarla para la clasificación, cualquier otro cálculo, como la creación de los datasets, debe realizarse en esta clase abstracta.

Dado que la función de creación de datasets es una de las más importantes para esta investigación, se procede a explicar en detalle su funcionamiento. El siguiente pseudocódigo refleja a grandes rasgos el paso a paso del algoritmo:

(Si GRAFO es dinámico obtener subgrafo estático en intervalo de tiempo)

Para todos los nodos I de GRAFO

Para todos los nodos restantes J de GRAFO

Si I es distinto de J

(Sino se filtra) o (Si I y J están cerca)

Eliminar enlace entre I y J

Para todas las funciones de similitud elegidas

Calcular similitud entre I y J

Agregar enlace entre I y J

Escribir instancia en archivo

Así, el algoritmo de creación de datasets, queda perfectamente representado por el pseudocódigo, reflejando las principales características y cuestiones tenidas en cuenta.

Como primer paso, se tuvo en cuenta la existencia de grafos dinámicos. Debido a que, la funcionalidad de este trabajo abarca solo grafos estáticos, se implementó un algoritmo que extrae un subgrafo estático perteneciente a un intervalo de tiempo de un grafo dinámico. El usuario en la interfaz posee la posibilidad de decidir el punto en el tiempo hasta el cual se considera la extracción del subgrafo. El mismo quedará conformado por las aristas y nodos presentes hasta ese momento. De este punto del algoritmo en adelante, el subgrafo estático es utilizado de la misma manera que un grafo estático común.

En el siguiente paso, se realiza la inicialización del archivo con el nombre elegido por el usuario. Además, se le inserta el encabezado que establece el formato ARFF, declarando el nombre y tipo de los predictores elegidos y la clase. En el caso de este trabajo, todos los predictores son de tipo `NUMERIC`, ya que las funciones retornan la similitud como un `double`. En el caso de la clase, la misma puede valer "Link" o "None", representado si entre los nodos existe enlace o no respectivamente.

A continuación, se declaran dos iteraciones para recorrer todas las combinaciones de nodos posibles del grafo. La segunda iteración solo recorre los nodos que no han sido considerados aún por la primera. De esta manera no se produce el caso de crear dos instancias distintas, representando una a los nodos I y J, y otra a los mismos nodos, pero en el orden inverso J e I. Esto solo afectaría negativamente a los grafos dirigidos donde puede existir enlace de I a J y no de J a I. Sin embargo, como todas las funciones, excepto las topológicas globales, retornan los mismos valores, sin importar el orden de los nodos, no vale la pena crear dos instancias. Además, dado que SP utiliza un algoritmo especial para grafos dirigidos, y que HT es en mayor parte aleatoria, no vale la pena agregar una nueva instancia por sólo dos valores diferentes. Y esto sólo ocurriría, en el caso de que el usuario las eligiese para la creación del dataset. En resumen, si un grafo posee M nodos, entonces se examinan $M * M / 2$ combinaciones de nodos para la creación de instancias. Esto beneficia a la performance del algoritmo, ya que reduce a la mitad la cantidad de instancias posibles (sino sería $M * M$).

Para calcular la similitud entre dos nodos, primero se deben cumplir dos condiciones. La primera pide que los nodos sean diferentes ($I \neq J$), lo cual implica que la cantidad final de instancias será de $(M * M / 2) - M / 2$, que es igual a $(M * M - M) / 2$. Esto se puede observar en la Figura 12, donde se observa la diferencia entre cantidades de nodos e instancias finales, reflejando la función anteriormente mencionada. La segunda condición tiene que ver con la funcionalidad de filtrado, reduciendo aún más la cantidad anterior de instancias. Sin embargo, en este caso pueden ocurrir dos situaciones: una es que no se decidió filtrar y entonces se tienen en cuenta todos los pares de nodos, y la otra que, si se decidió filtrar, sólo se tendrán en cuenta los nodos I y J que posean una distancia menor o igual al filtro.

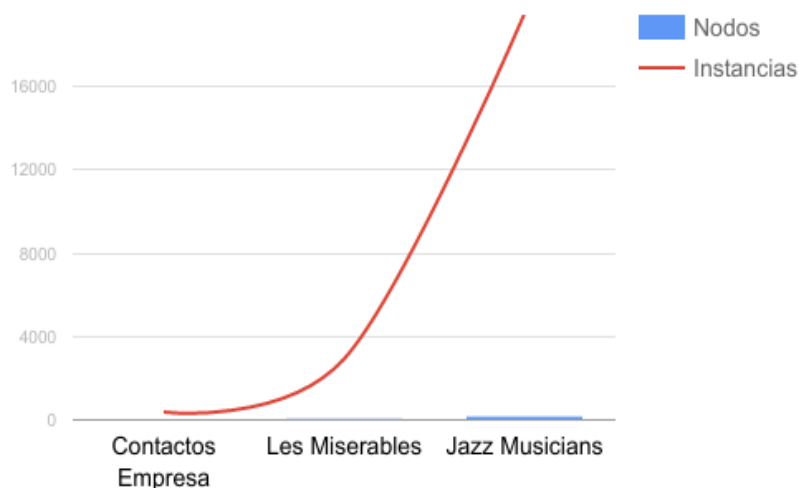


Figura 12 - Relación entre la cantidad de nodos y de instancias sin filtrar para tres datasets

Para aquellos pares de nodos que se tienen en cuenta, el funcionamiento del algoritmo procede de la siguiente manera. En primer lugar, en caso de existir link entre I y J, se borra temporalmente para volver a agregarlo una vez se hayan

calculado todas las funciones. Esto se realizó debido a que, al calcular la similitud entre un par de nodos, la hipótesis es que no se conoce si existe enlace entre ellos. En segundo lugar, se calculan aquellas funciones que haya seleccionado el usuario. Si alguna retorna “null” por no poder calcular la similitud, se marca como un dato faltante “?”. Finalmente, se insertan los resultados de las funciones y la etiqueta de clase como una nueva instancia directamente en el archivo. Esto se hizo para utilizar la menor cantidad de objetos posibles, disminuyendo así la cantidad de memoria necesaria. Dado que no existe una estructura intermedia de almacenamiento, la performance del algoritmo también se ve beneficiada. Este fue un caso donde el requerimiento no funcional de performance era clave, ya que en ciertos puntos Gephi puede tardar mucho o quedarse sin memoria. Por ejemplo, si se tiene un grafo con 100.000 nodos y se quiere crear un dataset sin filtrar, se sabe que debido a las decisiones anteriormente tomadas la cantidad de instancias sería $(100.000 * 100.000 - 100.000) / 2 = 4.999.950.000$ instancias finales. Es totalmente inviable mantener esa cantidad de instancias en memoria, y tardaría unos cuantos días de procesamiento para una computadora de gama media.

Volviendo al funcionamiento de las clases, todas aquellas que hereden de ClassifierAbsPanel, representaran el panel asociado a cada clasificador que se implemente. Dentro de las mismas se definen todos los componentes visuales y la interacción entre los mismos. Además, deben implementar todos los métodos abstractos para retornar la configuración seleccionada por el usuario de manera de cumplir el contrato que espera ClassifierAbsUI. A continuación, en la Figura 13, se muestra una captura del panel correspondiente a J48Panel a modo de ejemplo:

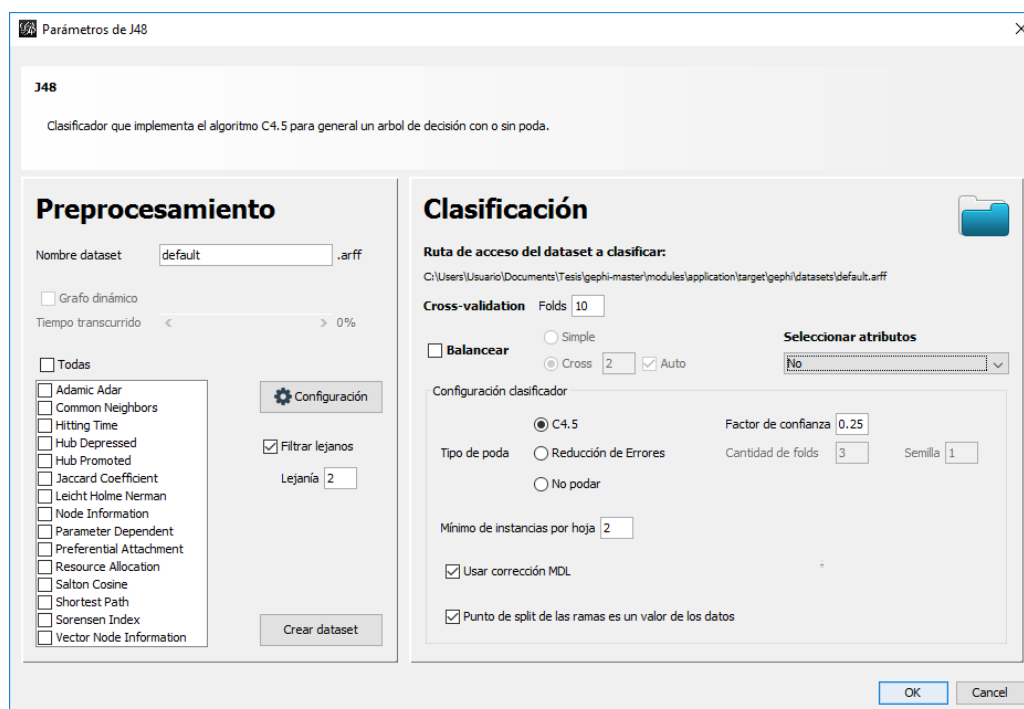


Figura 13 - Panel visual de configuración correspondiente al clasificador J48

En la captura se notan fácilmente secciones dedicadas a las distintas funcionalidades de cada una, mostrando en la sección superior el nombre y una breve descripción del clasificador en cuestión.

La sección izquierda está dedicada a la creación del dataset a partir del grafo. Se visualiza el listado de funciones de similitud a elegir, el botón de configuración que abre un panel para las funciones configurables, la caja para escribir el nombre, la funcionalidad de filtrado, y la funcionalidad para grafos dinámicos que se activa si se está trabajando con uno de ellos. El botón “Crear Dataset” invoca al algoritmo previamente explicado, inhabilitando el uso del resto del panel. Al finalizar, se notifica el tiempo que implicó la creación del dataset.

Por otro lado, la sección derecha se encarga de todo lo relacionado a la clasificación de los datasets. La parte superior se encarga de contener las cuestiones generales. Contiene el botón para seleccionar el dataset a clasificar, permite configurar la cantidad de folds de cross-validation, y provee las distintas opciones de balanceo y selección de atributos. En la parte inferior se encuentra la subsección dedicada a configurar los distintos parámetros posibles de los clasificadores. Esta configuración se traduce a un array de string al enviarse a backend. El botón “OK” invoca a backend para realizar la clasificación del dataset con la configuración elegida por el usuario.

Una decisión importante a remarcar, es la de mantener la sección izquierda y la superior derecha en todos los clasificadores. Lo que cada panel personaliza es el panel derecho inferior donde van los componentes para elegir su configuración. De esta manera toda la funcionalidad queda en el mismo panel, al alcance de todos los clasificadores, mejorando así la usabilidad para el usuario final.

Existe una excepción a los casos anteriores, la cual corresponde a la funcionalidad llamada “Comparison”, cuyo panel se observa en la Figura 14.

Figura 14 - Panel visual de configuración correspondiente a la funcionalidad Comparison

Esta funcionalidad se construyó utilizando el framework anteriormente definido, pero su objetivo es realizar una comparación rápida entre el aprendizaje supervisado y las funciones individuales. Debido a esto posee diferencias con cualquier otro clasificador, las cuales lo vuelven único. Como se puede observar, la sección derecha es totalmente diferente. Lo que se decidió fue condensar toda la configuración perteneciente a los clasificadores del lado izquierdo, y lo mismo para los predictores, pero del lado derecho.

Los listados de clasificadores y predictores poseen la capacidad de tener en cuenta futuras implementaciones de funciones de cada tipo. Sin embargo, hay una desventaja obvia la cual es la incapacidad de configurar cada función, ya sea clasificador o predictor, de forma individual. Por eso, esta función es considerada una forma simple de comparar métodos, ya que se usan las configuraciones por defecto. En caso de querer realizar una comparación en detalle, puede realizarse de forma manual ejecutando sólo las funciones más relevantes, y configurando individualmente cada una de ellas. Otra desventaja obvia es el tiempo de procesamiento, ya que el tiempo total es la suma del tiempo de procesamiento individual de cada componente que se elija para comparar.

Debido a que, el panel de Comparison posee más elementos de configuración, precisamente para los predictores, ComparisonUI se debe encargar de realizar el pasamanos de estos elementos adicionales. Para tal fin, se decidió que ComparisonUI reimplemente los métodos “setup” y “unsetup” de ClassifierAbsUI, para agregar el comportamiento específico para los nuevos elementos configurables. En la Figura 15 se observa un diagrama de clases con todo lo relacionado a los paneles visuales. Se omitieron atributos y métodos no relevantes para mejorar la legibilidad.

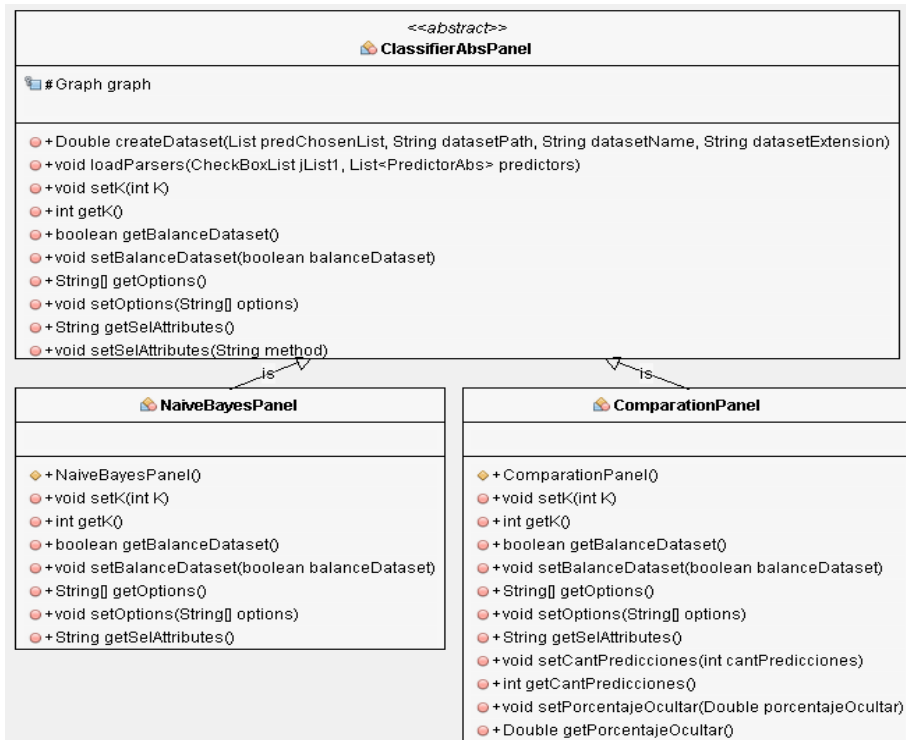


Figura 15 - Diagrama de las clases correspondientes a los paneles visuales

4.3.2. Backend

Esta sección corresponde a la implementación sobre el módulo StatisticsPlugin. Engloba toda la funcionalidad correspondiente al cálculo y creación de resultados de la clasificación. Cada función creada dentro de este módulo cumple una serie de requerimientos del diseño existente. Por cada función, debe crearse una clase encargada de todo el procesamiento que implemente la interfaz Statistics, acompañada por otra clase que implemente la interfaz StatisticsBuilder, responsable de crear las instancias de la primera. Esto es reflejo del patrón de diseño Factory Method.

Statistics define varios métodos de los cuáles, dos son los más importantes. El método “execute” implica realizar el cálculo de la función, y el método “getReport”, la creación de un string con contenido HTML, donde se vuelcan los resultados de la ejecución anterior. El módulo se encarga de mostrar el reporte HTML, y a su vez, provee acceso a las herramientas de exportación del mismo. Por su parte, StatisticsBuilder define métodos responsables de obtener la instancia y el nombre de la función. De forma adicional, se tuvo en cuenta que cada función puede implementar la interfaz LongTask. La misma provee soporte para conocer en tiempo real o cancelar el progreso del procesamiento. Resulta de suma utilidad para aquellas funciones que tomen una cantidad de tiempo considerable en completarse, como en este caso, la clasificación.

Debido a que, existe funcionalidad común a todos los clasificadores, se decidió abstraerla en una clase común y que cada clase que contiene cada clasificador herede de la anterior. De esta manera, se creó la clase abstracta ClassifierAbs que

encapsula toda la funcionalidad que tiene que ver con la clasificación y la creación de los resultados y, además, es la que implementa las interfaces `Statistics` y `LongTask`. Por lo tanto, cada clasificador posee su propia clase que hereda de `ClassifierAbs` implementando los métodos que tienen que ver con el modelo de clasificación y sus opciones de configuración. Dentro de cada clasificador de Gephi se define una instancia correspondiente a cada clasificador de la API de Weka. Acompañando a cada clasificador, existirán las distintas clases constructoras que implementan la interfaz `StatisticsBuilder` y, por lo tanto, son responsables de construir instancias del clasificador que les corresponda.

Para el caso de la clase `Comparison`, encargada del procesamiento en la comparación de métodos de predicción, la misma redefine los métodos “`execute`” y “`getReport`”, a pesar de heredar de `ClassifierAbs`. Esto se hizo ya que lo que se busca es tener en el mismo reporte, todos los reportes individuales de cada función ejecutada, ya sea función o clasificador. Para esto, en el método “`execute`” se configuran las funciones individuales y se ejecutan, mientras que en “`getReport`” se concatenan todos los reportes individuales para obtener un único reporte final. Además, la clase cuenta con métodos específicos para la configuración adicional que requieren los predictores.

Para la función de balanceo se crearon tres clases auxiliares. `ReportDataAbs` es la clase abstracta que define los métodos para almacenar y obtener datos de una clasificación. `ReportDataSimple` implementa los métodos de la clase anterior y sirve para almacenar los correspondientes a una sola clasificación. `ReportDataAvg` contiene una colección de `ReportDataAbs`. Implementa los métodos de este último para promediar los resultados de varias clasificaciones simples y retornar el promedio como si se tratara de una clasificación simple.

La Figura 16 contiene un diagrama de clases que refleja todo lo anteriormente comentado excepto las últimas tres clases, ya que no son tan importantes y su ausencia mejora la legibilidad.

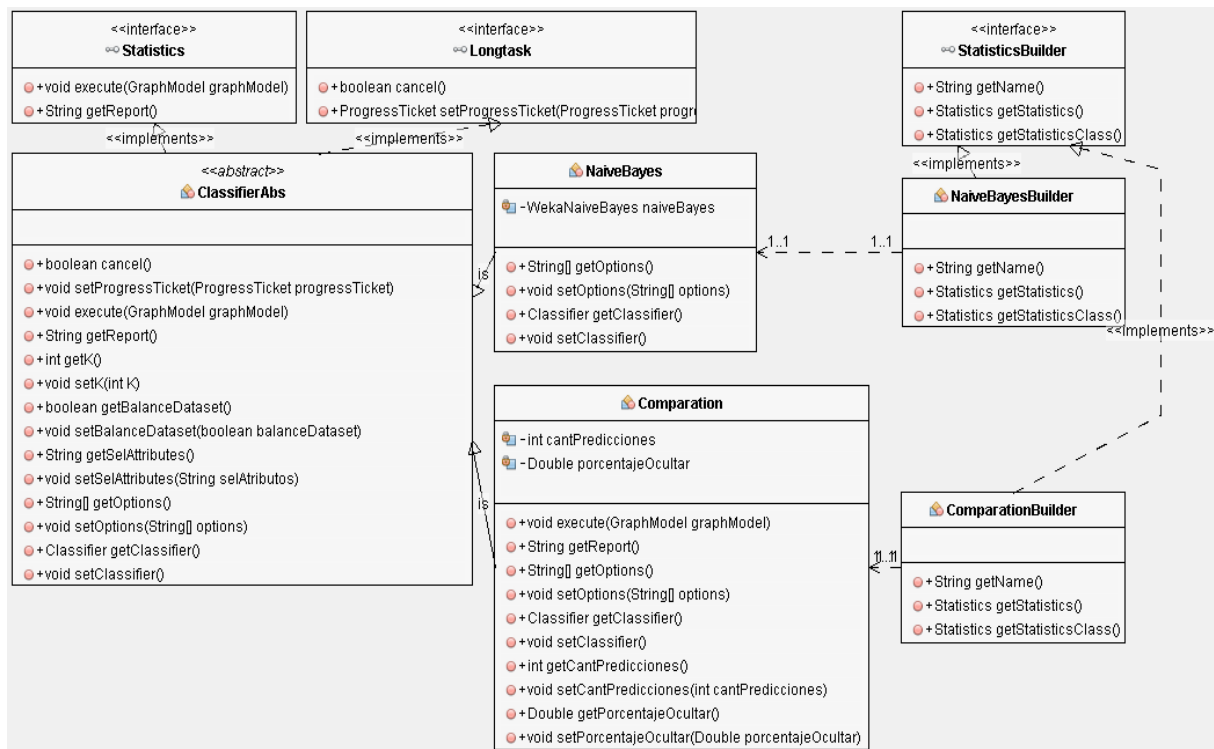


Figura 16 - Diagrama de las clases de backend

Debido a que, el algoritmo responsable de la clasificación de los datasets es de suma importancia para la presente investigación, se procede a explicar detalladamente el mismo. A continuación, se muestra el pseudocódigo para clasificar un dataset cualquiera.

Setear modelo clasificador

Si hay selección de atributos, filtrar atributos del dataset

Si hay balanceo cross

Por cada subdataset

Clasificar subdataset

Sino

Si hay balanceo simple, filtrar instancias del dataset

Clasificar dataset

Como se puede observar el algoritmo a grandes rasgos es bastante simple, pero en cada paso se resuelven cuestiones importantes. Como primer paso, se decide que modelo clasificador se utilizará. Esto depende de la función que haya elegido el usuario (por ejemplo, NaiveBayes, J48, etc). Cada algoritmo ofrece a la clasificación su modelo propio, tanto para entrenamiento como testeo, y afecta en gran medida al resultado final. En el momento que se setea el modelo, el mismo ya se encuentra configurado con las opciones que haya elegido el usuario, por eso este paso no es parte del algoritmo de clasificación.

Como segundo paso, se determina si el usuario seleccionó algún método de selección de atributos. Si aquel fuese el caso, el dataset debe ser filtrado con el

método de selección elegido. El tiempo de procesamiento de este paso depende mucho del método y del dataset y, por lo tanto, es muy variable. Como resultado se obtiene un dataset con menos atributos para la clasificación. En el caso de no elegir selección de atributos, el dataset queda igual, y no hay procesamiento. Es importante destacar, que en caso de elegir WrapperSubset, este recibe como configuración el mismo clasificador que posteriormente se utilizará para clasificar. Por lo tanto, los atributos que seleccione dependen del algoritmo clasificador elegido.

En el tercer paso hay una diferencia importante dependiendo del método de balanceo elegido. Si no se eligió balanceo cross, pero si balanceo simple, entonces el dataset es filtrado para obtener la misma cantidad de instancias de ambas clases. Como paso siguiente se realiza la clasificación del mismo y se guardan los resultados. En el caso de no haberse elegido ningún método de balanceo, el dataset no sufre ningún cambio y se realiza la clasificación.

En cambio, si se eligió balanceo cross, primero deben crearse los subdatasets, cuya cantidad está dada por el número que ingresó el usuario o por el algoritmo automático. Se procede a realizar la clasificación de cada subdataset, almacenando los resultados de cada una, para posteriormente promediar todas. Este método de clasificación es el más costoso en términos de tiempo, y debe utilizarse con consideración.

En el paso final de la clasificación, donde finalizan todas las ramas del algoritmo, primero se realiza el entrenamiento con las instancias que posea el dataset, y finalmente, se hace cross-validation para realizar la evaluación del mismo. Como salida se obtiene un reporte con toda la información relacionada a la clasificación.

Al principio de los reportes, se muestran los parámetros elegidos por el usuario para la clasificación: clasificador, folds, configuración del clasificador, balanceo, selección de atributos y predictores que quedaron en el dataset. Luego se muestra un conjunto de métricas que determinan y evalúan diferentes características de la clasificación, entre las que se encuentran Precisión, Recall, AUC y el gráfico de la curva ROC.

5. Experimentación

En esta sección se realizará una descripción de las redes elegidas, detallando las características más importantes de cada una y cualquier otra cuestión relevante relacionada al uso de las mismas. También se dispondrán los resultados de los experimentos con su respectivo análisis. Puede considerarse como una etapa muy significativa de la investigación, ya que verifica el conjunto de hipótesis sobre el cual se asienta este trabajo, y se determinan las causas del éxito o no de las técnicas implementadas.

5.1. Casos de estudio

Con el fin de contemplar la mayor cantidad de casos posibles, se seleccionaron cuatro redes con características diferentes para considerar durante la experimentación. De esta forma, se evitará obtener resultados sesgados a una única red, y se comprobará que la herramienta desarrollada es apta para analizar distintos tipos de redes.

5.1.1. Les Misérables

Esta red se corresponde con la novela “Les Misérables” escrita por Víctor Hugo y publicada en 1862. En la misma, los nodos representan a los personajes de la obra, y las aristas entre ellos indican que ambos personajes aparecen en el mismo capítulo del libro. Las principales características de la red son su tamaño e interrelación.

Cuenta con 77 nodos y 254 aristas, siendo su tamaño considerablemente pequeño para la construcción de clasificadores. Esto implica también que los resultados de las evaluaciones van a estar basados en una pequeña cantidad de ejemplos.

La interrelación dentro de la red por su parte, es otro factor importante a considerar. La red no posee una distribución uniforme de enlaces, y puede ser claramente dividida en diferentes zonas según la interconexión entre los distintos grupos de nodos. Además, la red cuenta con nodos que poseen un alto grado con respecto al promedio, teniendo algunos de ellos alrededor de 10 vecinos y hasta más de 30 vecinos en un caso particular, representando a los actores principales de la obra que están presentes en la mayoría de las escenas. En la Figura 17 se observa gráficamente la estructura de dicha red.

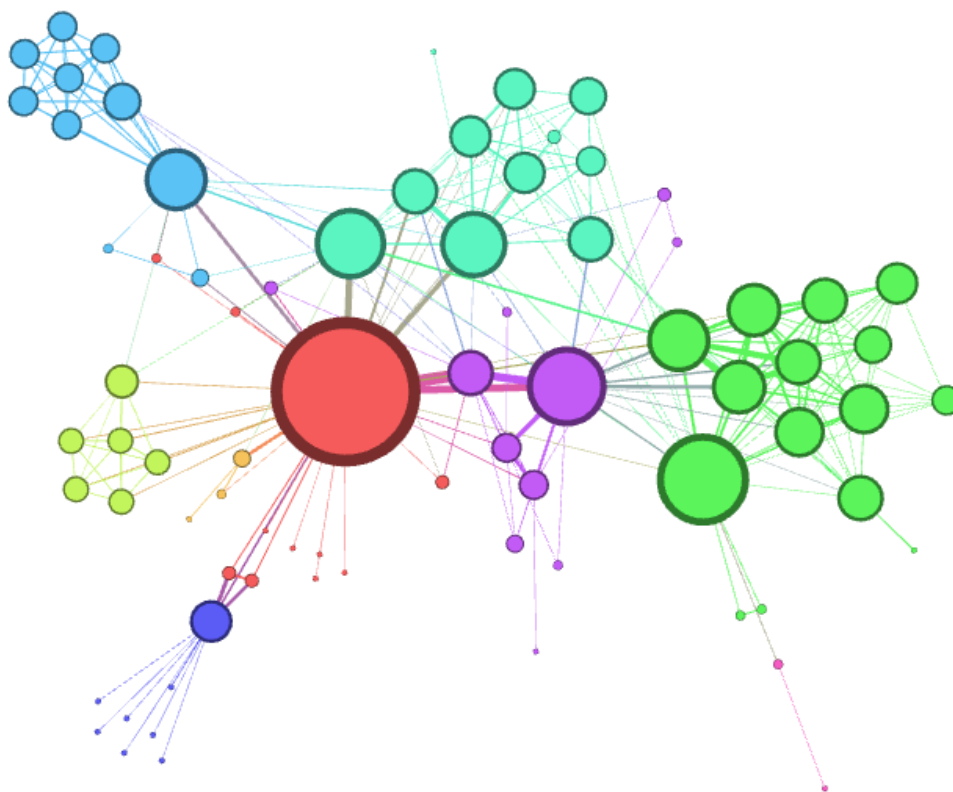


Figura 17 - Estructura visual de la red Les Misérables

En la representación de la red dentro de Gephi, se puede observar cómo los nodos con mayor grado son representados con un tamaño más grande, y a su vez, cómo los distintos agrupamientos de nodos son representados con distintos colores. En este caso, los nodos de mayor tamaño se corresponden a los personajes principales de la novela, y a medida que disminuye el tamaño de los mismos, implica que el papel de los personajes posee menos peso. Los distintos colores se corresponden a conjuntos de personajes relacionados entre sí y, por lo tanto, implica que pertenecen a distintos círculos sociales.

5.1.2. Jazz Musicians

Consiste en una red de músicos de jazz del año 2003, donde los nodos representan a músicos estadounidense y las aristas entre ellos indica que actuaron en una misma banda en algún momento.

El tamaño de la red es considerablemente mayor a la de Les Misérables, con un total de 198 nodos y 2742 aristas no dirigidas. Lógicamente, como el número de aristas es ampliamente mayor que la cantidad de nodos, el grado promedio de la red es relativamente alto. Si bien el nodo de mayor grado tiene 90 vecinos, en líneas generales el grado de los nodos es equilibrado en la red, por lo que, a diferencia de la red anterior, no existen grupos separados de nodos interconectados entre sí, sino que las aristas se distribuyen entre nodos dispersos en la red. Al poseer tal cantidad

de nodos y aristas, la representación gráfica de esta red comienza a ser confusa debido a las limitaciones visuales de Gephi. Tal cuestión se refleja en la Figura 18.

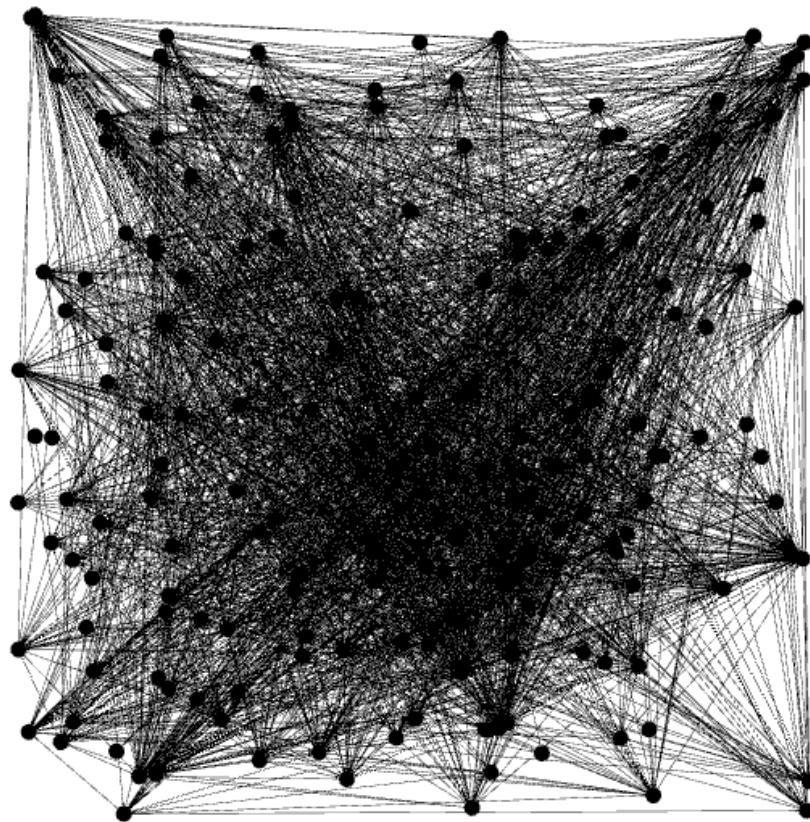


Figura 18 - Estructura visual de la red Jazz Musicians

Como puede observarse en la representación gráfica dentro de Gephi, los nodos son distribuidos de manera más uniforme, existiendo una fuerte relación entre todos ellos. Sin embargo, los nodos pueden seguir cierto patrón de ordenamiento para relacionarse entre ellos (como conjuntos de músicos más relacionados por vivir en el mismo país), y que por las limitaciones visuales de Gephi no se reflejan gráficamente. En tal caso, las funciones de similitud que mejor funcionen reflejarán algún patrón de ordenamiento de la red que se encuentre oculto a simple vista.

5.1.3. Contactos Empresa

Esta red fue diseñada manualmente en [Ricotta y Santamarina, 2016] dentro de la herramienta Gephi con el propósito de realizar pruebas que consideren la información de los nodos y evaluar el funcionamiento de los predictores “Node Information” y “Vector Node Information”.

La red cuenta con 28 nodos y 68 aristas no dirigidas, donde los nodos representan a los empleados de una empresa de desarrollo de software, y las conexiones entre ellos indican que se han intercambiado mails mutuamente. El grado promedio de la misma es de casi 5 conexiones promedio. Los atributos que posee cada nodo son:

- Identificador
- Cargo dentro de la empresa
- Ubicación de la sucursal de la empresa donde trabaja
- Proyecto al cual se encuentra asignado actualmente
- Descripción breve de la persona

Durante la creación de la red, se siguió un criterio lógico sobre las relaciones establecidas entre los nodos. Esto quiere decir que los nodos que disponen de atributos en común, tienen una mayor tendencia a relacionarse, especialmente si trabajan en el mismo proyecto, y en menor medida si trabajan en la misma sucursal. También existen conexiones esporádicas entre algunos nodos que no tienen ningún campo igual, con el fin de fortalecer la diversidad de la red y que pueda reflejar una situación real. La representación gráfica de la red se observa en la Figura 19.

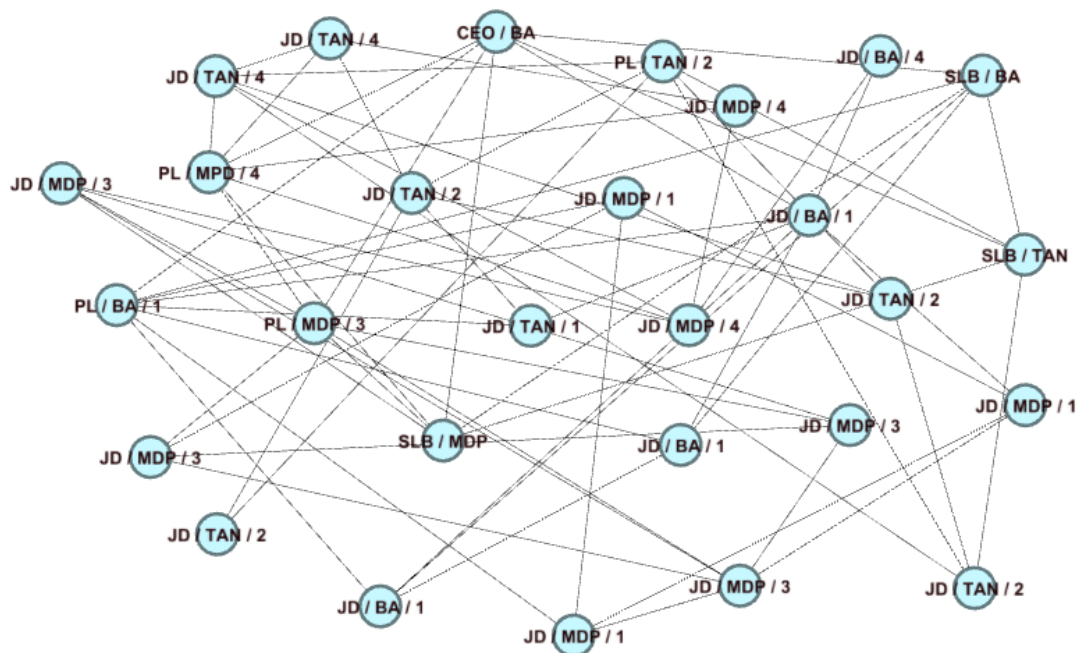


Figura 19 - Estructura visual de la red Contactos Empresa

Como puede observarse, la distribución de las aristas es relativamente uniforme, por lo que, dicha característica se asemeja a la de la red de músicos de Jazz, aunque en este caso la distribución es visible a simple vista. Sin embargo, en cuanto al tamaño de la red, es incluso más pequeña que la representación de los actores de Les Misérables. Este último hecho puede llegar a perjudicar a los clasificadores debido a la baja cantidad de instancias finales que se crearán para entrenar y validar.

En lo que respecta a la información de cada nodo, en la Figura 20 se visualizan los atributos que condensan los datos de cada uno.

Id	Label	Interval	cargo	sucursal	proyecto	descripción
0	JD / MDP / 4		Java Developer	Mar del Plata	Proyecto 4	Soy Fabián Dineno. Tengo 27 años y soy Java Developer Ssr. Me gradué en Ingeniería ...
1	SLB / TAN		Software Lab Manager	Tandil		Me llamo Santiago Toledo. Nací un 23 de enero de 1980 en Rauch, provincia de Buenos A...
2	PL / MDP / 4		Project Leader	Mar del Plata	Proyecto 4	Mi nombres es Juan Carlos López y tengo 38 años. Luego de titularme en la carrera de In...
3	PL / BA / 1		Project Leader	Buenos Aires	Proyecto 1	Soy Malena Cornejo y tengo 32 años. Al graduarme de Ingeniera Informática en la UBA, ...
4	JD / BA / 1		Java Developer	Buenos Aires	Proyecto 1	Mi nombre es Mateo Di Natale, tengo 25 años y soy de San Telmo. A los 17 años comenc...
5	SLB / BA		Software Lab Manager	Buenos Aires		Soy Rodrigo Bruno, oriundo de San Miguel del Monte. Luego de recibirme de Licenciado e...
6	JD / BA / 1		Java Developer	Buenos Aires	Proyecto 1	Mi nombre es Laura Bengochea y nació el 14 de septiembre de 1983 en Buenos Aires. Debi...
7	CEO / BA		CEO	Buenos Aires		Me llamo Diego Pérez y tengo 36 años. Soy un Analista de Sistemas con una amplia exper...
8	JD / TAN / 4		Java Developer	Tandil	Proyecto 4	Mi nombre es Lucas Patanian, tengo 27 años. Nací en General Pico, La Pampa. Al termina...
9	JD / TAN / 4		Java Developer	Tandil	Proyecto 4	Mi nombre es Mariano Pena, tengo 35 años y soy desarrollador de software. Si bien teng...
10	JD / TAN / 2		Java Developer	Tandil	Proyecto 2	Soy Lautaro Mendiola y tengo 27 años. Comencé a trabajar como desarrollador cuando e...
11	SLB / MDP		Software Lab Manager	Mar del Plata		Mi nombre es Javier Diaz, tengo 32 años y actualmente vivo en la ciudad de Mar del Plata...
12	JD / MDP / 4		Java Developer	Mar del Plata	Proyecto 4	Mi nombre es Alejandra Martinez, nació en la ciudad de Bariloche en el año 1987 y me enc...
13	JD / MDP / 1		Java Developer	Mar del Plata	Proyecto 1	Me llamo Julian Arreguy, tengo 24 años de edad y soy Ingeniero de Sistemas. Mi posició...
14	JD / MDP / 1		Java Developer	Mar del Plata	Proyecto 1	Soy Martin Motera, nació en la ciudad de Balcarce en el año 1976. He realizado diversos c...
15	JD / MDP / 1		Java Developer	Mar del Plata	Proyecto 1	Mi nombre es Lucas Poille, nació en Capital Federal y tengo 27 años. Comencé a desarrolla...
16	PL / TAN / 2		Project Leader	Tandil	Proyecto 2	Me llamo Javier Gonzalez, tengo 45 años de edad y me encuentro radicado en la ciudad d...
17	JD / TAN / 2		Java Developer	Tandil	Proyecto 2	Florencia Botta. Nací en 1990 en la ciudad de Montevideo, Uruguay. Dentro de mis estudi...
18	JD / TAN / 2		Java Developer	Tandil	Proyecto 2	Tomas Arevalo. Vivo en la ciudad de Córdoba, tengo 32 años de edad y me encuentro tr...
19	JD / TAN / 2		Java Developer	Tandil	Proyecto 2	Matias Vera. Egresado de la carrera de Ingeniería de Sistemas de la Universidad Nacional...
20	PL / MDP / 3		Project Leader	Mar del Plata	Proyecto 3	Eugenio Ditzel. Líder de proyecto para importante empresa chilena del rubro bancario. Du...
21	JD / MDP / 3		Java Developer	Mar del Plata	Proyecto 3	Mi nombre es Ricardo Perez, tengo 33 años de edad y soy especialista en la creación de ...
22	JD / MDP / 3		Java Developer	Mar del Plata	Proyecto 3	Me llamo Micaela Tassara. Nací en Santiago, Chile. Tengo 26 años y a hace ya 6 me radiq...
23	JD / MDP / 3		Java Developer	Mar del Plata	Proyecto 3	Marcelo Trujillo. Nacido en la ciudad de Barcelona en el año 1987. Soy Ingeniero en Siste...

Figura 20 - Información de los nodos de la red Contactos Empresa

5.1.4. Small Twitter

Este dataset proviene de un escaneo de una pequeña porción de Twitter, por ende, se trata de un caso real. Consta de dos archivos de tipo CSV: uno refleja las relaciones entre los usuarios, y el otro, contiene una colección de tweets publicados por los mismos. Ambos archivos están escritos en inglés.

Al tratarse de un caso real, el dataset posee un valor asociado mucho más alto desde el punto de vista de la investigación. Sin embargo, esto a su vez contrae algunos problemas. El primero de ellos, es que la información de los tweets ocupa demasiado espacio (aproximadamente 1.5 Gb) como para que la herramienta Gephi utilice el archivo. El segundo problema, es que los tweets de los usuarios se encuentran dispersos dentro de su archivo, y para poder ser útiles a la implementación de VNI presente en Gephi, los tweets pertenecientes a cada usuario deben agruparse de manera conjunta. De esta manera, los tweets se vuelven un atributo de nodo y pueden ser utilizados de manera análoga al atributo “Descripción” de la red Contactos Empresa.

Para resolver ambos problemas se utilizó un pequeño programa compuesto de un simple algoritmo, que se encargó de adaptar el archivo de tweets a un formato amigable a Gephi. El programa es externo a la extensión sobre Gephi debido a que, es exclusivo para resolver los problemas asociados a este dataset, y no de uso general. Para tal fin, el archivo fue procesado de varias maneras. Primero, se agruparon todos los tweets junto al usuario que corresponden. A continuación, se eliminaron de los tweets todos los tipos de datos no útiles a VNI (que se encarga de comparar palabras) como, por ejemplo, fechas, números, símbolos o links. Y finalmente, todas las palabras fueron procesadas por el “EnglishAnalyzer” de la librería Lucene, con el fin de borrar artículos, conectores y transformar las palabras a su base (por ejemplo, “juguemos” se transforma en “jug”). En consecuencia, los

tweets procesados son mucho más aptos para ser analizados por la técnica de VNI y el tamaño del archivo disminuyó a 545 Mb aproximadamente, un tercio del tamaño original. Como último paso se importaron ambos archivos a Gephi para crear el grafo.

La red consta de 118216 nodos y 54372 aristas dirigidas, donde cada nodo es un usuario de Twitter, y cada arista la relación de seguimiento de un usuario a otro. Además, cada nodo posee asociado el atributo "Tweets" que contiene la información de todos sus tweets. Al existir bastante menos aristas que usuarios, el grado medio del grafo es de 0,92, lo cual implica que el grafo se encuentra muy desconectado. Debido a las limitaciones de Gephi para manejar redes muy grandes, la representación visual de esta red no aporta información ni posee alguna estructura significativa. Lo mismo ocurriría de manera limitada en la red Jazz Musicians, pero para este caso, se puede apreciar perfectamente en la Figura 21.

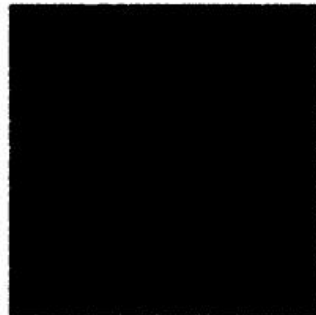


Figura 21 - Estructura visual de la red Small Twitter. Las limitaciones visuales son notorias

Debido a las limitaciones visuales se realizaron algunos filtrados, utilizando como criterio el grado de los nodos, para obtener distintas capas de la red con diferentes grados de conexión. En la Figura 22 se visualiza la estructura en distintas capas de la red donde los números indican: nodos / aristas / grado filtro.

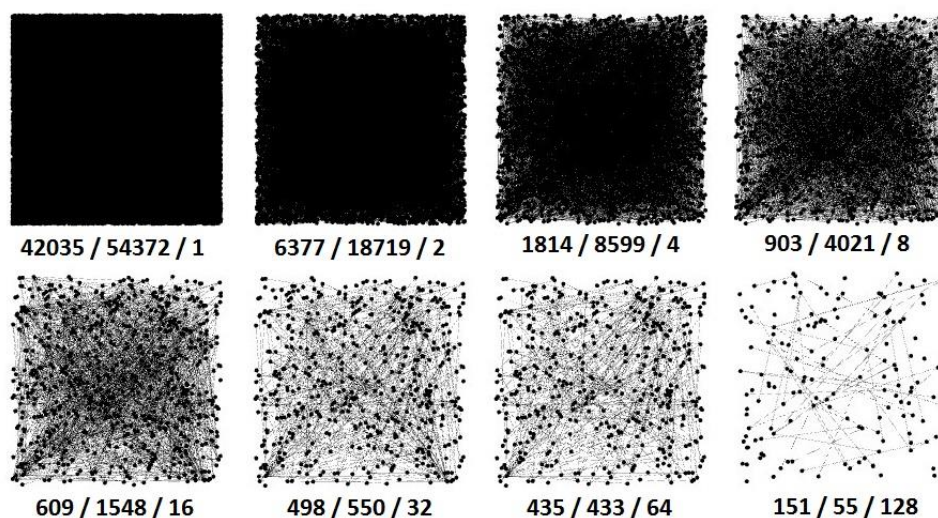


Figura 22 - Estructura de distintas capas de Small Twitter con nodos de grados diferentes

A partir de la figura anterior se concluye que, existe un gran conjunto central de nodos que concentra la mayor parte de conexiones, como, por ejemplo, los nodos con grado mayor o igual a cuatro, donde la cantidad de aristas es ocho veces mayor a la cantidad de nodos. También existe un subconjunto de nodos, pertenecientes al anterior, que poseen un grado muy alto (igual o mayor a 128), pero que no están muy interconectados entre ellos. Todo esto implica que hay un gran conjunto de usuarios de esta red que utilizan activamente Twitter porque poseen muchos seguidos y seguidores, y de esos usuarios, un pequeño grupo concentra la mayor parte de enlaces. Estos enlaces no son necesariamente entre ellos, pero debido a que, son personas más conocidas, resultan muy seguidas por el resto de usuarios de la red social.

De forma adicional se obtuvieron varias estadísticas de los datos de esta red para conocer mejor la distribución que presentan. Las mismas se encuentran en la Tabla 2.

	Promedio	Desvío	Mediana	Moda	Máximo	Mínimo
Tweets	81,65	62,82	73	1	218	1
Seguidos	1,29	11,98	0	0	171	0
Seguidores	1,29	1,25	1	1	49	0

Tabla 2 - Estadísticas por usuario de Small Twitter

Dadas las anteriores estadísticas, se pueden concluir varias cuestiones sobre la red. En el caso de los tweets, el promedio por usuario es normal. Sin embargo, debido al desvío y la moda obtenidas, se puede denotar que los usuarios de la red se dividen en dos tipos: los poco participativos que sólo poseen un tweet, tal vez a modo de presentación en la red ya que nunca más twittearon, y los muy participativos, con muchos tweets y que utilizan la red de forma activa. Las similitudes mediante la función VNI serán muy afectadas por este hecho, ya que, si la moda es de 1, significa que la mayoría de usuarios sólo tienen un tweet, disminuyendo así la probabilidad de ser parecidos.

Con respecto a los seguidos por usuario, las métricas muestran un promedio muy bajo y debido a los valores de las demás estadísticas, se concluye que, la gran mayoría de usuarios no siguen a nadie, y una escasa minoría sigue a varios a la vez. Por último, el caso de los seguidores es similar al anterior, aunque la mayoría de los usuarios posee al menos un seguidor, y el más popular es seguido por menos de 50 usuarios.

Estas últimas estadísticas son un reflejo del bajo grado medio de la red anteriormente mencionado, y como consecuencia, cualquier dataset que se calcule para posteriormente ser clasificado se encontrará extremadamente desbalanceado. Además, al existir tantos nodos, la cantidad de instancias finales será inmensa, acentuando aún más el desbalance y empeorando el rendimiento de todos los

componentes que procesan datos. Por eso se tendrán ciertas consideraciones en cada etapa, que hagan posible la experimentación con esta red.

5.2. Resultados y observaciones

En esta sección se procede a describir y realizar las diferentes etapas de experimentación definidas en la propuesta. Se explican las decisiones previas y durante cada etapa, se muestran los resultados, y finalmente, se realizan las observaciones correspondientes.

5.2.1. Pre-procesamiento

A partir de las cuatro redes mencionadas, se procesaron los correspondientes datasets con todas las consideraciones planteadas en la propuesta. Mediante el uso de la herramienta desarrollada, se generaron cuatro archivos ARFF con toda la información de cada red, representada por cada par de nodos y sus correspondientes atributos en base a los predictores. De todas formas, se tuvieron en cuenta algunas cuestiones particulares para cada red, que serán detalladas a continuación.

Parameter Dependent no fue considerado en ninguna de las redes, ya que como fue planteado en la propuesta, se utilizaron los valores de lambda correspondientes a Common Neighbors, Salton Cosine Similarity y Leicht-Holme-Nerman, descartando la infinidad de valores posibles que puede tomar este parámetro.

Al realizar pruebas pequeñas con las redes seleccionadas, se detectó que, el parámetro de cantidad de iteraciones para Hitting Time influye considerablemente en la cantidad de valores nulos. Con el fin de minimizar la cantidad de datos faltantes para este predictor, se estableció para todas las pruebas un valor de 10 en el parámetro de cantidad de iteraciones.

Como se mencionó anteriormente, sólo las redes Contactos Empresa y Small Twitter tienen información en los nodos, por lo que, los predictores Node Information y Vector Node Information no fueron tenidos en cuenta en los datasets de Les Misérables y Jazz Musicians. Tampoco fue tenido en cuenta el predictor Node Information en la red de Small Twitter, ya que la información de los nodos de la misma no está separada por atributos, sino que contiene sólo la información de los Tweets en un único atributo.

En cuanto a la red de la empresa, el predictor Node Information fue establecido con un valor de 0.33 para los atributos de “Cargo dentro de la empresa”, “Ubicación de la sucursal de la empresa donde trabaja” y “Proyecto al cual se encuentra asignado actualmente”. De esta forma, se le asigna el mismo peso a estos tres atributos, y se dejan de lado la identificación y la descripción, que no aportan información relevante para determinar la existencia de un enlace entre un par de nodos.

Debido a que, la red de Twitter es sumamente grande en relación a la capacidad de procesamiento disponible (notebooks de gama media), no se incluyeron las métricas de Shortest Path y Hitting Time para su dataset, ya que poseen una complejidad computacional considerable, que aumenta de forma proporcional al tamaño de la red.

Los cuatro datasets mencionados fueron creados incluyendo únicamente los pares de nodos que tienen vecinos en común, con el fin de eliminar el ruido provocado por la cantidad de atributos nulos, principalmente de los atributos topológicos que se basan en los vecinos en común. Sin embargo, también se crearon dos datasets adicionales para cada red, en donde sólo se varió el filtrado de instancias respecto del original, considerando en un caso los pares de nodos con un valor de Shortest Path promedio de la red, e ignorando completamente el filtrado en el otro caso.

Estos últimos datasets fueron generados con el fin de evaluar el desempeño del filtrado de pares de nodos según la distancia entre ellos. De todas formas, no se generaron estos datasets para la red de Twitter, debido a que, los cálculos de los predictores para todas las combinaciones de pares de nodos posibles demora semanas en procesarse dada la capacidad de procesamiento disponible, principalmente por la cantidad de nodos existentes y por el costo computacional del algoritmo de Shortest Path.

Entonces, a excepción de Small Twitter, se disponen de tres datasets para cada red:

- Un dataset original, donde no se aplica ningún filtro
- Un dataset donde se aplica el filtro con valor 2, es decir, donde se incluyen sólo los pares de nodos que tienen al menos un vecino en común (la métrica de Shortest Path es igual a 2, o Common Neighbors mayor a 0)
- Un dataset donde se aplica el filtro con un valor intermedio entre el valor de Shortest Path más bajo de la red para los pares de nodos que no son vecinos (siempre 2), y el valor de Shortest Path más alto de la red.

También es importante aclarar que, la métrica de Shortest Path no es tenida en cuenta en las redes generadas filtrando los pares de nodos con vecinos en común, ya que no aporta información relevante para la predicción de enlaces (su valor es siempre 2).

Estas consideraciones para la creación de los datasets son resumidas en la siguiente tabla:

Dataset	Filtro	Funciones						Cant. Instancias	Cant. "Link"	Cant. "None"
		Topológicas*	PD	SP	HT	NI	VNI			
Les Miserables	No	Si	No	Si	Si	No	No	2926	254	2672
	Medio (3)	Si	No	Si	Si	No	No	2500	254	2246

	2	Si	No	No	Si	No	No	1227	232	995
Jazz Musicians	No	Si	No	Si	Si	No	No	19503	2742	16761
	Medio (5)	Si	No	Si	Si	No	No	13813	2742	11071
	2	Si	No	No	Si	No	No	13386	2734	10652
Contactos Empresa	No	Si	No	Si	Si	Si	Si	378	68	310
	Medio (3)	Si	No	Si	Si	Si	Si	322	68	254
	2	Si	No	No	Si	Si	Si	191	61	130
Small Twitter	No	-	-	-	-	-	-	-	-	-
	Medio	-	-	-	-	-	-	-	-	-
	2	Si	No	No	No	No	Si	3034808	541	3034267

Tabla 3 - Consideraciones en la creación de datasets

Las funciones topológicas corresponden a todas las locales descritas en el marco teórico de este trabajo, exceptuando Parameter Dependent (PD):

- Common neighbors (CN)
- Jaccard coefficient (JC)
- Preferential attachment (PA)
- Adamic-Adar coefficient (AA)
- Resource allocation (AR)
- Sørensen Index (SI)
- Salton Cosine Similarity (SC)
- Leicht-Holme-Nerman (LHN)
- Hub Promoted (HP)
- Hub Depressed (HD)

Como puede verse en la Tabla 3, al filtrar los pares de nodos que no tienen ningún vecino en común, se pierden algunos enlaces reales. Estos corresponden con los nodos que están enlazados, pero no tienen ningún vecino en común. Esto implica que, si bien las métricas obtenidas en los clasificadores para estos datasets pueden mejorar debido a la disminución del ruido, no considerarán estos pocos enlaces de la red que realmente existen.

Para reflejar el funcionamiento de los filtrados, y la importancia de los mismos a la hora de disminuir el tamaño y el ruido de los datasets, se muestra en la Figura 22 un gráfico de barras con el total de valores faltantes para cada red y filtro aplicado.

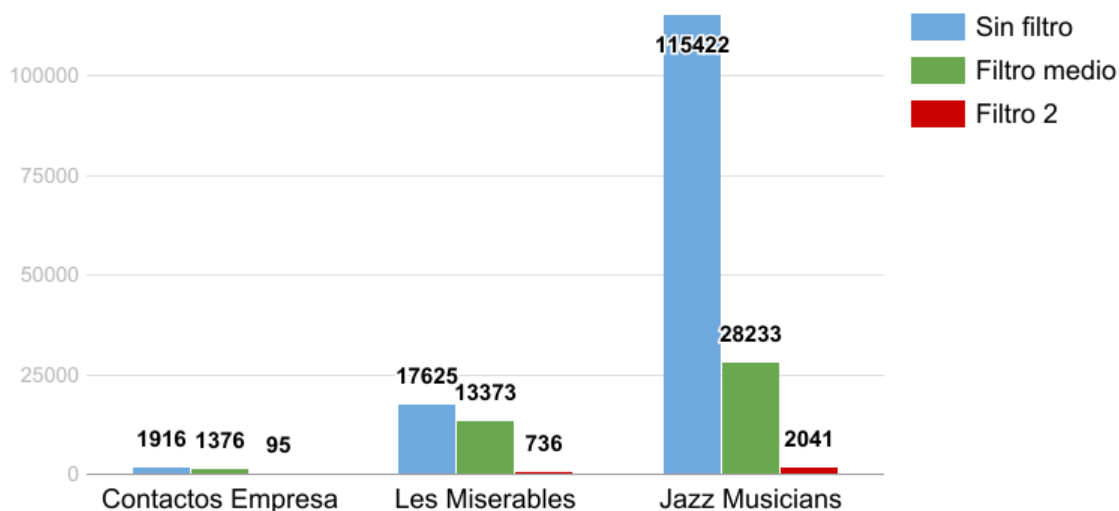


Figura 22 - Cantidad de valores faltantes para cada red y filtro

Un valor faltante en un archivo ARFF es representado con el símbolo “?”. Los valores de la tabla indican el total de símbolos “?” de cada dataset, para todas las instancias y atributos de los mismos.

Como puede observarse, la cantidad de valores faltantes en los datasets con filtro 2 es mucho menor al resto. Si bien el filtro medio disminuye considerablemente los valores faltantes, al aplicar el filtro 2 la diferencia es mucho más notoria, disminuyendo en más del 95% la cantidad de los mismos para los tres datasets. Cabe destacar que, los valores faltantes encontrados en las redes de Les Miserables y Jazz Musicians al aplicar el filtro 2, corresponden únicamente al atributo Hitting Time, mientras que a la red de “Contactos Empresas” se le suma además el atributo Node Information con algunos valores faltantes. No se incluyó la red Small Twitter en el gráfico, ya que sólo se generó el dataset con filtro 2, pero de generar un dataset sin filtro para dicha red, la cantidad de valores faltantes sería sumamente mayor.

Por otro lado, el filtrado implica una mejora en la performance del resto del proceso de clasificación, debido a la disminución del tamaño de los datasets. De esta manera cualquiera de los componentes que interactúe con los mismos, procesa una menor cantidad de instancias. En la Figura 23 se exponen las cantidades finales de instancias para las primeras tres redes y los distintos filtrados.

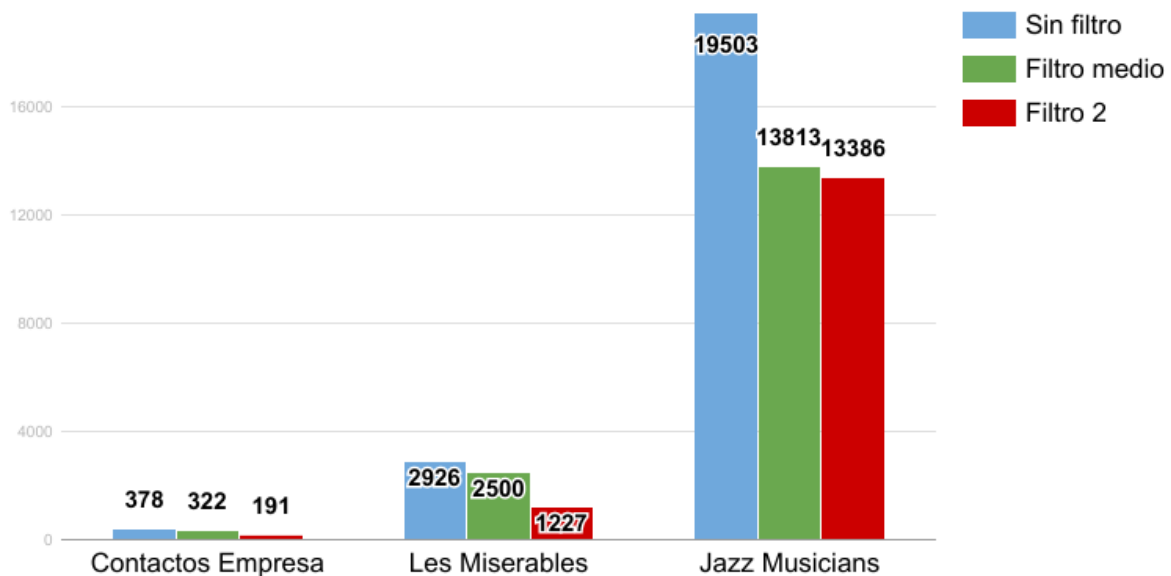


Figura 23 - Cantidad de instancias finales para cada red y filtro

Dado el gráfico de barras de la Figura 23 se concluye que, la mejora de rendimiento en procesamiento será notable a causa del filtrado. Para el filtro 2, Contactos Empresa disminuye su tamaño 49,47%, Les Miserables 58,06% y Jazz Musicians 31,36%. En este último caso la disminución fue menor debido a que, la red posee un grado medio de conexión mucho más alto que las otras dos (27,697), lo que implica una mayor presencia de nodos con vecinos comunes.

La red Small Twitter no es práctica para visualizarla con los demás datasets en un mismo gráfico debido a la diferencia de escala. Y a pesar de no haberse podido obtener el dataset sin filtro por limitaciones de hardware, si es posible calcular la cantidad de instancias que tendría y compararla con la cantidad obtenida por el filtro 2. Dada la función planteada en la sección de diseño e implementación: si una red posee M nodos, la cantidad de instancias finales sin filtrado es igual a $(M * M - M) / 2$. Esta red posee 118.216 nodos, por ende, sin filtrado se obtendrían 6.987.452.220 de instancias. Dado que el filtrado por distancia 2 produjo sólo 3.034.808 instancias, entonces la disminución del tamaño del dataset fue de 99,96%. La disminución es muy grande debido a que, esta red tiene un grado medio muy bajo (0,92), lo que implica menor cantidad de nodos con vecinos comunes.

Gracias a las mejoras con respecto a la disminución de instancias y valores faltantes, las pruebas a realizar en el resto de etapas de experimentación estarán siempre basadas en los datasets generados a partir de aplicar el filtro 2. De esta manera, las siguientes etapas del proceso utilizan los datasets con un tamaño moderado y que representan a las redes de la forma más fiel posible, como fue explicado en detalle en la propuesta.

5.2.2. Selección de atributos

Si bien en todos los datasets se consideraron la mayor cantidad de atributos posibles, muchos de estos pueden no aportar información relevante para la clasificación, ser redundantes, o incluso perjudicar los resultados de los algoritmos de clasificación. Por este motivo, se realizó una ejecución de los diferentes clasificadores para cada red, comparando el uso o no de los algoritmos de selección de atributos.

Debido a la gran cantidad de combinaciones posibles, sólo se eligió utilizar el algoritmo que teóricamente posee mayor calidad, ya que está basado en un esquema de aprendizaje: WrapperSubset - BestFirst (W). Dado que el anterior algoritmo posee un costo computacional considerable, principalmente en tiempo, su utilización en redes grandes y/o con algoritmos de clasificación costosos resulta impracticable. Para estos casos particulares, que tarden como mínimo varias horas, se optó por utilizar el algoritmo Correlation - Best First (C) en su lugar. El mismo posee menor costo computacional y menor calidad que el anterior algoritmo. Sin embargo, se presupone que es mejor que no utilizar selección de atributos directamente. Tal cuestión será resuelta de forma empírica en los experimentos.

Para las clasificaciones se consideró la técnica 10-Fold Cross Validation, ya que se busca analizar el funcionamiento general de cada uno. Se utilizaron 5 clasificadores distintos y variados para evitar sesgo. De esta forma, las conclusiones que se realicen a partir de los experimentos son independientes del modelo de clasificación utilizado.

La métrica seleccionada como la más importante para la comparación de experimentos, es el Área Bajo la Curva ROC (AUC). Esta decisión tiene como sustento lo explicado en el marco teórico, donde se concluye que, la misma engloba el desempeño del clasificador tanto para los casos positivos como para los casos negativos. A su vez es independiente de la distribución de clases, por lo que, es ideal para determinar qué dataset es más representativo sin importar si el conjunto de datos está desbalanceado. Es importante remarcar que el algoritmo Wrapper fue configurado para buscar el conjunto de atributos que maximice AUC.

En la siguiente tabla se indica el algoritmo de selección utilizado para cada dataset y clasificador, y se marca con una cruz (X) los atributos elegidos por dicho algoritmo. Además, para cada red, se marcaron con azul los atributos más elegidos, y con gris los atributos que no pudieron ser considerados.

Red	Clasificador	Sel.	CN	JC	PA	AA	RA	SI	SC	LHN	HP	HD	HT	NI	VNI
Les Miserables	Naive Bayes	W	X				X				X			N/A	N/A
	J48	W					X					X		N/A	N/A
	Random Forest	C	X			X	X							N/A	N/A
	IBk	W				X	X				X			N/A	N/A

	SMO	W					X					X	X	N/A	N/A
Jazz Musicians	Naive Bayes	W					X							N/A	N/A
	J48	W	X			X	X				X	X		N/A	N/A
	Random Forest	C				X	X		X		X		X	N/A	N/A
	IBk	C				X	X		X		X		X	N/A	N/A
	SMO	W			X		X			X	X			N/A	N/A
Contactos Empresa	Naive Bayes	W				X					X				
	J48	W	X	X		X	X				X		X		
	Random Forest	C				X			X	X	X	X			X
	IBk	W	X			X	X				X				
	SMO	W							X		X				X
Small Twitter	Naive Bayes	C	X	X						X				N/A	
	J48	C	X	X						X				N/A	
	Random Forest	C	X	X						X				N/A	
	IBk	-	X	X						X				N/A	
	SMO	C	X	X						X				N/A	

Tabla 4 - Atributos seleccionados en cada dataset

Se puede observar que en aquellos datasets donde se utilizó Correlation como algoritmo de selección, los atributos elegidos son los mismos sin importar el clasificador. En cambio, para los casos donde se utilizó Wrapper, existen variaciones en las selecciones para un mismo dataset. Esto se debe a que, el algoritmo de Wrapper depende del clasificador, como fue explicado en la implementación, mientras que Correlation sólo considera la información del dataset. Ambas técnicas de selección utilizan BestFirst como algoritmo de búsqueda, por lo tanto, las diferencias se encuentran en el algoritmo de evaluación. Correlation es más rápido, pero al ser independiente del algoritmo de clasificación, su criterio puede no favorecer los resultados de las clasificaciones en todos los casos, por lo que, no garantiza un aumento en el AUC. Como contrapartida, Wrapper es mucho menos eficiente, pero su criterio se adapta a cada clasificador, implicando selecciones de mayor calidad.

Conociendo ya los atributos que fueron seleccionados para cada dataset, se comenzarán a analizar las métricas obtenidas por las clasificaciones, y las diferencias con respecto a las clasificaciones con los datasets completos.

Adicional al AUC ya mencionado, se incluyen otras métricas importantes que permiten interpretar más detalladamente los resultados de las evaluaciones. Estas son: Precisión y Recall (incluyendo también los valores para positivos y negativos), y RAE como métrica representativa del error.

Para cada métrica a considerar, se marcó con colores la diferencia de rendimiento entre cada clasificación, usando la técnica de selección de atributos en un caso, y no usándola en el otro. Por ende, se remarcó la mejora o desmejora de la técnica con respecto a la clasificación común. Si se trata de mejora se marcó en verde, si es una desmejora, en rojo. Para el caso particular del RAE, donde un mayor valor implica un peor desempeño del clasificador, se considera una desmejora cuando su valor es más grande al utilizar el algoritmo de selección.

Clasificador	Selección	AUC	Precisión	Recall	Precisión (+)	Precisión (-)	Recall (+)	Recall (-)	RAE
Naive Bayes	No	93,86	93,22	93,32	83,80	95,40	80,20	96,40	22,09
	W	94,17	94,36	94,46	88,00	95,80	81,90	97,40	20,04
J48	No	91,26	93,69	93,81	85,80	95,50	80,60	96,90	31,47
	W	87,78	93,85	93,97	86,60	95,50	80,60	97,10	34,39
Random Forest	No	95,29	93,53	93,64	85,30	95,40	80,20	96,80	25,19
	C	90,19	94,68	94,78	90,40	95,70	81,00	98,00	24,58
IBk	No	83,60	87,78	86,47	61,70	93,90	75,00	89,10	44,26
	W	94,01	95,20	95,27	89,90	96,40	84,50	97,80	17,22
SMO	No	87,46	94,02	94,13	90,80	94,80	76,70	98,20	19,11
	W	88,00	94,09	94,21	90,00	95,00	78,00	98,00	18,84
Promedios	No	90,29	92,44	92,27	81,48	95,00	74,54	95,48	28,42
	Si	90,83	94,43	94,53	88,98	95,68	81,20	97,66	23,01
Mejora de la selección		+ 0,54	+ 1,99	+ 2,26	+ 7,50	+ 0,68	+ 6,66	+ 2,18	- 5,41

Tabla 5 - Resultados de la selección en Les Misérables

Clasificador	Selección	AUC	Precisión	Recall	Precisión (+)	Precisión (-)	Recall (+)	Recall (-)	RAE
Naive Bayes	No	95,89	91,39	90,45	72,30	96,30	86,20	91,50	29,65
	W	96,27	92,53	92,64	83,70	94,80	79,50	96,00	30,86
J48	No	94,56	93,66	93,75	86,50	95,50	82,20	96,70	27,90
	W	95,25	93,74	93,84	87,20	95,40	81,90	96,90	28,10
Random Forest	No	96,23	94,00	94,00	85,40	96,20	85,30	96,20	25,47
	C	96,28	94,16	94,19	86,30	96,20	85,10	96,50	24,33
IBk	No	85,18	90,64	90,74	78,30	93,80	75,70	94,60	28,49
	C	84,98	90,63	90,77	78,70	93,70	75,10	94,80	28,42
SMO	No	88,61	93,43	93,55	87,20	95,00	80,20	97,00	19,83
	W	88,85	93,49	93,61	87,00	95,20	80,80	96,90	19,64
Promedios	No	91,99	92,62	92,49	81,94	95,36	81,92	95,18	26,26
	Si	92,32	92,91	93,01	84,58	95,06	80,48	96,22	26,27
Mejora de la selección		+ 0,33	+ 0,29	+ 0,52	+ 2,64	-0,30	- 1,44	- 1,04	+ 0,01

Tabla 6 - Resultados de la selección en Jazz Musicians

Clasificador	Selección	AUC	Precisión	Recall	Precisión (+)	Precisión (-)	Recall (+)	Recall (-)	RAE
Naive Bayes	No	79,57	77,06	77,49	66,10	82,20	60,70	85,40	54,09
	W	78,65	77,25	78,01	70,20	80,60	54,10	89,20	65,13
J48	No	69,90	73,68	74,87	65,10	77,70	45,90	88,50	71,51
	W	74,96	75,37	75,92	63,60	80,90	57,40	84,60	66,14
Random Forest	No	84,54	74,87	74,87	60,70	81,50	60,70	81,50	59,86
	C	84,61	79,67	79,58	67,70	85,30	68,90	84,60	55,95
IBk	No	67,43	72,25	71,73	55,40	80,20	59,00	77,70	65,49
	W	85,37	81,81	82,20	75,50	84,80	65,60	90,00	46,37

SMO	No	69,59	76,09	76,96	69,80	79,10	49,20	90,00	52,89
	W	69,87	78,41	78,53	77,80	78,70	45,90	93,80	49,28
Promedios	No	74,20	74,79	75,18	63,42	80,14	55,10	84,62	60,76
	Si	78,69	78,50	78,84	70,96	82,06	58,38	88,44	56,57
Mejora de la selección		+ 4,49	+ 3,71	+ 3,66	+ 7,54	+ 1,92	+ 3,28	+ 3,82	- 4,19

Tabla 7 - Resultados de la selección en Contactos Empresa

Clasificador	Selección	AUC	Precisión	Recall	Precisión (+)	Precisión (-)	Recall (+)	Recall (-)	RAE
Naive Bayes	No	97,40	99,97	97,84	0,50	100,00	57,90	97,80	6424,67
	C	97,34	99,97	99,42	0,50	100,00	17,40	99,40	2258,59
J48	No	94,86	99,98	99,98	85,90	100,00	15,70	100,00	82,82
	C	49,92	99,96	99,98	0,00	100,00	0,00	100,00	99,89
Random Forest	No	74,03	99,98	99,98	67,60	100,00	26,20	100,00	64,76
	C	54,94	99,97	99,98	18,60	100,00	3,90	100,00	90,76
IBk	-	-	-	-	-	-	-	-	-
	-	-	-	-	-	-	-	-	-
SMO	No	50,00	99,96	99,98	0,00	100,00	0,00	100,00	49,95
	C	50,00	99,96	99,98	0,00	100,00	0,00	100,00	49,95
Promedios	No	79,07	99,97	99,44	38,50	100,00	24,95	99,45	1655,55
	C	65,75	99,96	99,84	4,77	100,00	5,32	99,85	624,65
Mejora de la selección		- 13,32	- 0,01	+ 0,40	- 33,73	-	- 19,63	+ 0,40	- 1030,90

Tabla 8 - Resultados de la selección en Small Twitter

A partir de los datos obtenidos en los experimentos de esta etapa, se concluyen varias cuestiones.

En las clasificaciones donde se usa Wrapper como mecanismo de selección, se esperaba encontrar la mejor combinación de atributos que maximice el AUC. Sin embargo, existen algunos casos en los que esto no sucede. Esto se debe a que, el

algoritmo de búsqueda utilizado no explora todas las alternativas posibles, sino que comienza a añadir atributos a su selección hasta que ninguno de los atributos restantes implique una mejora en el resultado del AUC final.

Por lo tanto, si bien Wrapper intenta garantizar una mejora en el AUC, el algoritmo de búsqueda seleccionado no lo hace. En su lugar, y para buscar garantizar que nunca empeore el AUC con respecto a la clasificación con todos los atributos, se podría utilizar un algoritmo que comenzara la búsqueda a partir de todos los atributos, y descartar aquellos que impliquen una mejora en el resultado del AUC. Sin embargo, esta decisión implicaría un tiempo de procesamiento muy elevado, principalmente en las redes grandes o con los clasificadores más complejos.

Por su parte, para los casos donde se utiliza Correlation en lugar de Wrapper, no se esperaba necesariamente una mejora en el AUC, ya que esta no es tenida en cuenta en el criterio de selección. De todas formas, se pretendía que la clasificación general obtenga mejores resultados, lo cual no ocurrió en muchos de los casos.

En la Figura 24 se puede apreciar la diferencia de los valores del AUC para cada clasificación y red, comparando las clasificaciones que utilizaron todos los atributos posibles y las que aplicaron un algoritmo de selección.

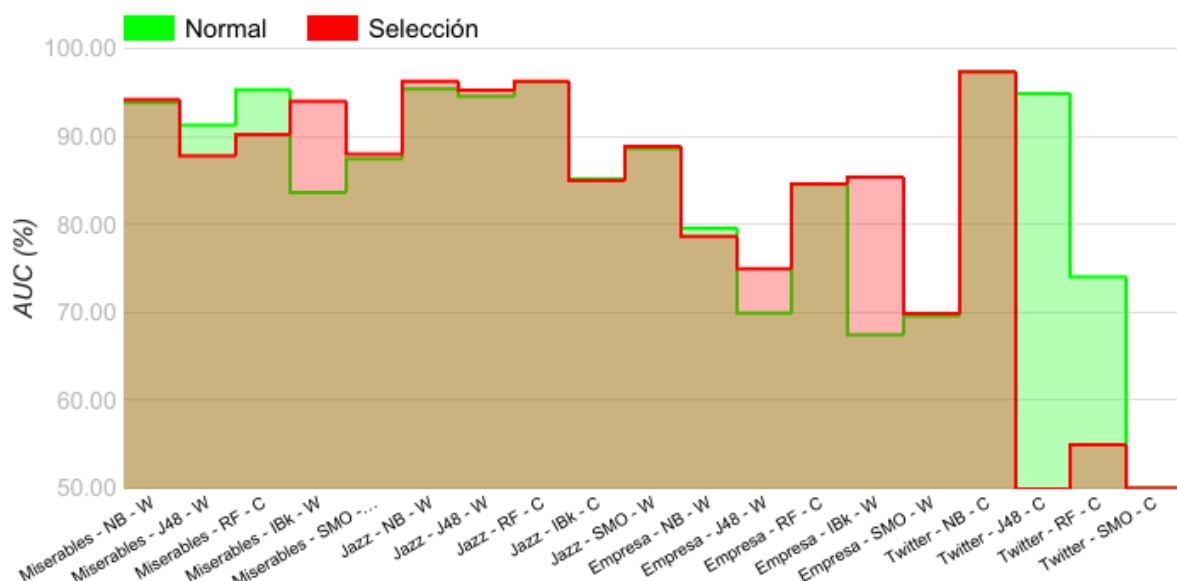


Figura 24 - Comparación entre seleccionar atributos o no, para cada red y clasificador

En términos de costo computacional, existe una disminución en aquellos casos en los cuales el costo de procesamiento asociado a la selección, sumado al costo de realizar la clasificación con sólo el conjunto de atributos seleccionados, es menor al costo de realizar la clasificación con todos los atributos. Esta mejora en el costo computacional se aplica principalmente en los casos donde se utiliza Correlation como algoritmo de búsqueda, y en los datasets de mayor tamaño. En cambio, para datasets pequeños, la mejora es imperceptible.

Dado un clasificador cualquiera, existe una cantidad de instancias y tiempo determinados a partir de los cuáles conviene utilizar la selección en conjunto a la clasificación. Tal cuestión se visualiza en la Figura 25, en donde se relaciona la

cantidad de instancias y el tiempo de clasificación mediante una función aproximada. En términos de tiempo computacional, para una cantidad de instancias menor o igual a Q es conveniente sólo clasificar, y para cantidades mayores a Q , conviene seleccionar los atributos y después clasificar. Dependiendo del dataset, el algoritmo de selección y el modelo clasificador, el punto Q varía su posición en el eje X, pero la relación entre ambas funciones se mantiene.

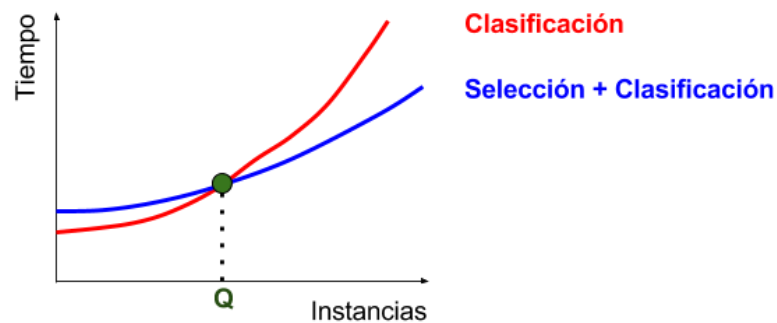


Figura 25 - Relación entre instancias y tiempo, para selección y clasificación

Como análisis general, puede observarse que para los datasets más pequeños, como Les Miserables y Contactos Empresa, las clasificaciones con selección de atributos obtuvieron mejores resultados en todas las métricas analizadas. Por su parte, las redes más grandes no reflejaron mejoras significativas, e incluso arrojaron peores resultados para determinadas métricas.

A continuación, se realiza un análisis individual de cada dataset, a partir de los resultados arrojados en las diferentes clasificaciones.

En primer lugar, se realizaron experimentos sobre el dataset Les Miserables, ubicados en la Tabla 5. En el mismo, los algoritmos de selección eligieron Resource Allocation como un atributo representativo del dataset, sin importar el clasificador elegido. Esto se debe a que, RA no sólo utiliza la información de los vecinos comunes, sino de los vecinos de los vecinos como AA, obteniendo una visión más amplia de la red. De esta manera los personajes más secundarios de la obra también tienen oportunidad de aportar información con respecto a las posibilidades de formar relaciones.

Con respecto al resto de los clasificadores, las elecciones de los algoritmos de selección varían mucho y se enfocan en beneficiar a cada clasificador particular. Para los valores finales de AUC, la técnica de selección produce una mejora para tres de los cinco clasificadores, con un aumento mayor al 0,5% en promedio. La mejora de rendimiento en costo computacional es imperceptible ya que el dataset es muy pequeño.

Como se visualiza en la Tabla 6, para el dataset Jazz Musicians los predictores más elegidos son Resource Allocation y Hub Promoted. En el caso de RA, el motivo es el mismo que en el anterior dataset, pero aún más justificado debido a que, el grado promedio del grafo (27,7) es muy alto. En el caso de HP, la elección de este predictor implica que los nodos de la red tienden a conectarse por nodos centrales (hubs), los cuales son aquellos músicos que participaron en muchos grupos.

Con respecto a la mejora de AUC, la misma se observa en mayor o menor medida para todos los clasificadores, excepto IBk, lo cual se debe al uso de Correlation como algoritmo de selección de atributos, como consecuencia del gran tamaño de este dataset y de la complejidad del algoritmo de clasificación. Sin embargo, el mismo algoritmo funciona de forma satisfactoria junto al clasificador Random Forest, aunque este último caso se debe a que, los criterios de la selección y la clasificación con respecto a los atributos importantes coinciden.

En términos generales, la técnica de selección también fue satisfactoria para este dataset, y aunque el aumento de AUC promedio fue menor a 0,5%, la mejora en el tiempo de cómputo se empieza a notar para los clasificadores que exigen más procesamiento.

En el caso del dataset Contactos Empresa, cuyos resultados se encuentran en la Tabla 7, los atributos más elegidos por la selección son Adamic Adar y Hub Promoted. AA funciona de manera muy similar a RA, sólo que AA fue mejor considerada por el algoritmo de selección en esta red. En la misma también existen varios hubs, los cuales son aquellos trabajadores que debido a su origen y posición conectan muchos trabajadores de distintas ciudades y/o puestos, lo cual beneficia a HP. Es importante destacar que, NI sólo fue elegido en dos ocasiones y VNI en ninguna, por lo tanto, la información de los nodos no fue considerada de suma importancia por los algoritmos de selección para la predicción de enlaces.

Por otro lado, la mejora de AUC se refleja para casi todos los clasificadores con un promedio de aumento de 4,49%, lo cual se considera bastante significativo. En términos de costo computacional, la mejora es nuevamente imperceptible debido a que, este dataset es sumamente pequeño.

En último lugar, se analizó la Tabla 8 correspondiente al dataset Small Twitter. Debido a la gran cantidad de instancias pertenecientes al mismo, ocurrieron varios problemas que limitaron el análisis de la selección de atributos. Utilizar Wrapper como algoritmo de selección es inviable debido a que, se necesita demasiado procesamiento para obtener los resultados a partir de semejante cantidad de instancias. Por eso, se optó por realizar todos los experimentos con Correlation que, a pesar de no ser el mejor algoritmo de selección, es lo suficientemente rápido como para obtener una respuesta en un tiempo moderado. Al existir un único criterio de selección, no es posible determinar los atributos más representativos del dataset, y por este motivo no fueron marcados en la Tabla 4.

Otro de los problemas que surgieron fue la incapacidad de respuesta del clasificador IBk. Como se explicó en el marco teórico, el mismo realiza la mayor parte del procesamiento en la etapa de evaluación de la clasificación, y al necesitar clasificar tantas instancias, el tiempo de procesamiento es demasiado elevado. Por lo tanto, se descartó como clasificador a considerar para este dataset.

Con respecto a la eficacia y eficiencia de la selección de atributos en este dataset, los resultados son negativos. Por un lado, la mejora en el costo computacional es notoria debido al gran tamaño del dataset, incluso reduciéndose a la mitad en ciertos casos. Sin embargo, dado que la variación de AUC fue negativa o nula en

todos los casos, esta mejora queda completamente opacada ya que la selección de atributos no tiene ninguna utilidad.

La baja eficacia de la selección de atributos en este caso es producto de diversos factores. Uno de los factores se puede atribuir a que el criterio con el que el algoritmo de selección Correlation evaluó a los atributos no coincide con los criterios considerados importantes por cada clasificador. Para una buena eficacia en la selección debería haberse utilizado Wrapper, lo cual resulta impracticable dado el tamaño de este dataset. Por este motivo se concluye que, a menos que se cuente con una máquina con el hardware necesario, no es posible realizar selección sobre este dataset sin afectar negativamente al valor de AUC final.

Otro de los factores que pudo afectar negativamente al valor de AUC final es el desbalance desmesurado que presenta el dataset. En el mismo, el 0.018% de sus instancias pertenece a la clase “Link”, mientras que el 99,982% restante a la clase “None”. Este tipo de desbalances excesivos afectan negativamente a muchos de los aspectos de la clasificación, lo cual se refleja en las métricas finales de los experimentos, especialmente las que se relacionan con la clase minoritaria. Tales cuestiones son abordadas en la siguiente etapa del proceso de experimentación.

5.2.3. Balanceo

En este tipo de datasets, el balanceo es un problema constante y ocurre en gran parte debido a que, el grado medio del grafo que representa una red social suele ser muy bajo. Esto conlleva a que los datasets que representan los diferentes pares de nodos contengan por lo general una gran cantidad de clases con valor negativo, y sólo unas pocas con valor positivo. Esta relación entre desbalance y el grado medio se puede observar en la Figura 26. Las cantidades de instancias de cada clase se ponen como porcentajes del total para visualizar todas las redes en el mismo gráfico, y el desbalance se calcula restando el porcentaje de la clase mayoritaria con el de la minoritaria.

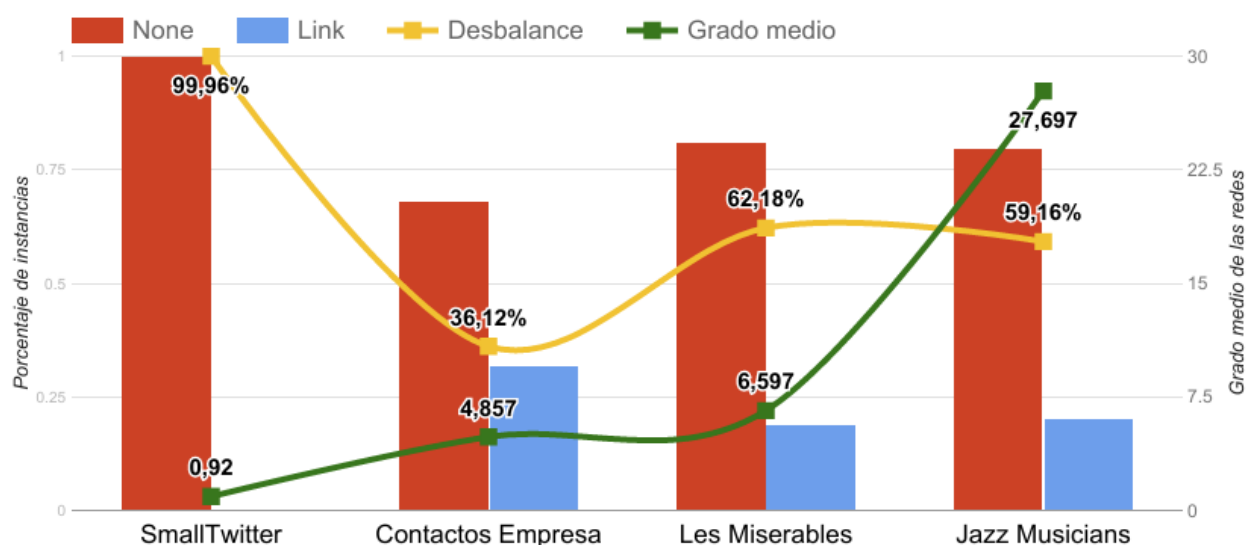


Figura 26 - Relación entre desbalance y grado medio para los datasets

Existen otros factores que también afectan al desbalance, como puede ser la estructura topológica de la red, o nodos hiperconectados cuyos vecinos no poseen otro enlace además de ellos mismos.

En consecuencia, se propusieron diferentes técnicas de balanceo que permiten tratar este problema. A pesar de que AUC no es afectado por el desbalance, existen ciertas métricas que si lo son, y que reflejan en cierta medida la calidad del clasificador.

Las principales métricas que se ven afectadas al tratar un dataset desbalanceado son las métricas de error, que reflejan de diferentes formas la correlación entre los valores reales y los predichos. Para esta experimentación, se tendrá en cuenta la métrica de Error Absoluto Relativo (RAE), ya que, al ser un valor relativo, indica un valor porcentual que permite comparar con mayor facilidad los diferentes casos. A diferencia del resto de las métricas que se han analizado, mientras más bajos son los valores del RAE, mejor resulta la clasificación, ya que es una medida de error.

Otra métrica a considerar en los datasets desbalanceados es la precisión. En un dataset desbalanceado, la precisión puede ser alta debido al sesgo del clasificador. En estos casos, el clasificador puede tener un gran porcentaje de aciertos en general, pero puede estar implicado por la cantidad de aciertos de casos negativos. Por lo tanto, si bien el área bajo la curva es una métrica que abarca los casos positivos y negativos, es importante analizar la precisión de ambas variantes por separado.

La misma consideración mencionada para la precisión, es también aplicable al recall. Entonces, una buena forma de analizar estos casos de forma más general, es a través de la métrica llamada F-Measure. Esta última combina ambas métricas a través del cálculo de la media armónica de sus valores. De la misma forma que se calcula la precisión y el recall para los casos negativos y positivos, también puede calcularse el F-Measure en ambos casos, así como su valor ponderado.

Con el fin de analizar el funcionamiento de las técnicas de balanceo implementadas, se corrieron para cada red los mismos clasificadores del punto anterior, utilizando los mismos algoritmos de selección, pero agregando un tipo de balanceo. De esta forma, los resultados de ambas clasificaciones pueden ser comparados teniendo en cuenta las métricas más importantes a considerar para dicha comparación: AUC, RAE, F-Measure ponderado y F-Measure para los casos positivos y negativos.

Es importante destacar que, para las ejecuciones con balanceo se utilizaron los mismos algoritmos de selección que para las ejecuciones sin balanceo. Sin embargo, para la red Small Twitter, no se utilizó ningún algoritmo de selección para ninguno de los dos casos debido al bajo rendimiento obtenido en la anterior etapa. En su lugar, se considerarán todos los atributos iniciales del dataset para ambos casos. Otra razón para descartar la selección en este dataset, es que no considera VNI como atributo representativo, siendo esta una información condensada de los tweets de los usuarios, considerada de importancia para los experimentos.

En cuanto a los algoritmos de balanceo, se utilizará la técnica de balanceo de tipo “Cross Automático”, la cual consiste en la más precisa de todas ya que evita la obtención de resultados sesgados por la selección de casos negativos de forma aleatoria.

A continuación, en las siguientes tablas se muestran los resultados de las métricas obtenidas para las cuatro redes, tanto para los casos sin balancear como balanceados. Para cada balanceo, se indica entre paréntesis la cantidad de datasets generados para las clasificaciones. A su vez, para las clasificaciones balanceadas se remarca en color verde o rojo sobre cada resultado según si el resultado mejoró o empeoró respectivamente de la clasificación sin balancear. También se indica para cada métrica, la diferencia entre los promedios de todos los clasificadores en ambas pruebas.

Clasificador	Tipo balanceo	AUC	RAE	F-Measure promedio (ponderado)	F-Measure de Positivos	F-Measure de Negativos
Naive Bayes	No	94,17	20,04	94,40	84,80	96,60
	Cross Auto (4)	93,91	21,62	89,53	88,50	90,50
J48	No	87,78	34,39	93,90	83,50	96,30
	Cross Auto (4)	88,34	34,78	89,20	88,18	90,18
Random Forest	No	90,19	24,58	94,70	85,50	96,80
	Cross Auto (4)	92,88	27,66	88,97	88,30	89,68
IBk	No	94,01	17,22	95,20	87,10	97,10
	Cross Auto (4)	92,82	20,05	91,30	90,90	91,68
SMO	No	88,00	18,84	94,10	83,60	96,50
	Cross Auto (4)	88,08	23,25	88,25	86,82	89,65
Promedios	No	90,83	23,01	94,46	84,90	96,72
	Cross Auto (4)	91,22	25,47	89,45	88,54	90,33
Mejora del balanceo		+0,40	+2,46	-5,01	+3,64	-6,39

Tabla 9 - Impacto del balanceo en Les Miserables

Clasificador	Tipo balanceo	AUC	RAE	F-Measure promedio (ponderada)	F-Measure de Positivos	F-Measure de Negativos
Naive Bayes	No	96,27	30,86	92,60	81,50	95,40
	Cross Auto (3)	96,28	28,42	90,13	88,30	91,50
J48	No	95,25	28,10	93,80	84,50	96,20
	Cross Auto (3)	94,96	25,58	91,37	89,97	92,43
Random Forest	No	96,28	24,33	94,20	85,70	96,40
	Cross Auto (3)	96,42	22,28	91,90	90,70	92,80
IBk	No	84,98	28,42	90,70	76,90	94,20
	Cross Auto (3)	87,40	25,62	87,40	85,37	88,97
SMO	No	88,85	19,64	93,50	83,80	96,00
	Cross Auto (3)	91,17	17,39	91,43	90,07	92,50
Promedios	No	92,32	26,27	92,96	82,48	95,64
	Cross Auto (3)	93,24	23,85	90,44	88,88	91,64
Mejora del balanceo		+0,92	-2,42	-2,52	+6,40	-4,00

Tabla 10 - Impacto del balanceo en Jazz Musicians

Clasificador	Tipo balanceo	AUC	RAE	F-Measure promedio (ponderada)	F-Measure de Positivos	F-Measure de Negativos
Naive Bayes	No	78,65	65,13	77,10	61,10	84,70
	Cross Auto (2)	78,40	65,49	70,80	66,05	75,30
J48	No	74,96	66,14	75,60	60,30	82,70

	Cross Auto (2)	76,02	67,33	72,60	72,45	72,80
Random Forest	No	84,61	55,95	79,60	68,30	84,90
	Cross Auto (2)	81,57	59,09	74,20	73,25	75,10
IBk	No	85,37	46,37	81,80	70,20	87,30
	Cross Auto (2)	81,02	52,73	73,40	72,65	74,10
SMO	No	69,87	49,28	76,70	57,70	85,60
	Cross Auto (2)	71,07	57,18	71,05	66,95	74,85
Promedios	No	78,69	56,57	78,16	63,52	85,04
	Cross Auto (2)	77,61	60,36	72,41	70,27	74,43
Mejora del balanceo		-1,08	+3,79	-5,75	+6,75	-10,61

Tabla 11 - Impacto del balanceo en Contactos Empresa

Clasificador	Tipo balanceo	AUC	RAE	F-Measure promedio (ponderada)	F-Measure de Positivos	F-Measure de Negativos
Naive Bayes	No	97,40	6424,67	98,90	0,90	98,90
	Cross Auto (5608)	97,33	10,54	94,73	94,89	94,59
J48	No	94,86	82,82	100,00	26,60	100,00
	Cross Auto (5608)	96,22	11,37	96,46	96,53	96,38
Random Forest	No	74,03	64,76	100,00	37,80	100,00
	Cross Auto (5608)	97,95	10,39	96,45	96,52	96,39
IBk	No	-	-	-	-	-
	Cross Auto (-)	-	-	-	-	-

SMO	No	50,00	49,95	100,00	0,00	100,00
	Cross Auto (5608)	95,03	9,93	95,03	95,20	94,83
Promedios	No	79,07	1655,55	99,72	16,32	99,72
	Cross Auto (5608)	96,63	10,55	95,66	95,78	95,54
Mejora del balanceo		+17,56	-1645,00	-4,06	+79,46	-7,18

Tabla 12 - Impacto del balanceo en Small Twitter

Al utilizar el balanceo Cross Automático, la cantidad de datasets creados es proporcional al desbalance del dataset original. Analizando este parámetro, queda en evidencia la diferencia en el grado de desbalance entre el dataset de la red Small Twitter y el resto. Por su parte, el dataset Les Misérables presenta la mayor diferencia entre clases considerando sólo las tres redes restantes, presentando una cantidad de casos negativos cuatro veces mayor que la de positivos.

La primera métrica a analizar es el área bajo la curva. Como fue mencionado anteriormente, el AUC es independiente de la distribución de clases, por lo que, no se esperaban cambios significativos en los resultados para las clasificaciones balanceadas. Efectivamente, para las redes pequeñas, las diferencias entre los promedios de esta métrica con cada clasificador utilizado no supera el 1,08%.

Sin embargo, para el dataset de Small Twitter, se obtuvieron diferencias significativas, especialmente para el clasificador SMO. En la clasificación sin balanceo, este algoritmo se ve completamente sesgado por la cantidad de instancias negativas del dataset, clasificando a todas como tal, y obteniendo así un 50% de área bajo la curva, siendo este el peor resultado posible para dicha métrica. En cambio, con la clasificación balanceada, los datasets generados (que contienen la misma cantidad de instancias positivas y negativas) permitieron que el área bajo la curva promedio de todas las clasificaciones sea de 95,03%, un valor sumamente alto para esta métrica. El rendimiento general de AUC se visualiza en la Figura 27. En la misma los puntos oscuros redondos corresponden a las clasificaciones sin balanceo, y los claros cuadrados, considerando balanceo cross. La función celeste representa el aumento gradual del desbalance, ya que los experimentos se ordenaron de esa manera, para comparar la variación de las métricas a medida que aumenta el mismo.

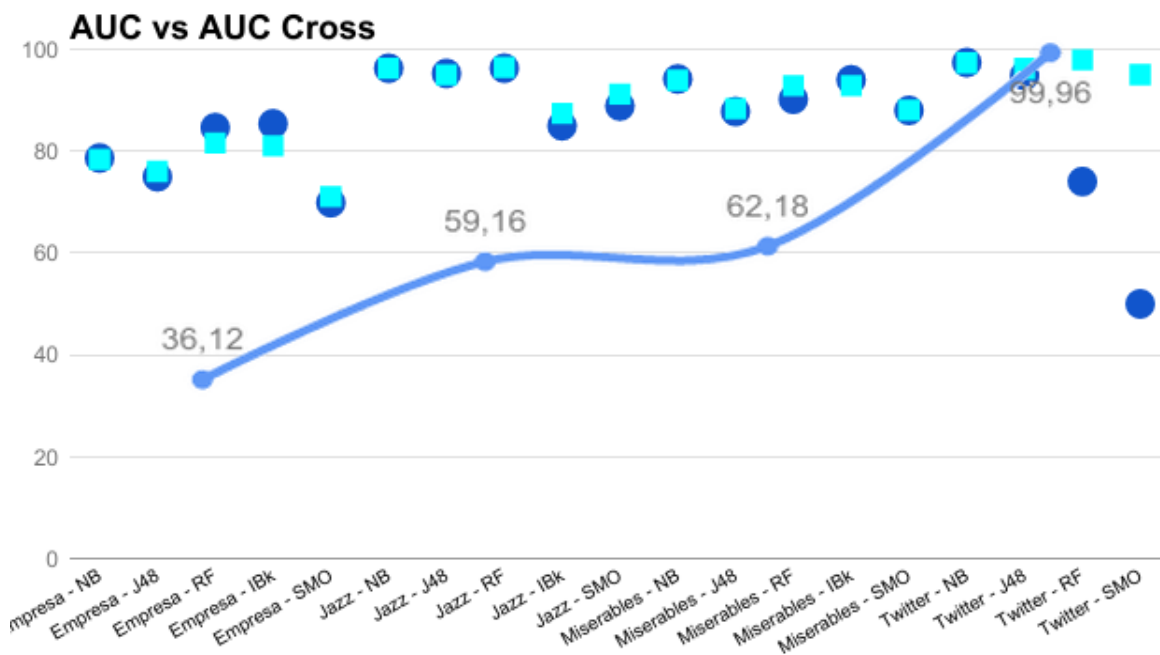


Figura 27 - Relación entre desbalance y AUC, con y sin balanceo

Como fue planteado en la propuesta, el objetivo principal del balanceo del dataset es disminuir las métricas de error de los resultados. Las redes pequeñas utilizadas no son un buen ejemplo para analizar este parámetro, ya que además de tener poca cantidad de instancias, tampoco presentan un desbalance notorio. Es por este motivo que, si bien hubo variaciones entre ambas pruebas, la diferencia del RAE entre ambas es pequeña. Para la red Jazz Musicians, que representa el dataset más grande de las tres redes pequeñas, se obtuvo una leve mejoría en todos los clasificadores, mientras que las redes más pequeñas de todas (Les Miserables y Contacto Empresa) tuvieron resultados levemente peores en las pruebas con balanceo para todos los algoritmos de clasificación.

El caso más importante a analizar para el RAE es el dataset Small Twitter, ya que al ser una red mucho más grande que las demás, presenta un desbalance mucho mayor. En esta red, todas las pruebas realizadas con balanceo obtuvieron un valor de RAE considerablemente menor que sin balanceo, el cual es uno de los objetivos principales que se esperaba que cumpliera el método. El caso donde más se refleja este hecho, es al utilizar el clasificador NaiveBayes, cuyo RAE sin balanceo es de 6400%, mientras que con balanceo es de sólo 10,5%. La causa de esto es que NaiveBayes sin balanceo clasifica aproximadamente 65000 instancias None como Link, y el resto de clasificadores sin balanceo realizan lo mismo, pero para sólo unas pocas decenas de instancias. Este hecho se observa perfectamente en las matrices de confusión de tales experimentos. De manera análoga a la anterior figura, se visualiza el rendimiento del RAE a medida que aumenta el desbalance en la Figura 28.

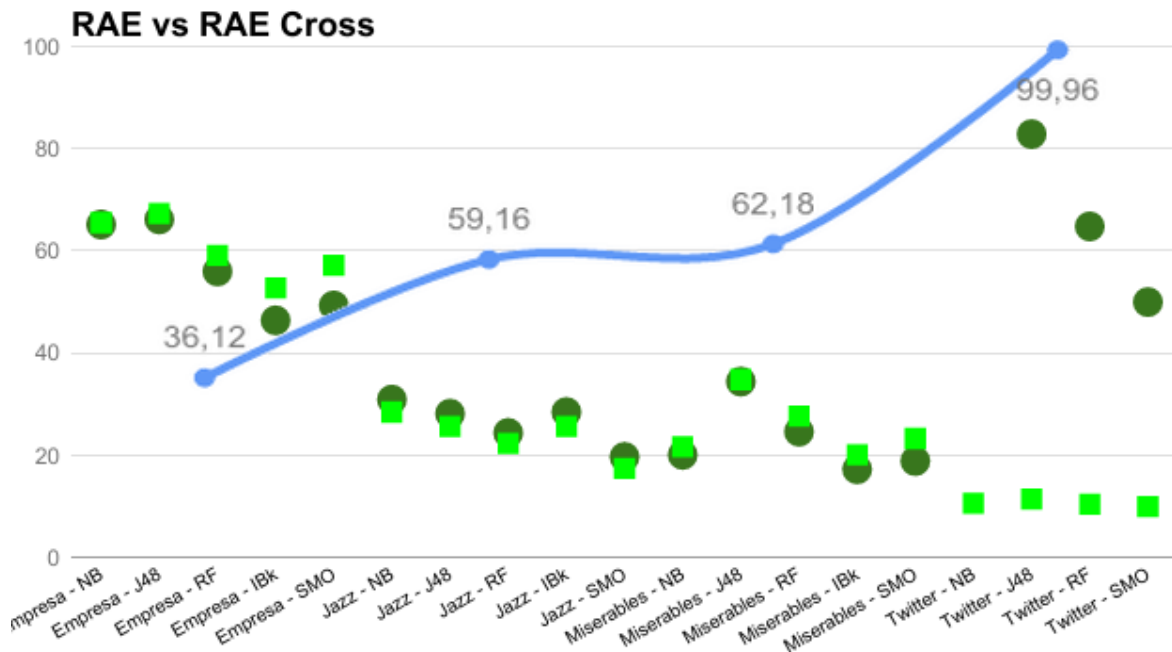


Figura 28 - Relación entre desbalance y RAE, con y sin balanceo

La última métrica a analizar es el F-Measure, considerando los casos positivos y negativos por separado. Los resultados obtenidos son un fiel reflejo del planteo previo a la experimentación. Para evaluaciones realizadas con balanceo, disminuyó el F-Measure ponderado y el F-Measure de los casos negativos, pero mejoró el resultado de los casos positivos, para las cuatro redes y para todos los clasificadores. Esta constante se debe a que, en las evaluaciones sin balancear, la precisión y recall ponderados están sumamente influenciados por la clase mayoritaria (en estos ejemplos, los casos negativos), y a su vez, los algoritmos tienden a clasificar mejor estos casos que los de la clase minoritaria ya que existen más instancias de la misma.

Las diferencias entre los valores promedios de ambas pruebas para el F-Measure ponderado, oscilan en todos los casos entre un 2% y un 5%, y los valores de la misma métrica para los casos negativos entre un 4% y un 11%. Para los casos positivos varía entre un 3% y un 7% en las redes Les Misérables, Jazz Musicians y Contactos Empresa.

Por su parte, la diferencia para los casos positivos en la red Small Twitter es de un 79,46%, marcando nuevamente una clara diferencia sobre el resto de las redes. Analizando los resultados de cada clasificador en particular para esta red, se observa que el F-Measure para los casos positivos en las evaluaciones sin balancear son de 0,9%, 26,6%, 37,8% y 0% para Naive Bayes, J48, Random Forest y SMO respectivamente. En cambio, los resultados luego de aplicar el balanceo, varían entre un 95% y 97% aproximadamente. Estos números reflejan que, si bien el área bajo la curva obtenida sin el balanceo es razonablemente aceptable, esta no implica que la precisión y recall para los casos positivos también lo sea. En la Figura 29 se observa el rendimiento del F-measure positivo a medida que aumenta el desbalance.

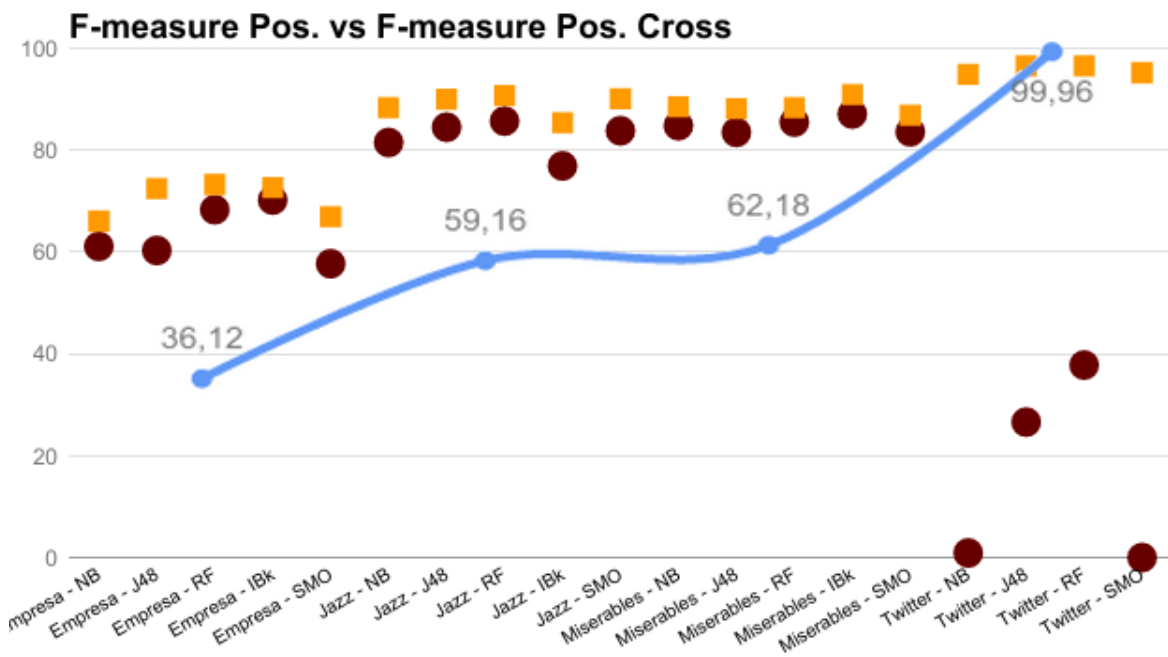


Figura 29 - Relación entre desbalance y F-measure Positivo, con y sin balanceo

Si bien la mejora en los casos positivos no logra mejorar el F-Measure ponderado en ninguno de los casos, utilizar un balanceo de clases para optimizar esta métrica sigue siendo sumamente válido, ya que en líneas generales permite mejorar la precisión y el recall en las predicciones de la clase minoritaria.

5.2.4. Aprendizaje supervisado vs. métricas de similitud individuales

En esta última etapa de experimentación se busca comparar las dos maneras de predicción de enlaces implementadas sobre Gephi. De esta forma, el objetivo es determinar a grandes rasgos, cuál manera es mejor dadas ciertas métricas finales, más allá del funcionamiento interno de cada tipo de predicción. Por un lado, se encuentra el aprendizaje supervisado, desarrollado y estudiado en detalle a lo largo del presente trabajo, y, por otro lado, las métricas de similitud individuales, abordadas en [Ricotta y Santamarina, 2016]. En resumen, la comparación busca determinar qué enfoque es mejor, desde distintos puntos de vista.

Se decidió realizar la comparación utilizando los resultados de los experimentos realizados en el trabajo de Ricotta y Santamarina sobre las redes Les Misérables, Jazz Musicians y Contactos Empresa. Se debe destacar que, el último dataset no posee la misma cantidad de experimentos que los dos anteriores, ya que su uso fue exclusivo para la prueba de las funciones de similitud que utilizan información de los nodos. Por ende, los datos que provengan de los experimentos que utilicen este último dataset no poseen el mismo peso que los demás, limitando de esa forma, el análisis sobre el mismo.

Para el caso de SmallTwitter, se realizaron una serie de pruebas utilizando las funciones de similitud para completar la comparación de todos los datasets. Sin

embargo, debido a problemas de procesamiento por la alta cantidad de nodos de esta red, se debieron tomar ciertas consideraciones. Se seleccionaron los 400 usuarios con más seguidores y seguidos a la vez (nodos de mayor grado), y se eligieron 100 usuarios pertenecientes a este grupo al azar, los cuáles se utilizan para realizar las predicciones. De esta forma es posible experimentar sobre la red en una cantidad de tiempo moderada utilizando un conjunto de datos más pequeño, pero relativamente representativo de la red.

Todas las redes poseen diferencias cruciales en su estructura como para realizar una comparación variada y completa. Como ya se ha observado, diferentes configuraciones y elementos de cada enfoque funcionan mejor o peor dependiendo del dataset que se utilice. Por lo tanto, como un enfoque puede llegar a funcionar mejor para ciertos datasets y peor para otros, es necesario un estudio individual por dataset, y finalmente, un estudio del rendimiento general de ambos enfoques.

Por otro lado, para las comparaciones se utilizaron los mejores tres experimentos de cada enfoque. Para el aprendizaje supervisado, esto quiere decir que se utilizan sólo los clasificadores que mejor se ajustan a cada dataset teniendo en cuenta las métricas finales, además de las técnicas de filtrado, selección de atributos y balanceo, que ya fueron consideradas con anterioridad. En cambio, desde el punto de vista de las funciones de similitud, se utilizan sólo aquellas que mejor funcionan, junto a la configuración necesaria. Así la comparación es más variada y permite conocer mejor las configuraciones que mejor funcionan para cada enfoque.

En [Ricotta y Santamarina, 2016] las únicas métricas que miden el desempeño de una predicción son recall (o exhaustividad), precisión (o eficiencia) y AUC. Por lo tanto, esas tres serán las únicas utilizadas para la comparación de enfoques, a pesar de que la clasificación posea muchas más métricas finales importantes, pero que, al no existir en el otro enfoque no es posible utilizarlas.

Es importante destacar que, los experimentos de la investigación de Ricotta y Santamarina que se utilicen en la comparación, no contarán con la técnica de preprocesamiento de clustering desarrollada en tal trabajo. Esto se decidió como consecuencia de que los experimentos no demostraron una eficacia contundente y general para todos los casos al utilizar clustering. La técnica sólo mejora ciertos casos puntuales, mientras que el resto quedaron invariantes o empeoraron. Además, tal técnica implica una modificación sobre el grafo existente, lo cual es incompatible con la creación de datasets del presente trabajo. Modificar los grafos implica recalcular los datasets, ya que las etiquetas de clase de las instancias cambiarían (enlaces y nodos que dejan de existir), como así también, los valores de los atributos predictores.

En las siguientes tablas se muestran las configuraciones utilizadas por enfoque por cada dataset, junto a los valores correspondientes a las métricas finales. En azul se encuentran marcados los mejores casos de cada enfoque, y en verde y rojo, las diferencias positivas o negativas del aprendizaje sobre las funciones individuales. De forma posterior se procede a realizar conclusiones sobre cada caso particular, y finalmente, una comparación de los enfoques en forma general.

En las tablas, la configuración para los experimentos posee dos formatos:

- Aprendizaje: Filtro / Selección de atributos / Balanceo / Clasificador
- Funciones: Predicciones / Aristas ocultas / Función

Enfoque	Configuración	Precisión	Recall	AUC
Aprendizaje	2 / Correlation AA-CN-RA / Cross / RandomForest	89,12	89,01	92,88
	2 / Wrapper CN-HP-RA / Cross / NaiveBayes	90,18	89,58	93,91
	2 / Wrapper AA-HP-RA / Cross / IBk	91,34	91,30	92,82
Funciones	1 / 10% / AA	82,92	73,78	97,35
	1 / 10% / Combine CN + SI	85,36	76,21	95,56
	1 / 10% / Combine AA + CN	85,36	78,04	96,30
Mejora del enfoque de aprendizaje (mejores resultados)		+5,98	+13,26	-3,48

Tabla 13 - Comparación de enfoques para Les Miserables

Enfoque	Configuración	Precisión	Recall	AUC
Aprendizaje	2 / Wrapper RA / Cross / NaiveBayes	90,21	90,15	96,28
	2 / Wrapper AA-CN-HD-HP-RA / Cross / J48	91,39	91,39	94,96
	2 / Correlation AA-HT-HP-RA-SC / Cross / RandomForest	91,89	91,88	96,42
Funciones	15 / 50% / PD 0,25	61,81	67,93	93,92
	10 / 35% / RA	62,24	68,07	96,11
	10 / 35% / Combine RA + SC	63,05	68,94	96,11
Mejora del enfoque de aprendizaje (mejores resultados)		+28,84	+22,94	+0,31

Tabla 14 - Comparación de enfoques para Jazz Musicians

Enfoque	Configuración	Precisión	Recall	AUC
Aprendizaje	2 / Wrapper AA-HP / Cross / NaiveBayes	72,78	71,43	78,40
	2 / Wrapper AA-CN-HP-RA / Cross / IBk	73,49	73,41	81,02
	2 / Correlation AA-HD-HP-LHN-NI-SC / Cross / RandomForest	74,21	74,21	81,57
Funciones	2 / 40% / VNI	20,10	22,33	46,47
	2 / 40% / NI	62,50	64,88	87,25
	-	-	-	-
Mejora del enfoque de aprendizaje (mejores resultados)		+11,71	+9,33	-5,68

Tabla 15 - Comparación de enfoques para Contactos Empresa

Enfoque	Configuración	Precisión	Recall	AUC
Aprendizaje	2 / No / Cross / NaiveBayes	94,91	94,74	97,33
	2 / No / Cross / J48	96,53	96,46	96,22
	2 / No / Cross / RandomForest	96,52	96,46	97,95
Funciones	10 / 20% / LHN	0,50	0,30	51,69
	50 / 30% / RA	0,60	0,76	51,04
	50 / 10% / RA	0,16	0,80	68,31
Mejora del enfoque de aprendizaje (mejores resultados)		+96,36	+95,66	+29,64

Tabla 16 - Comparación de enfoques para Small Twitter

En primer lugar, se compararon las técnicas de aprendizaje para la red Les Misérables como se visualiza en la Tabla 13. En la misma, los resultados correspondientes a cada enfoque no varían demasiado con respecto a los valores finales de las métricas. En cambio, al comparar los mejores resultados de ambos enfoques existe una diferencia notoria entre los valores de las métricas, especialmente precisión y recall.

En el caso de la primera métrica se debe a que, el aprendizaje supervisado es capaz de predecir mejor los casos positivos en su totalidad al disminuir los falsos positivos que el modelo predictor produce, o sea, se predicen menos enlaces que en realidad no existen. Para el recall, los clasificadores también disminuyen la cantidad de falsos negativos con respecto a las funciones de similitud, o sea, enlaces que el modelo predice como no existentes. En el caso de AUC la variación es menor al 5% y, por lo tanto, no se considera sumamente significativa. Esta pequeña variación es producto de que la cantidad de falsos en comparación a los verdaderos positivos aumenta un poco más rápidamente para los clasificadores. En resumen, la cantidad de falsas predicciones que producen los clasificadores es menor, y tal hecho se refleja en las métricas de precisión y recall. Se concluye que, para este dataset, el enfoque supervisado posee un mejor desempeño al predecir enlaces.

Otro punto importante a tener en cuenta son las funciones de similitud que mejor aprovechan las características de esta red. Como se vio en la sección anterior, la selección de atributos benefició a CN, AA, y RA principalmente, como atributos representativos de los datasets. Estas funciones de forma individual también obtuvieron buenos resultados, aunque el mejor es la combinación de CN y AA. En este enfoque, RA quedó relegado por las dos funciones anteriores, ya que tampoco la técnica de Combine al utilizar RA en conjunto a otras funciones produjo buenos resultados.

En general, ambos tipos de modelos predictivos coinciden al seleccionar el mismo conjunto de funciones. Sin embargo, cada enfoque posee mayor eficacia con distintas funciones de similitud, dependiendo que tan bien se ajustan a los modelos subyacentes. Cabe destacar que, utilizando la función Combine se pueden usar como máximo dos funciones de similitud de forma conjunta, mientras que el enfoque de aprendizaje puede aprovechar la información de todas las funciones a la vez, a pesar de que implique la necesidad de mayor procesamiento.

En segundo lugar, se analizó la red Jazz Musicians, cuyos experimentos se encuentran en la Tabla 14. En este caso, aunque los mejores valores son muy similares para los experimentos pertenecientes a un mismo enfoque, las diferencias entre diferentes enfoques son más marcadas que en el anterior dataset. La precisión y el recall son mayores para los experimentos pertenecientes al aprendizaje supervisado, con una mejora de casi 29% y 23% respectivamente. Por su parte, para el AUC no existe una diferencia significativa (menor al 1%). Aunque se mantiene la misma tendencia que en los anteriores experimentos, se debe resaltar que en este caso la mejora del aprendizaje supervisado frente a las funciones es más del doble que en el anterior dataset. La causa de esto se debe al mayor tamaño de la red Jazz, la cual posee casi el triple de nodos y más del décuplo de aristas que la anterior red. Se concluye que, los modelos clasificadores con el soporte de las técnicas desarrolladas en este trabajo, son capaces de manejar volúmenes de datos mayores con mejor eficacia que las funciones de similitud por sí solas. Las desventajas son un mayor costo computacional en tiempo y memoria, y la necesidad de ajustar una gran cantidad de parámetros y

componentes del proceso de clasificación. Sin embargo, esto último también refleja la mayor flexibilidad del enfoque supervisado para adaptarse a distintos tipos de casos de estudio de forma efectiva.

Con respecto a los atributos que mejor representan al dataset, ambos enfoques coinciden en seleccionar a RA como la función que mejor sirve para predecir enlaces, por su capacidad de utilizar la información de vecinos de vecinos y actuar de forma más efectiva en redes con alto grado medio. En cambio, para el resto de atributos las elecciones no coinciden demasiado. El enfoque supervisado considera que HP y CN funcionan bien, mientras que el otro enfoque, beneficia el uso de SC en combinación con RA, y PD con 0,25 de exponente, funcionando como un punto medio entre CN y SC.

En tercer lugar, se analizó la red Contactos Empresa. Como se explicó anteriormente, se debe remarcar que en [Ricotta y Santamarina, 2016] los experimentos sobre esta red se centraron en el uso de NI y VNI exclusivamente, y en consecuencia, el análisis sobre la misma es limitado. Los resultados de estos experimentos se encuentran en la Tabla 15.

La tendencia que se venía cumpliendo para los anteriores datasets, deja de cumplirse en este caso. A pesar que los experimentos del enfoque supervisado son bastante homogéneos en cuanto a las métricas finales, los correspondientes a las funciones individuales no lo son. Al comparar los enfoques, los clasificadores siguen teniendo un rendimiento superior en cuanto a precisión y recall, o sea, menor cantidad de falsos, con una mejora de alrededor de 10% para ambas métricas. Sin embargo, la disminución de AUC es mayor al 5%, lo cual implica que las funciones de similitud son ligeramente mejores en cuanto a mantener estable los verdaderos positivos. Esto puede deberse al pequeño tamaño del dataset (28 nodos), lo que implica una baja cantidad de instancias con las que entrenar a los modelos clasificadores. Mientras que, en el otro enfoque, con ajustar los parámetros de porcentaje de aristas ocultas y de cantidad de predicciones, el modelo de valores de similitud se ajusta fácilmente al tamaño del dataset.

No es posible determinar las funciones más importantes debido a la falta de experimentos sobre la red. Sólo se puede remarcar que los experimentos de las funciones individuales obtuvieron buenos resultados utilizando NI, y malos resultados con VNI, a causa de la falta de procesamiento sobre el texto que describe a los trabajadores de la empresa. Desde el punto de vista de los clasificadores, en sólo un experimento se incluyó a NI dentro de los atributos importantes, aunque en conjunto con otros predictores. De forma análoga, VNI tampoco fue tenido en cuenta en ninguna de las selecciones de atributos, debido a la misma causa descrita anteriormente.

Finalmente, se realizó la comparación sobre la red Small Twitter, cuyos mejores experimentos se encuentran resumidos en la Tabla 16. En este caso, los experimentos de las funciones individuales sobre la red se tuvieron que realizar en el marco de este trabajo.

Además de la baja eficiencia de la técnica de funciones individuales debido a las consideraciones que debieron tomarse, la eficacia de la misma también fue muy baja, lo cual se reflejó en las métricas finales. Sin importar la función de similitud, el porcentaje de aristas ocultas, o el número de predicciones que se elija, el enfoque de funciones individuales es totalmente inservible en esta red, funcionando de forma similar a una predicción al azar. La causa principal de este problema es el bajo grado medio de la red. Esto provoca que casi todas las predicciones positivas (de existencia de enlace) que se realicen mediante la similitud de los nodos estén erradas, debido a que, más del 99% de pares de nodos que se comparan no poseen enlace alguno entre sí.

Este mismo problema se refleja en el desbalanceo extremo que posee el dataset utilizado por el enfoque de aprendizaje. Sin embargo, y como se describió anteriormente, en este trabajo se crearon y utilizaron herramientas que controlan el problema de los datasets grandes. Para el desbalance se utilizaron las técnicas que permiten balancear los datasets, y de esta manera la falta de enlaces entre nodos no afecta a la eficacia final de las clasificaciones. Además, las técnicas de filtrado de instancias permiten disminuir el tamaño de los datos a procesar por los clasificadores, beneficiando en gran medida al rendimiento del proceso de clasificación entero. Debido a la falta de técnicas que controlen este tipo de problemas para las funciones de similitud individuales, este enfoque resulta inútil en redes de gran tamaño.

Una vez realizados los análisis de comparación sobre los datasets es posible deducir varias cuestiones generales. Primero se deben remarcar las configuraciones que mejor funcionan y las similitudes para ambos enfoques, independientemente de las redes. Un resumen del rendimiento general de las primeras tres redes es expuesto por la Figura 30, y en la Figura 31 para la red Small Twitter específicamente.

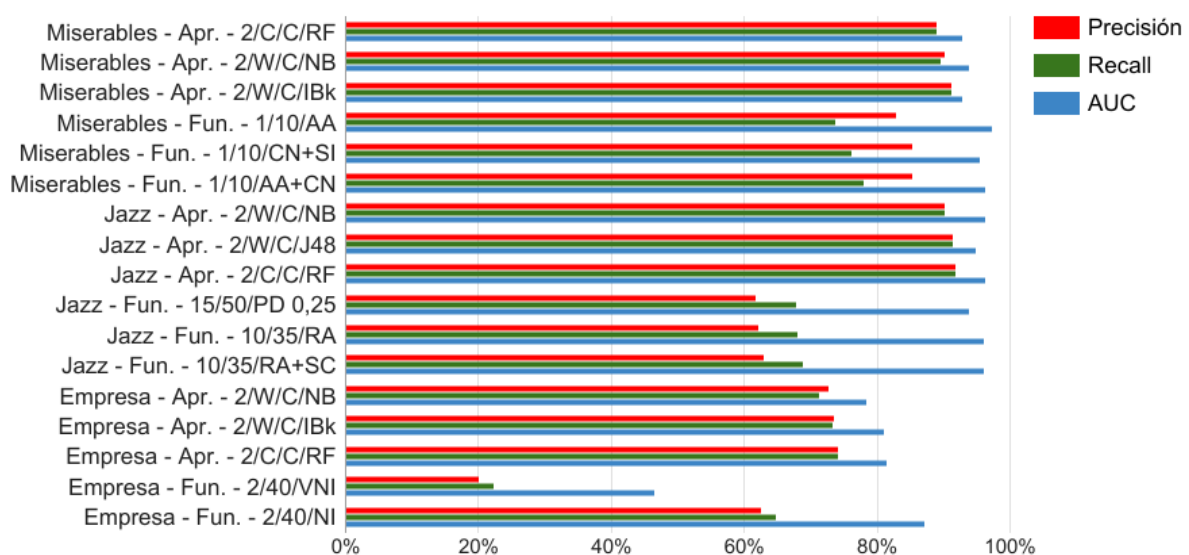


Figura 30 - Comparación general de enfoques para las tres primeras redes

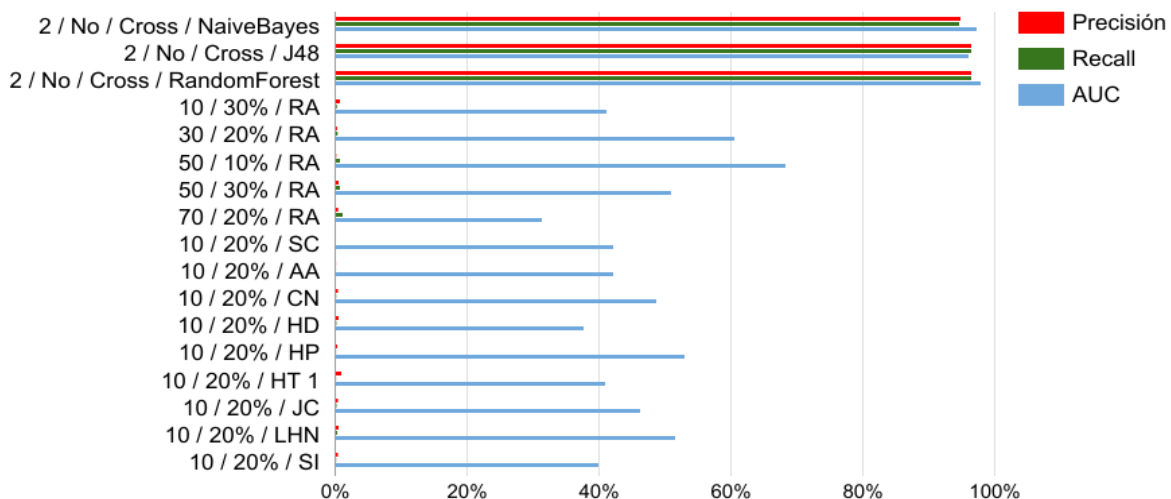


Figura 31 - Comparación de los enfoques para Small Twitter

En el aprendizaje supervisado, los clasificadores que mejor rendimiento obtuvieron fueron Naive Bayes y Random Forest, con J48 en segundo lugar, mientras que los atributos más seleccionados fueron las funciones AA, RA y HP. En cuanto a las funciones individuales que mejores resultados obtuvieron, pueden mencionarse AA y RA, aunque debe ajustarse finamente el porcentaje de aristas a ocultar y el número de predicciones a cada red, para obtener el máximo rendimiento de las mismas.

Sin embargo, la técnica Combine es la que mejora aún más el rendimiento de las funciones, ya que ciertas combinaciones de las mismas son mejores que cada una de forma individual. La superioridad de estas dos funciones se debe a, como ya se ha mencionado, su capacidad de utilizar la información de vecinos, obteniendo una visión más amplia de la red que las demás funciones topológicas locales.

Además, se comprobó que RA funciona con mayor efectividad en redes con mayor grado medio como Jazz Musicians, y que ambas son similares para redes pequeñas como Les Miserables y Contactos Empresa. En contraposición las funciones topológicas globales y de información de nodos no han tenido buena efectividad. Esto se atribuye a un conjunto de causantes como, por ejemplo, falta de preprocesamiento sobre los atributos de nodos, falta de poder de procesamiento que afecta al tiempo total de cálculo, entre otras.

En segundo lugar, es importante remarcar las diferencias entre ambos enfoques de predicción. El aprendizaje supervisado en general obtiene mejores resultados debido a múltiples causas. La primera es que es capaz de aprovechar varias características de una red a la vez para modelar el conocimiento y clasificar, mientras que las funciones sólo utilizan el criterio de similitud entre nodos (o dos si se utiliza Combine). La segunda, consiste en que, al existir múltiples modelos clasificadores, cada uno posee ciertas ventajas y desventajas que aprovechan de diferente manera las características, ofreciendo una flexibilidad que permite adaptar el proceso de clasificación a cada red de forma efectiva. La tercera y última causa es que, debido a las técnicas de filtrado, selección y balanceo, es posible resolver

ciertas desventajas inherentes a la clasificación, que podrían impactar de forma negativa en los resultados y el rendimiento general del proceso.

Por otro lado todos estos componentes que flexibilizan y mejoran el enfoque de aprendizaje, producen desventajas a tener en cuenta. Una desventaja tiene que ver con el rendimiento con respecto al tamaño de las redes a analizar. El uso de clasificadores necesita de mucho más tiempo y memoria para obtener los resultados finales, los cuales escalan con el tamaño de los datasets. Y aunque algunas de las técnicas permiten disminuir esos requisitos sacrificando efectividad, otras realizan lo contrario, por lo cual es importante encontrar un punto medio entre la eficiencia y eficacia. Sin embargo, para redes más grandes como Small Twitter, el enfoque de aprendizaje posee un rendimiento muy superior al de funciones de similitud debido a las técnicas de filtrado, selección y balanceo que controlan el problema de este tipo de redes, y que el enfoque de similitud no controla. En consecuencia, la desventaja de procesamiento del aprendizaje sólo se da en redes medianas o chicas donde las funciones de similitud poseen buena eficiencia.

Por otro lado, la gran cantidad de componentes y parámetros a configurar que hacen factible la flexibilidad del enfoque de aprendizaje, implican una gran cantidad de ajustes sobre los mismos. Como es difícil probar todas las combinaciones, se necesita seguir una serie de hipótesis que acoten la cantidad de casos a probar, y de esa manera, posibiliten la experimentación. Dado que tales hipótesis pueden no ser del todo ciertas, existe la chance de descartar combinaciones y configuraciones potencialmente mejores. En cambio, las funciones de similitud sólo necesitan ajustar el porcentaje de ocultación y el número de predicciones a la red, para posteriormente probar las distintas variantes. En consecuencia, es un enfoque de análisis mucho más directo, aunque eso implique sacrificar eficacia en las predicciones finales. Sin embargo, para analizar redes grandes o de bajo grado medio, la única opción útil es el de aprendizaje.

6. Conclusiones

En esta sección final se procede a realizar un análisis general sobre los distintos hechos desarrollados a lo largo del presente trabajo. Por un lado, se hace énfasis sobre los problemas, conclusiones y aportes del mismo al área de aprendizaje supervisado para la predicción de enlaces. Y, por otro lado, se proponen futuras posibles extensiones de este trabajo, así como las consecuencias de solucionar ciertas desventajas existentes, y de esa forma, ampliar el valor agregado de la herramienta desarrollada.

6.1. Aportes de este trabajo

A partir de la herramienta y los experimentos desarrollados sobre la misma, se pueden concluir varias cuestiones generales acerca de la predicción de enlaces. Dados los diferentes casos de estudio utilizados en la experimentación, es fácil deducir que las técnicas abordadas en el presente trabajo no están limitadas a las redes sociales online. De los casos de estudio, sólo Small Twitter corresponde a una red social online, mientras las demás se tratan de redes conceptuales que representan colaboraciones de músicos de jazz, relaciones laborales ficticias y relaciones de personajes de una novela. Por lo tanto, el enfoque utilizado en el presente trabajo es útil para cualquier tipo de problemática que sea posible de representar mediante un grafo, y donde la solución se refleje en la predicción de enlaces no existentes en el mismo.

Una de las desventajas de este tipo de análisis, tiene que ver con los grafos muy pequeños como Contactos Empresa. En estos casos el enfoque supervisado no posee la suficiente cantidad de instancias como para entrenar correctamente los modelos clasificadores, disminuyendo así la eficacia general de la clasificación. Otra de las desventajas tiene que ver con los grafos más grandes, como Small Twitter, ya que afectan negativamente al rendimiento total del proceso de clasificación, especialmente debido a que, el número de instancias totales crece exponencialmente con respecto al número de nodos. En este otro tipo de casos es muy importante sacar provecho de las distintas técnicas para hacer factible la clasificación del dataset, fundamentalmente del filtrado. Sin embargo, ciertos modelos clasificadores poseen un costo computacional muy elevado para las computadoras ordinarias debido a sus características y, por lo tanto, el uso de los mismos será limitado o nulo para redes más grandes, como ocurrió con IBk en la red Small Twitter.

Otro de los aspectos que posibilita el uso de este enfoque sobre cualquier tipo de red, es la flexibilidad de los distintos componentes que son parte de la clasificación. La creación de datasets, el filtrado de instancias, la selección de atributos, el balanceo y los distintos clasificadores, poseen diferentes alternativas y campos

configurables que flexibilizan el proceso entero. Esto permite buscar el modelo que mejor se ajusta a cada red particular, y a la vez, salvar en cierta medida varios problemas típicos de la clasificación: desbalance, sobreajuste, atributos irrelevantes, maldición de la dimensionalidad, entre otros. Sin embargo, esta flexibilidad posee la desventaja de crear un gran espacio de opciones y parámetros posibles dentro del proceso de clasificación. Por lo tanto, para encontrar una alternativa que funcione de manera efectiva, se deben realizar distintas pruebas variando los componentes a elegir. Esto se debe a que, el funcionamiento de cada una de estas combinaciones es sumamente dependiente de las características de la red sobre la cual se busca predecir enlaces, ya sea el tamaño, la información disponible, la topología, el grado medio, entre otras.

Existen ciertas alternativas del proceso de clasificación implementadas en este trabajo que se destacan por sobre las demás, ya sea porque mejoran la efectividad o el rendimiento de forma general.

En primer lugar, el filtrado de instancias por distancia es una buena manera de eliminar ejemplos innecesarios que no aportan información relevante, minimizar los datos faltantes, y en consecuencia disminuir el tamaño total del dataset.

En segundo lugar, la selección de atributos ofrece varias maneras de eliminar los atributos irrelevantes y redundantes, aumentando la eficacia en la mayoría de los casos, y la eficiencia en todos los casos, ya que los datasets son filtrados para mantener las características importantes disminuyendo su tamaño en el proceso.

En tercer lugar, el balanceo ofrece algunas maneras de eliminar el problema del desbalance característico de este tipo de datasets, donde la mayoría de instancias representan pares de nodos que no poseen enlace. Así, se mejora el resultado final de ciertas métricas, especialmente las relacionadas a los casos positivos donde existe enlace.

En cuarto y último lugar, es importante destacar los modelos clasificadores y atributos que mejor funcionaron en todos los experimentos de forma general. En cuanto a los atributos más elegidos, los mismos fueron AA, RA y HP, los primeros dos por su capacidad de utilizar la información de vecinos de vecinos, y el último por la topología de las redes utilizadas, donde ciertos nodos tienden a concentrar gran cantidad de enlaces. Por su parte los clasificadores que mejores resultados dieron en todas las redes fueron Naive Bayes, J48 y Random Forest. El primero se caracteriza por su simplicidad y rendimiento, el segundo por su fácil interpretación, y el tercero por su efectividad. Se debe remarcar además que al utilizar Random Forest, no se pudo usar el algoritmo de selección de atributos Wrapper por cuestiones de rendimiento, y a pesar de eso, el clasificador mantuvo una efectividad similar a los demás clasificadores. Esto es importante debido a que, Wrapper es considerado el algoritmo de selección de mayor calidad, ya que utiliza un esquema de aprendizaje y se configuró particularmente para maximizar el AUC.

Por último, es importante recalcar el desarrollo realizado sobre la herramienta Gephi. De manera previa a este trabajo, la misma contaba sólo con herramientas de

predicción de enlaces según varias funciones de similitud. Debido a los requerimientos necesarios que surgieron en la presente investigación, se agregó un conjunto de funcionalidades que dan soporte a la predicción de enlaces mediante aprendizaje supervisado. Este conjunto está compuesto por:

- Creación de datasets. Permite elegir las características, configurarlas y filtrar las instancias según la distancia de los nodos, para crear un archivo ARFF. Esto posibilita que sea utilizable por otros softwares de aprendizaje supervisado.
- Selección de atributos. Habilita el filtrado de los atributos menos importantes para disminuir el ruido, y mejorar la performance en general.
- Balanceo: Resuelve el problema del desbalance de datasets mejorando principalmente las métricas de la clase perjudicada.
- Clasificación: es la funcionalidad principal que permite predecir enlaces a partir del uso de múltiples modelos clasificadores configurables.

Además de las funcionalidades anteriormente mencionadas, la herramienta cuenta con una interfaz amigable al usuario final, pero manteniendo la mayor cantidad de opciones configurables posibles. Por lo tanto, es relativamente fácil adaptar el proceso de clasificación a las necesidades de cada experimento, sólo que deben configurarse los componentes para encontrar el modelo que mejor se ajuste.

Por otro lado, el desarrollo de la herramienta se realizó teniendo en cuenta la flexibilidad, para que cualquier posible extensión posea un bajo costo en tiempo y esfuerzo. La inclusión de nuevos clasificadores es bastante simple y sólo depende de las opciones de configuración que se le deseen proveer al usuario en la interfaz. Sin embargo, la inclusión de nuevos tipos de filtros, selecciones o balanceos necesita de un esfuerzo adicional debido a su protagonismo en el proceso de clasificación. Este tipo de cuestiones podrían ser resueltas en otros trabajos que sigan extendiendo Gephi con más herramientas para el uso de grafos, especialmente si utilizan el enfoque de aprendizaje.

6.2. Trabajos futuros

Con el continuo crecimiento de las redes sociales se espera que, a través del paso del tiempo, el interés sobre la predicción de enlaces siga aumentando a medida que se vayan desarrollando nuevas utilidades de la técnica en cuestión. Por otro lado, el enfoque de aprendizaje automático para la resolución de problemas también se encuentra en auge, y se utiliza en múltiples aplicaciones tales como conducción autónoma [Geiger et al., 2012], inteligencia artificial [Bottou, 2014], corrección de errores en computación cuántica [Mavadia et al., 2017], procesamiento de big data [Suthaharan, 2014], entre otros. Debido al crecimiento de ambos campos de

investigación se espera que, se realicen muchas más investigaciones con respecto a los temas tratados en el presente trabajo.

Por ejemplo, la herramienta puede ser fácilmente extendida con nuevas funcionalidades que se incorporen a ambos enfoques de predicción, o que inclusive pertenezcan a uno totalmente nuevo. En el caso del aprendizaje supervisado, las funcionalidades desarrolladas son totalmente extensibles. Como ya se explicó en la implementación, es fácil agregar nuevos clasificadores si se cumple el framework planteado. Como nuevas funcionalidades, sería interesante que se agreguen metaclasificadores capaces de utilizar cualquiera de los demás clasificadores, incluso a ellos mismos. También sería posible agregar nuevos filtros de datasets basados en otros criterios, más algoritmos de selección de atributos y algún otro tipo de balanceo, aunque en cualquiera de estos casos el esfuerzo necesario sería mayor.

Sin embargo, existen dos cuestiones que mejorarían mucho el estado actual de la herramienta. La primera es un refactorio de las clases de interfaz de usuario para aumentar la usabilidad y extensibilidad de la herramienta. Y la segunda cuestión es mejorar la performance del programa, especialmente del código que no involucra el uso de la API de Weka, como la creación de datasets o el balanceo. De esta manera la herramienta permitirá realizar más experimentos en menos tiempo, especialmente para redes grandes que representan casos reales, donde el rendimiento se encuentra verdaderamente comprometido.

Más allá de la herramienta Gephi, los trabajos futuros pueden enfocarse en experimentar con otros tipos de redes que cuenten con características diferentes, o que no representen redes sociales, sino redes de cualquier otro campo de aplicación. Así se pondría a prueba la eficacia de los modelos clasificadores y las técnicas con un abanico de posibilidades mayor, obteniendo nuevas conclusiones potencialmente útiles.

7. Referencias

[Salzberg, 1994] Steven L. Salzberg; C4.5: Programs for Machine Learning by J. Ross Quinlan; Morgan Kaufmann Publishers, Inc.; Vol. 16, Issue 3, pp. 235-240; 1993.

[Platt, 1998] John Platt; Fast Training of Support Vector Machines using Sequential Minimal Optimization; Advances in Kernel Methods – Support Vector Learning, ed. by B. Schölkopf and C. J. C. Burges and A. J. Smola, MIT Press; pp. 41-65; 1998.

[Wang et al., 2015] Peng Wang, BaoWen Xu, YuRong Wu, XiaoYu Zhou; Link prediction in social networks: the state-of-the-art.; Science China: Information Sciences; Vol, 58, Issue 1, pp. 1-38; 2015.

[Scellato et al., 2011] Salvatore Scellato, Anastasios Noulas, Cecilia Mascolo; Exploiting place features in link prediction on location-based social networks; Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining; pp. 1046-1054; 2011.

[Beladia y Vats, 2011] Neeral Beladia, Neera Vats; Influence Based Link Prediction Using Supervised Learning; 2011.

[Pavlov y Ryutaro, 2007] Milen Pavlov, Ryutaro Ichise; Finding experts by link prediction in co-authorship networks; Proceedings of the 2nd International Conference on Finding Experts on the Web with Semantics; Vol. 290, pp 42-55; 2007.

[Ricotta y Santamarina, 2016] Matías Ricotta y Esteban Santamarina; Predicción de enlaces en redes sociales modeladas por grafos; Tesis de grado, Facultad de Ciencias Exactas, Universidad Nacional del Centro de la Provincia de Buenos Aires; 2016.

[Lozares, 1996] Carlos Lozares; La teoría de redes sociales; Universitat Autònoma de Barcelona. Departament de Sociologia; Vol. 48, pp. 103-126; 1996.

[Freeman, 2006] Linton C. Freeman; The Development of Social Network Analysis; Vancouver: Empirical Press; 2006.

[Godsil y Royle, 2001] Chris Godsil, Gordon Royle; Algebraic Graph Theory; Graduate Texts in Mathematics, Springer New York; Vol. 207; 2001.

[Barabási, 2003] Albert-László Barabási; Linked: The New Science of Networks; Ed. Basic Books; 2003.

[Mitchell, 1997] Tom M. Mitchell; Machine Learning; McGraw Hill; 1997.

[Flach, 2012] Peter Flach; Machine Learning: The Art and Science of Algorithms that Make Sense of Data; Cambridge University Press; 2012.

[Zytkow y Rauch, 1999] Principles of Data Mining and Knowledge Discovery; Third European Conference, PKDD'99 Prague, Czech Republic; 1999.

[Arrieta y Mera, 2015] Jose Arrieta, Carlos Mera; Estudio Comparativo de Técnicas de Balanceo de Datos en el Aprendizaje de Múltiples Instancias; LACNEM2015 - Latin American Conference on Networking and Electronic Media, Medellín, Colombia; 2015.

[Pozzolo et al., 2015] Andrea Dal Pozzolo, Olivier Caelen, Gianluca Bontempi; Comparison of balancing techniques for unbalanced datasets; 2015.

[Guyon y Elisseeff, 2003] Isabelle Guyon, André Elisseeff; An Introduction to Variable and Feature Selection; pp. 1157-1182; 2003.

[James et al., 2013] Gareth James; Daniela Witten; Trevor Hastie; Robert Tibshirani; An Introduction to Statistical Learning; Springer Texts in Statistics; pp. 203-264; 2013.

[Nasa y Suman, 2012] Chitra Nasa, Suman; Evaluation of Different Classification Techniques for WEB Data; International Journal of Computer Applications (0975 – 8887); Vol. 52, No.9; 2012.

[Rennie et al., 2003] Rennie, J., Shih, L., Teevan, J., Karger, D.; Tackling the poor assumptions of Naive Bayes classifiers; Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge; 2003.

[Rish, 2001] Irina Rish; An empirical study of the naive Bayes classifier; Proceedings of IJCAI-2001 workshop on Empirical Methods in AI; pp. 41-46; 2001.

[Ho, 1995] Tin Kam Ho; Random Decision Forests; Proceedings of the 3rd International Conference on Document Analysis and Recognition, Montreal, QC, 14–16 August 1995; pp. 278–282; 1995.

[Altman, 1992] N. S. Altman; An introduction to kernel and nearest-neighbor nonparametric regression; The American Statistician; Vol. 46, No.3, pp. 175–185; 1992.

[Kotsiantis, 2007] S. B. Kotsiantis; Supervised Machine Learning: A Review of Classification Techniques; Informatica 31; pp. 249–268; 2007.

[Novak y Novotny, 2010] A. Novak, B. Novotny; Electrons in Matter; WDS'10 Proceedings of Contributed Papers: Part III – Physics (eds. J. Safrankova and J. Pavlu), Prague, Matfyzpress; pp. 134–137; 2010.

[Refaeilzadeh et al., 2008] Payam Refaeilzadeh, Lei Tang, Huan Lui; K-fold Cross-Validation; Arizona State University; 2008.

[Fawcett, 2006] Tom Fawcett; An Introduction to ROC Analysis; Pattern Recognition Letters; Vol. 27, Issue 8, pp. 861–874; 2006.

[Geiger et al., 2012] A. Geiger, P. Lenz, R. Urtasun; Are we ready for autonomous driving? The KITTI vision benchmark suite; 2012 IEEE Conference on Computer Vision and Pattern Recognition, Providence, RI; pp. 3354-3361; 2012.

[Bottou, 2014] León Bottou; From machine learning to machine reasoning; Journal Machine Learning; Vol. 94, Issue 2, pp. 133-149; 2014.

[Mavadia et al., 2017] Sandeep Mavadia, Virginia Frey, Jarrah Sastrawan, Stephen Dona, Michael J. Biercuk; Prediction and real-time compensation of qubit decoherence via machine learning; Nature Communications; 2017.

[Suthaharan, 2014] Shan Suthaharan; Big data classification: problems and challenges in network intrusion prediction with machine learning; SIGMETRICS Performance Evaluation Review; Vol. 41, Issue 4, pp. 70-73; 2014.